ZKProof Community Reference 日本語訳

Version 0.3 - 2022 年 7 月 17 日

原著: ZKProof / 編集: D. Benarroch, L. Brandão, M. Maller, E. Tromer

日本語訳版: 2026 年 4 月 27 日

翻訳版について

本 PDF は、アップロードされた **ZKProof Community Reference, Version 0.3, July 17, 2022** の日本語訳である。原文は ZKProof 標準化活動の成果物として公開され、Creative Commons Attribution 4.0 International (CC BY 4.0) の下で利用可能である。翻訳では、暗号学上の専門用語を日本語として自然に読めるようにしつつ、必要に応じて英語の原語・略語を併記した。固有名詞、論文題名、引用キー、数式、プロトコル記法、URL は原則として原文表記を保持した。

出典表示: ZKProof. ZKProof Community Reference. Version 0.3. Ed. by D. Benarroch, L. Brandão, M. Maller, and E. Tromer. Published by zkproof.org. July 2022. Updated versions at https://docs.zkproof.org/reference

ライセンス: Attribution 4.0 International (CC BY 4.0)

ZKProof Community Reference

Version 0.3

2022 年 7 月 17 日

本書は作成途上の文書である。

フィードバックおよび貢献を歓迎する。

最新版は https://zkproof.org で入手できる。

コメントは editors@zkproof.org へ送付されたい。

Attribution 4.0 International (CC BY 4.0)

要旨

ゼロ知識証明は、裏付けとなるデータの機密性を維持しながら数学的命題を証明することを可能にする。これは、幅広い応用においてプライバシーを高める暗号学的道具として機能し得るが、その有用性は、安全で、実用的で、相互運用可能な配備に依存する。本 **ZKProof Community Reference** は、ZKProof 標準化活動の成果物であり、ゼロ知識証明技術の開発のための参照資料として役立つことを意図している。本書は、コミュニティによる、コミュニティのための貢献から成立している。定義と理論に関する理論的側面に加え、実装と応用に関する実践的側面を扱う。

キーワード: 暗号学、相互運用性、プライバシー、セキュリティ、標準、ゼロ知識証明。

このバージョンについて

本書は **ZKProof Community Reference** のバージョン 0.3 である。これは、謝辞、バージョン履歴、および過去の ZKProof ワークショップの文書に記載されているように、多数の貢献の結果である。0.x バージョンである間、本書は未完成の草稿とみなされるべきであり、さらなる発展の基礎として機能する。将来の安定版に到達するには、コミュニティからの追加の改訂と実質的な貢献が必要である。

このバージョンの引用: ZKProof. ZKProof Community Reference. Version 0.3. Ed. by D. Benarroch, L. Brandão, M. Maller, and E. Tromer. Pub. by zkproof.org. July 2022. Updated versions at https://docs.zkproof.org/reference

本コミュニティ参照文書について

「ZKProof Community Reference」は、ゼロ知識証明 (ZKP) 暗号の普及を目指す ZKProof オープン・イニシアチブの範囲内で生まれた。これは、相互運用性とセキュリティに焦点を当て、ZKP スキームおよびプロトコルの実装に関する信頼できる仕様の発展を目指す、包摂的なコミュニティ主導のプロセスである。

ZKProof は毎年ワークショップを開催しており、世界的に著名な暗号研究者、実務家、産業界のリーダーが参加する。これらのイベントは、新しい提案を議論し、最先端プロジェクトをレビューし、参照資料を前進させるためのフォーラムである。本書はそのような経緯から生まれ、ZKP システムの理解と開発支援のためにコミュニティが構築する参照資料であることを意図している。

以下の項目は、本書の期待される開発プロセスに関する指針を示す。本書は、コミュニティからの、またコミュニティのための貢献に開かれている。

目的。 ZKProof Community Reference 文書を開発する目的は、ZKProof 憲章に掲げられた原則のもと、安全で、実用的で、相互運用可能なゼロ知識証明技術の開発のための参照資料を提供することである。

狙い。本書の狙いは、ZKProof ワークショップ中の協働プロセスで開発または議論された参照資料を統合することである。本書は、一般市民、メディア、産業界、開発者、暗号研究者を含む広い読者にとってアクセスしやすいものとなることを意図している。

範囲。本書は、その目的、すなわち安全で実用的かつ相互運用可能な技術の開発に関連する資料を扱うことを意図している。また、より高度な技術を理解しやすくするため、導入的な概念や研究について詳述することもできる。複数の選択肢から焦点を選ぶ場合、本書は比較上の利点、欠点、適用可能性を説明する根拠を含むべきである。ただし、本書は ZKP に関する網羅的なサーベイであることを意図しておらず、考え得るすべてのシナリオを扱う必要はない。

形式。アクセシビリティ目標を達成し、広い範囲を考慮するため、本書では以下を含めることを重視する。明確に定義された構造（章、節、小節など）、導入的説明（エグゼクティブ・サマリーおよび各章の導入など）、主要概念を説明する例、番号付きの推奨事項と要求事項、要約表、専門用語の用語集、提示された主張と結果に対する適切な参照。

編集方法。本コミュニティ参照文書の開発プロセスは、4 つの段階からなるサイクルで進むことが提案されている。

1. ZKProof ワークショップにおける公開討論、および以後の開発の参照となる注釈の作成。
2. 一連の貢献提案に従い、定められた期間中に、任意参加の貢献者によって内容を開発する。
3. 編集者によって貢献を文書へ統合する。
4. 文書の状態に関する公開フィードバックを受け、次のサイクルの開発の基礎として用いる。

編集者チームはこのプロセスを調整し、貢献とフィードバックの公開募集を通じて透明性を促進し、文書品質向上のために編集上の裁量を用い、変更点とその根拠を容易に特定できるようにする。

ZKProof 憲章

ZKProof Charter（Boston, May 10th and 11th 2018）。 ZKProof 標準化活動の目標は、産業界と学術界の専門家を結集することによって、ゼロ知識証明技術の利用を前進させることである。この活動の目標をさらに進めるため、以下の指導原則を定める。

- 本イニシアチブは、誰にでも開かれ、自由に利用できる文書を作成することを目的とする。
  - オープン・イニシアチブとして、ZKProof Standards Workshop から発行されるすべての内容は Creative Commons Attribution 4.0 International ライセンスの下に置かれる。
- 本技術、研究、コミュニティのすべての側面を包摂的に代表することを目指す。
- 可能な場合には合意形成を目指し、合意に至らなかった場合には対立する見解を適切に表現することを目標とする。
- オープン・イニシアチブとして、イベントにおけるすべての声が聞かれることを目指し、成果を産業界、メディア、一般市民に伝えることを望む。
  - イベント参加者は写真撮影または映像撮影されることがある。
  - ハッシュタグ #ZKProof を用いてツイート、ブログ投稿、共有を行うことを奨励する。公式 Twitter ハンドルは @ZKProof である。

詳細については contact@zkproof.org を参照されたい。

編集者注: Creative Commons ライセンスの要件は当初、第 1 回 ZKProof ワークショップの範囲内にあった。以下の節（知的財産に関する期待）は、その範囲を本 Community Reference 以降に広げるものである。

知的財産 - 開示およびライセンスに関する期待

ZKProof は、ゼロ知識証明の安全で相互運用可能な利用を促進することを目指すオープン・イニシアチブである。オープンな開発と広範な採用を促すため、オープンソース実装を備え、ロイヤルティ負担のある特許に妨げられない技術を促進することには価値がある。しかし、有用な技術の一部は特許請求の範囲に含まれる可能性がある。ZKProof は技術、研究、コミュニティを包摂的に代表することを目指すため、知的財産の開示および特許請求の扱いについて期待事項を設定することには価値がある。

ZKProof コミュニティのメンバーは、ZKProof 文書に含まれる指針、要求事項、推奨事項、提案、例に適用され得る既知の特許請求（自己のものおよび他者のもの）に関する情報を提供することを強く奨励される。これには、既知の係属中の特許出願または関連する存続中の特許の開示が含まれる。とりわけ、特許権者またはその代理として行動する者は、「Community Reference」への内容貢献およびコミュニティによる検討のため ZKProof に提出される「Proposals」への貢献の条件として、そのような開示を速やかに行うことが求められる。ZKProof 文書は、関連する特許請求に関して受領した開示に基づき更新される。

ZKProof は、誰にでも開かれ、自由に利用できる文書の作成を目指す。そのため、ZKProof 標準化活動の文脈で公開される内容は、Creative Commons Attribution 4.0 International ライセンスの下で利用可能にされるべきである。さらに、ZKProof 文書で推奨され、かつ特許請求の範囲に含まれる技術は、合理的で、不公正な差別がないことを明示できるライセンス条件の下で利用可能にされるべきであり、自由なオープンソース実装を可能にすることが望ましい。

関連情報は editors@zkproof.org へ送付されたい。

目次

本翻訳版では PDF ビューアのしおりおよび冒頭の目次機能を用いることを想定し、原文の章節構成を維持する。原文の主要構成は以下の通りである。

- エグゼクティブ・サマリー
- 第 1 章 セキュリティ
- 第 2 章 パラダイム
- 第 3 章 実装
- 第 4 章 応用
- 謝辞
- 参考文献
- 付録 A 略語および用語集
- 付録 B バージョン履歴

エグゼクティブ・サマリー

ゼロ知識証明 (ZKPs) は、暗号学に由来する重要なプライバシー強化ツールである。ZKP は、機密データに関連する命題の真実性を、その命題の妥当性以外の情報を明かすことなく証明できるようにする。ZKP は 1980 年代に学術コミュニティによって初めて開発され、その後大きく改良されてきた。現在では、産業界や開発者・研究者の大きなコミュニティにとって関心のある複数の領域で、実用的な適用が可能になっている。ZKP は、指定された私的データを第三者に開示することなく有用化するプライバシー保護型アプリケーションを可能にすることによって、産業、行政機関、個人利用に良い影響を及ぼし得る。

本参照文書の開発は、ZKP システムを理解し、その進展に影響を与え、関連製品を利用することに関心を持つ、より広いコミュニティに役立つことを目指している。これは、将来の標準の確立に先立ち、ZKP 技術のより広い採用を可能にする一歩である。ただし、本書は研究論文、技術書、標準を代替するものではない。本書は、導入的概念、基本技術、実装上の示唆、応用ユースケースに関する参照ハンドブックとして役立つことを意図している。

ZKP システムには少なくとも 2 つの当事者、すなわち証明者と検証者が関与する。証明者の目標は、追加情報を明らかにせずに、ある命題が真であることを検証者に納得させることである。たとえば、証明者が権限ある機関によりデジタル署名された出生証明書を保持しているとする。あるサービスにアクセスするために、証明者は 18 歳以上であること、すなわち、本人の識別情報に紐づき、信頼された認証機関によってデジタル署名され、年齢主張と整合する出生年月日を記載した証明書を知っていることを証明しなければならない場合がある。ZKP は、証明者が出生年月日を明らかにすることなく、これを可能にする。

本書は、ZKP のセキュリティ、パラダイム、実装、応用に関する現在の最先端の重要な側面を説明する。ZKP が価値を付加できるユースケースおよび応用は多数ある。これをよりよく評価するには、複数の指標の下で実装をベンチマークし、セキュリティと効率のトレードオフを評価し、相互運用性の基盤を発展させることが有用である。証明システムのセキュリティはシステム利用者にとって最重要であるが、効率もユーザー体験に不可欠である。

「セキュリティ」の章では、ZKP システムの基礎的な用語とセキュリティ概念を導入する。ZKP システムは、セットアップ、証明、検証という 3 つの構成要素で記述できる。セットアップは、秘密鍵や公開鍵、共通参照文字列など、秘密要素および共通要素を含む証明者と検証者の初期状態を決定する。証明と検証の構成要素は、それぞれ証明者と検証者のアルゴリズムであり、場合によっては対話的である。全体として、これらは完全性、健全性、ゼロ知識性という 3 つの主要なセキュリティ要求を満たす必要がある。

完全性は、証明と検証がともに正しく、かつ命題が真であるなら、対話の終了時に検証者がその事実を納得することを要求する。健全性は、悪意ある証明者でさえ、偽の命題を検証者に納得させられないことを要求する。ゼロ知識性は、悪意ある検証者でさえ、与えられた命題が真であること以外の情報を証明過程から抽出できないことを要求する。簡潔性、非対話性、移転可能性、合成可能性など、他の性質や変種も、望ましい特徴として設計に組み込まれ得る。

「パラダイム」の章では、一般命題、すなわち非決定性多項式時間 (NP) 関係に関するゼロ知識証明を構成するパラダイムの概観を提示する。説明は、理想的な構成要素に基づく情報理論的 (IT) システム、暗号要素が関与する暗号コンパイラ (CC)、および実際の証明を決定する公開データ（命題とインスタンス）と秘密データ（証人）を符号化する算術化システムの合成という現代的観点に基づく。この整理により、多くが簡潔な証明を可能にする現代的 ZKP システムの体系的な分類と特徴づけが可能になる。本章の将来の更新では、異なるアプローチに従い得る特殊な言語・関係の追加例が含まれるべきである。

「実装」の章では、ZKP 実装のためのフレームワークの策定に焦点を当てる。これは相互運用性にとって重要である。最初に考慮すべき重要な側面の 1 つは、命題の表現である。ZKP プロトコルでは、命題を数学的対象へ変換する必要がある。たとえば、年齢が 18 歳以上であることを証明する場合、その命題は、秘密の出生年月日 `Y1-M1-D1`（年-月-日）が現在日付 `Y2-M2-D2` とある関係を満たすこと、すなわち両者の距離が 18 年以上であることを証明することと等価である。この単純な例は条件の選言として表せる。`Y2 > Y1 + 18`、または `Y2 = Y1 + 18` かつ `M2 > M1`、または `Y2 = Y1 + 18` かつ `M2 = M1` かつ `D2 >= D1` である。ZKP に適した実際の変換、とくにより複雑な命題に対する変換は、実装上の課題となり得る。それでも、命題を回路などの数学的対象へ変換できる各種技術が存在する。本書では、Rank-1 Constraint System (R1CS) および Quadratic Arithmetic Program (QAP) に基づく表現に特に注意を払う。これらは現在利用されている複数の ZKP ソリューションで採用されている。また、本書は非対話型証明システムの実装を特に重視する。

ZKP が提供するプライバシー強化は、幅広いシナリオに適用できる。「応用」の章では、ZKP システムから利益を得られる 3 つのユースケース、すなわちアイデンティティ・フレームワーク、資産移転、規制遵守を提示する。プライバシー保護型アイデンティティ・フレームワークでは、たとえば年齢や居住州といった有用な個人属性を、出生年月日や住所などの詳細な個人データを明らかにせずに証明できる。資産移転の設定では、取引を促進する金融機関は通常、送信者と受信者の身元、資産種別、金額を知る必要があ

る。ZKP システムは、取引が匿名当事者間で行われる一方で、それらの当事者と資産が規制要件を満たすことを同時に保証する、プライバシー保護型の変種を可能にする。規制遵守の設定では、ZKP により監査者は、どのように達成されたかの詳細を知ることなく、プロセスが多数の要求を満たす証明を得ることができる。これらのユースケース、および想定可能な広範なプライバシー保護型アプリケーションは、コミットメント、署名、暗号化、回路などを含む共通の道具またはガジェットによって実現できる。

安全で実用的かつ相互運用可能な ZKP アプリケーションの開発には、セキュリティ概念と実装指針のバランスの取れた相互作用が必要である。ZKP 技術が提供する解決策は、慎重なセキュリティ実践と現実的な仮定によって保証されなければならない。本書は、これらの目標達成に役立つセキュリティ性質と実装技術を要約することを目指す。

第 1 章 セキュリティ

1.1 導入

1.1.1 ゼロ知識証明とは何か

ゼロ知識証明 (ZKP) は、秘密情報の機密性を保ちながら、ある命題が真であることを証明可能にする [GMR89]。これは、その命題の真実性がそれ自体では明白ではないが、証明者が証明を生成できる関連する秘密情報を知っている、または超計算能力のような能力を持つ場合に意味を持つ。ここで秘密という概念は、禁止された漏えいという意味で用いられる。ただし ZKP は、その「秘密」（またはその一部）が検証者に事前に知られている場合でも意味を持つ。

ZKP には多数の用途があり、機密データに関する主張を証明するために有用である。例として以下がある。

1. 出生年月日を明らかにせずに成人であることを証明する。
2. ポートフォリオ構成を示さずに支払能力がある（破産していない）ことを証明する。
3. 過去の取引を明らかにしたりリンクしたりせずに、資産の所有権を証明する。
4. 合法なチェスの手順を明らかにせずに、チェス盤面の構成が妥当であることを証明する。
5. 数学的証明を明らかにせずに、定理の正しさ（証明可能性）を証明する。

これらの主張の一部（証明者と検証者が共通に知っており、ここでは非形式的命題として記述されるもの）は、機密情報（**証人**、証明者に知られ、証明過程で漏えいしてはならない）との関連を支える基盤（**インスタンス**、これも証明者と検証者が共通に知る）を必要とする。たとえば、支払能力の証明（命題）は、暗号化され認証された銀行記録（インスタンス）に依拠し、対応する復号鍵と平文（証人）を検証者に漏えいしてはならない秘密として証明者が知っていることに依拠し得る。1.2 節の表 1.1 は、いくつかの例についてこれらの要素を区別している。具体的なインスタンス化では、例示された ZKP は、証人の知識に関するより形式的な命題によって指定される。

FF1.1 今後の図案提案: 証明者と検証者の相互作用を図示し、命題、インスタンス、証人、メッセージ送信、納得という証明の要素を示す。

ZKP システムとは、証明者が命題が真であることを検証者に納得させるために、証明者と検証者がどのように相互作用できるかの仕様である。証明システムは、完全性、健全性、ZK 性を満たさなければならない。

- **完全性:** 命題が真であり、証明者と検証者がともにプロトコルに従うなら、検証者は受理する。
- **健全性:** 命題が偽であり、検証者がプロトコルに従うなら、検証者は納得しない。
- **ゼロ知識性:** 命題が真であり、証明者がプロトコルに従うなら、検証者は証明者との相互作用から、命題が真であるという事実以外の機密情報を学ばない。

証明とアーギュメント。 ZKP の理論では、証明者と検証者の計算能力に関連して、proof（証明）と argument（アーギュメント）を区別する。証明は、計算能力に制限のない証明者に対しても健全である必要がある。一方、アーギュメントは、計算量的に制限された証明者（しばしば確率的多項式時間アルゴリズムとして定義される）に対してのみ健全性を保てばよい。簡潔性のため、以下では「証明」という語を証明とアーギュメントの双方を指すために用いる。ただし、理論的にはこの区別が重要となる状況がある。

1.1.2 ゼロ知識証明システム仕様に必要な事項

完全な証明システム仕様は、以下を含まなければならない。

1. 証明システムが扱うよう設計された命題の種類の正確な仕様。
2. 証明者と検証者が用いるアルゴリズムを含む構成の詳細。
3. 該当する場合、証明者と検証者が用いるセットアップの説明。
4. 証明システムが提供することを意図するセキュリティの正確な定義。
5. ZKP システムがセキュリティ目標を満たすことを証明するセキュリティ分析、およびセキュリティの根拠となる未証明の仮定の一覧。

ZKP システムに関する効率の主張は、公平かつ正確に報告されなければならない。意図された利用に関連する性能パラメータを含むべきであり、他の ZKP システムと比較する場合には、同じ条件同士を比較するための最善の努力に基づくべきである。

1.2 用語

インスタンス: 証明者 (P) と検証者 (V) の双方に共通に知られる入力であり、何を証明する必要があるかという命題を支えるために用いられる。この共通入力は、証明者・検証者間の相互作用に局所的なものでも

よいし、外部当事者にも知られるという意味で公開であってもよい。記法: x。一部の科学論文では「インスタンス」と「命題」を同義に用いるが、本書では両者を区別する。

証人: 証明者への秘密入力。他者が証人について何かを知っている場合もあれば、知らない場合もある。記法: w。

関係: インスタンスと証人の間の関係の仕様。関係は許容されるペア (instance, witness) の集合として見ることができる。記法: R。

言語: R において許容されるペアとして現れるインスタンスの集合。記法: L。

命題: インスタンスと関係によって定義される。インスタンスがその関係における証人を持つことを主張する（真または偽）。記法: x ∈ L。

セキュリティ・パラメータ: 望ましいセキュリティ水準（たとえば 128 または 256）を示す正整数。セキュリティ・パラメータが大きいほど安全性が高い。多くの構成では、計算量的セキュリティ・パラメータと統計的セキュリティ・パラメータを区別する。記法: k（計算量的）または s（統計的）。

セットアップ: インスタンス x と証人 w を除いて、証明者および検証者に与えられる入力。各当事者のセットアップは、秘密要素（それぞれ相手に知られない PrivateSetup_P または PrivateSetup_V）と共通要素 CommonSetup = CRS（両者に知られる）に分解できる。ここで CRS は「共通参照文字列」を表し、一部のゼロ知識証明システムで必要となる。記法: setup_P = (PrivateSetup_P, CRS) および setup_V = (PrivateSetup_V, CRS)。

簡潔性のため、セキュリティ・パラメータや、言語・関係を定義する補助要素など、セットアップの一部パラメータは暗黙に扱う（場合によっては CRS 内に含める）。詳細は 1.5.3 節を参照。証人 w とインスタンス x は、具体的な ZKP プロトコル実行のセットアップ要素として仮定することもできるが、しばしばそれ自体のカテゴリとして区別される。実務上、「セットアップ」という語は、証明システムに関するセットアップを指して用いられることが多く、その後、変化するインスタンス x と証人 w による複数の実行にインスタンス化され得る。

表 1.1: ゼロ知識証明の例示シナリオ

#	シナリオ	証明される命題	基盤として用いられるインスタンス	機密として扱われる証人
1	購入のための法定年齢	私は成人である	改ざん耐性のある識別チップ	出生年月日および個人データ（認証機関により署名済み）
2	ヘッジファンドの支払能力	我々は破産していない	暗号化され認証された銀行記録	ポートフォリオデータおよび復号鍵
3	資産移転	私はこの資産を所有している	ブロックチェーンまたは他のコミットメント	取引列（および所有権を確立する秘密鍵）
4	チェス盤面構成	この構成は到達可能である	チェスのルール	有効なチェスの手の列
5	定理の妥当性	この式は定理である	公理集合および論理的推論規則	論理的含意の列

1.3 ZK のための命題指定

本書では、インスタンス x と証人 w の間の関係 R によって定義される命題の種類を考える。関係 R は、どのペア (x, w) が互いに関連しているとみなされ、どのペアが関連していないとみなされるかを指定する。この関係は、R 内に証人 w を持つインスタンス x からなる対応する言語 L を定義する。

命題は、「x ∈ L」という形のメンバーシップ主張、または「関係 R の範囲で、私はインスタンス x の証人を知っている」という形の知識主張のいずれかである。場合によっては、知識型とメンバーシップ型の命題は非形式的には交換可能とみなせる。しかし形式的には、両概念を区別すべき技術的理由がある。とくに、知識の命題をメンバーシップの命題へ変換できないシナリオ、およびその逆のシナリオが存在する（1.4 節で例示）。本書の例は、しばしば知識の命題に基づく。

関係 R は、たとえばプログラム（C や Java など）として指定できる。このプログラムは入力 x と w が与えられると受理し、これは (x, w) ∈ R を意味する。または拒否し、これは w が x ∈ L の証人ではないこと

を意味する。このような関係指定の例は、応用トラックで詳述される。学術文献では、関係はしばしばランダムアクセスメモリ (RAM) プログラム、または以下で説明するブール回路および算術回路として指定される。

1.3.1 回路表現

回路は、ノードとノードラベルからなる有向非巡回グラフ (DAG) であり、次のように構成される。

- **入力ノード:** 入次数 0 のノード。定数（例: 0, 1, …）または入力変数名（例: v1, v2, …）でラベル付けされる。
- **出力ノード:** 出次数 0 の単一ノード。
- **ゲート:** 計算を記述する内部ノード。

パラメータ。応用によっては、回路内のゲート数、インスタンス変数数 n_x、証人変数数 n_w、回路深さ、回路幅など、さまざまなパラメータが重要となる。

ブール回路充足可能性。関係 R は x = (C, v1, ..., v_{n_x}) の形のインスタンスと、w = (w1, ..., w_{n_w}) の形の証人を持つ。(x, w) が関係に属するには、C がファンイン 2 のゲートノードを持つ回路であり、それらが XOR や AND などのブール演算でラベル付けされていなければならない。v1, ..., v_{n_x} は一部の入力ノードの真理値を指定し、w1, ..., w_{n_w} は残りの入力変数の真理値を指定する。回路を評価したとき、出力ノードが 1（真）になる必要がある。

算術回路充足可能性。関係は x = (F, C, v1, ..., v_{n_x}) の形のインスタンスと、w = (w1, ..., w_{n_w}) の形の証人を持つ。(x, w) が関係に属するには、F が有限体（例: 素数 p を法とする整数）であり、C が体演算、すなわち加算または乗算でラベル付けされたゲートノードを持つ回路でなければならない。v1, ..., v_{n_x} は一部の入力ノードの体要素を指定し、w1, ..., w_{n_w} は残りの入力変数の体要素を指定する。回路を評価したとき、出力ノードが 1 になる必要がある。

FF1.2 今後の図案提案: 変数を含む、回路のさまざまな構成要素を図示する。

1.3.2 R1CS 表現

Rank-1 Constraint System (R1CS) は、ある体の要素からなるベクトル三つ組 (a, b, c) のリストによって表される方程式系である。各三つ組は (A) · (B) - (C) = 0 の形の方程式として「制約」を定義する。この方程式における 3 つの要素 (A), (B), (C) は、それぞれいわゆる解ベクトル s の変数 s_i の線形結合（例: (C) = c1 · s1 + c2 · s2 + ...）である。

R1CS 充足可能性。 R1CS 内のすべてのベクトル三つ組 (a, b, c) について、解ベクトル s は <a, s> · <b, s> - <c, s> = 0 を満たさなければならない。ここで <·, ·> は 2 つのベクトルの内積を表す。s の第 1 要素は変数ではなく定数 1 に固定され、制約中の定数を符号化できるようにする。残りの要素は、複数種類の変数を表す。

- **証人変数:** 証明者だけが知る。制約系への外部入力を表し、ZK 証明システムの証人に対応する。
- **内部変数:** 証明者だけが知る。制約系内部の変数であり、乗算ゲートの入力および出力を表す。
- **インスタンス変数:** 証明者と検証者の双方に知られる。

R1CS は、回路のように入力から出力を生成するものではないが、（論理ゲートおよび／または算術ゲートを持つ回路などによって実行された）計算の正しさを検証するために用いることができる。R1CS は、出力変数（通常は証明者と検証者の双方に知られる）が、解ベクトル中の他のすべての変数（場合によっては証明者だけが知る）と整合していることを確認する。R1CS は中間表現にすぎない。ZKP システムで実際に用いるには、秘密変数を明らかにせず検証を可能にするため、QAP などの後続の定式化が必要となる。

R1CS は、ブール回路充足可能性問題を表現するためにも、算術回路内の計算を検証するためにも使用できる。任意の回路は多項式の乗算と線形結合を用いて表現でき、これらが R1CS 制約に対応することを見れば十分である。たとえば以下の通りである。

FF1.3 今後の図案提案: R1CS を図示する。

- ブール回路演算:
  - NOT 演算: x がブール変数なら、1 - x は x の否定である。言い換えると、x が 0 または 1 なら、1 - x はそれぞれ 1 または 0 である。
  - AND 演算: (A) × (B) として実装できる。
  - XOR 演算 (c = a XOR b): (2 · a) × (b) = (a + b - c) として実装できる。あるいは c = a + b - (a AND b) * 2 と等価に実装できる。
- 算術回路演算:

- 乗算ゲートは a * b = c の形の方程式として直接表される。
- 線形制約は、これらのゲート間の入力と出力の追跡、および加算や定数倍を表すために用いられる。

1.3.3 関係の種類

特殊目的の関係。回路充足可能性は非決定性多項式時間 (NP) クラスにおける完全問題、すなわち NP 完全であるが、関係が必ずしもそうである必要はない。暗号用途に現れる命題の例として、コミットされた値が特定の範囲 [A; B] に入る、または集合 S に属すること、暗号文の平文が 0 であること、2 つの暗号文が同じ値を暗号化していること、証明者が署名方式の公開検証鍵集合に関連する秘密鍵を持つこと、などがある。

セットアップ依存の関係。ときには、関係 R に追加入力 setup_R を取らせる、すなわち関係が三つ組 (setup_R, x, w) を含むようにするのが便利である。入力 setup_R は持続的情報の指定に用いることができる。たとえば算術回路充足可能性において、同じ有限体 F と回路 C が何度も使われるなら、setup_R = (F, C)、x = (v1, ..., v_{n_x}) とできる。入力 setup_R は、関係が検査しない信頼された入力、たとえば信頼された Rivest-Shamir-Adleman (RSA) 法のモジュラスを捉えるためにも使用できる。

1.4 知識の ZKP とメンバーシップの ZKP

ZKP の理論では、命題の種類（およびセキュリティ性質の種類。1.6.2 節および 1.6.3 節を参照）に基づいて、2 種類の証明を区別する。

- **知識の ZKP (ZKPoK)** は、知識の命題の真実性を証明する。すなわち、命題を支える秘密データの知識を、その秘密データを明らかにせずに証明する。
- **メンバーシップの ZKP** は、メンバーシップの命題の真実性を証明する。すなわち、命題に関連する言語にインスタンスが属することを、計算量的に制限された検証者が生成できなかった情報を明らかにせずに証明する。

表 1.1 で例示された命題は事実として表現されていたが、それぞれ、インスタンスの文脈において命題を支える秘密証人の知識に対応する。たとえばシナリオ 1 の「私は成人である」という命題は、「私は、今日時点で成人であることと整合する出生年月日を知っており、かつ、その出生年月日を私の身元と関連付ける、信頼された認証機関によって署名された証明書を知っている」という省略形として解釈できる。

表 1.1 の最初の 3 つのユースケース（成人性、支払能力、資産所有）は、物理的アクセス制御、暗号化、署名、コミットメントなど、何らかの保護を含むインスタンスを持つ。「チェス盤面構成」と「定理の妥当性」のユースケースは異なり、それらのインスタンスは暗号的支援や物理的保護を含まない。これら 2 つの命題はいずれも、式または構成がそれぞれ、有効なチェス盤面構成の言語（すなわち手の列によって到達可能）または定理の言語（すなわち証明可能な式）に属する、というメンバーシップ主張として見ることができる。同時に、命題をさらに指定すると、合法手の列または論理的含意の列を知っているという知識主張として表現できる。

1.4.1 例: 離散対数の知識の ZKP

離散対数の知識を証明する古典的な例 [Sch90] を考える。p を p = 2q + 1 の形の大きな素数（例: 4096 ビット）とし、q も素数であるとする。g を、p を法とする乗算の下で群 Z_p^* = {1, ..., p - 1} = {g^i : i = 1, ..., p - 1} の生成元とする。この群において離散対数を計算することは計算量的に実行不可能であり、p と q の素数性は証明者と検証者の双方によって検証済みであると仮定する。w を証明者が知る秘密要素（証人）とし、x = g^w (mod p) を証明者と検証者の双方に知られるインスタンスとする。これは証明者による「私はインスタンス x の（底 g の）離散対数を p を法として知っている」という命題、言い換えれば「私は生成元 g をインスタンス x へ写す秘密指数を p を法として知っている」という命題に対応する。

ここで関係 R = {(x, w) : g^w = x (mod p)} を考える。この場合、対応する言語 L = {x : ∃w : (x, w) ∈ R} は単に集合 Z_p^* = {1, 2, ..., p - 1} であり、これは w の知識なしに自明にメンバーシップを判定できる。その意味で、メンバーシップの証明は意味を持たない（または空ビット列でさえ自明に達成されたとみなせる）。これに対し、証明者が証人を知っているかどうかは非自明な問題である。現在公に知られている最先端技術では、量子コンピュータを用いる場合を除き、素数モジュラスのサイズに対して多項式時間で離散対数を計算する方法は提供されていない。要するに、これは ZKPoK は意味を持つが、メンバーシップの ZKP は意味を持たない例である。

1.4.2 例: ハッシュ原像の知識の ZKP

二値入力長 512 に制限された暗号学的ハッシュ関数 H : {0,1}^512 -> {0,1}^256 を考える。この H の定義では、すべての 256 ビット列の集合が余域であり、これは H の像 L = {H(x) : x ∈ {0,1}^512}（範囲とも呼ばれる）の上位集合であり得る。証明者が知り、検証者にとって予測不能な証人（ハッシュ原像）を w とし、証明者が検証者に提示する何らかのインスタンスを x = H(w) とする。暗号学的ハッシュ関数は一方向性を持つため、原像の ZKPoK を提供することには意味がある。これは関係 R = {(x, w) : H(w) = x} における証人の知識を証明する。この証明は、言語 L、すなわち x が H の像のメンバーであることのメンバーシップ証明にも直接なる。しかし興味深いことに、H の既知の性質によって、このメンバーシップ述語はインスタンス x から自明である場合も、自明でない場合もある。

- H の像がすべての 256 ビット列の集合であること、すなわち L = {0,1}^256 であることが知られている場合、メンバーシップは自明である。この場合、256 ビット列であるという性質の検証は自明なので、メンバーシップの ZKP は冗長である。
- そうではなく、余域 {0,1}^256 から一様に選んだ要素 x が H の像に属さない確率が無視できない（たとえば H をランダム関数としてモデル化すれば約 0.368）という性質を H が持ち、かつメンバーシップ述語を判定することが困難である場合がある。この設定では、メンバーシップの ZKP を実行できることが有用となり得る。

1.4.3 例: グラフ非同型性に対するメンバーシップの ZKP

超多項式的な計算能力を持つ証明者（たとえば無制限の証明者）という理論的文脈では、証人という概念なしにメンバーシップの証明を考えることができる。したがって、この場合、知識の ZKP という双対的概念は適用されない。古典的な例は非同型グラフのペアの言語 [GMW91] を用いるものであり、2 つのグラフが同型でないことを検証者に納得させることが証明の対象である。この古典例は、証人からではなく、グラフ間の同型性を判定する証明者の超能力から生じる対話型証明を用いる。検証者は、元の 2 つのグラフのいずれかのランダム置換と同型なグラフが、どちらの元グラフに由来するかを検出するよう証明者に挑戦する。証明者が十分な回数、失敗することなく正しく判断すれば、検証者は非同型性を納得する。

本書は、漸近的設定において超多項式能力を持つ証明者を必要とする設定に焦点を当てない。しかし、証人なしのメンバーシップの ZKP という概念は、他の想定可能な応用、とくに（漸近的ではなく）具体的設定においても意味を持つ。これは、proof of work の文脈、あるいは証明者が「スーパーコンピュータ」または量子コンピュータであり、計算資源が大幅に少ない検証者と相互作用する文脈に適用され得る。別の想定可能な設定は、証明者が初めて効率的な技術を発見したと主張する数学的問題、たとえばグラフ同型性を高速に判定できる能力を、検証者が確認したい場合である。

FF1.4 今後の図案提案: グラフ非同型性の ZKP プロトコルを図示する。

1.5 構文

（言語 L を定義する関係 R に対する）証明システムは、証明者と検証者が互いにメッセージを送るプロトコルである。証明者と検証者は、ここでは Prove と Verify と呼ぶ 2 つのアルゴリズムによって定義される。Prove と Verify は確率的であってよく、呼び出し間で内部状態を保持してもよい。

1.5.1 Prove(state, m) -> (state, p)

ある状態にある Prove アルゴリズムは、メッセージ m を受け取り、状態を更新し、メッセージ p を返す。

- Prove の初期状態は、インスタンス x と証人 w を含まなければならない。初期状態は追加のセットアップ情報 setup_P を含んでもよい。例: state = (setup_P, x, w)。
- 初回呼び出し時に特別な初期化メッセージ m = start を受け取った場合、それは証明者がプロトコルを開始することを意味する。
- Prove が特別なエラー記号 p = error を出力した場合、その後のすべての呼び出しでも error を出力しなければならない。

1.5.2 Verify(state, p) -> (state, m)

ある状態にある Verify アルゴリズムは、メッセージ p を受け取り、状態を更新し、メッセージ m を返す。

- Verify の初期状態はインスタンス x を含まなければならない。
- Verify の初期状態は追加のセットアップ情報 setup_V を含んでもよい。例: state = (setup_V, x)。

- 特別な初期化メッセージ p = start を受け取った場合、それは検証者がプロトコルを開始することを意味する。
- Verify が特別な記号 m = accept を出力した場合、検証者が命題 x ∈ L の証明を受理したことを意味する。この場合、Verify は将来のすべての呼び出しでも m = accept を返さなければならない。
- Verify が特別な記号 m = reject を出力した場合、検証者が命題 x ∈ L の証明を拒否したことを意味する。この場合、Verify は将来のすべての呼び出しでも m = reject を返さなければならない。

セットアップ情報 setup_P と setup_V は多くの形を取り得る。暗号文献でよく見られる例は、setup_P = setup_V = k であり、ここで k は証明システムの望ましいセキュリティ水準を示すセキュリティ・パラメータである。setup_P と setup_V が、証明システムの具体化に用いるプリミティブの特定の選択、たとえば SHA-256 ハッシュ関数の使用や特定の楕円曲線の使用の記述を含むことも考えられる。セットアップ情報は確率的過程によって生成されてもよい。たとえば、setup_P と setup_V が共通参照文字列を含む場合や、指定検証者証明の場合に setup_P と setup_V が特定の仕方で相関している場合がある。この過程を特に参照したいとき、本書では確率的セットアップ・アルゴリズム Setup を用いる。

1.5.3 Setup(params) -> (setup_R, setup_P, setup_V, aux_i)

入力。 Setup アルゴリズムは、次のような入力パラメータ params を取ることがある。

- 望ましいセキュリティ水準を示す計算量的または統計的セキュリティ・パラメータ。
- 証明システムが扱うべき命題サイズを指定するパラメータ。
- SHA-256 ハッシュ関数や楕円曲線などの暗号プリミティブの選択。

出力。セットアップ・アルゴリズムは以下を返す。

- setup_R: 証明システムが対象とする関係への入力。重要な特殊例は、setup_R が空文字列であり、関係がいかなるセットアップにも依存しない場合である。
- 証明者用の setup_P と検証者用の setup_V。
- aux_i: 任意の追加補助出力。

入力が不正な形式である場合、またはエラーが発生した場合、Setup アルゴリズムはエラー記号を出力してもよい。

可能なセットアップの例:

- トラップドア置換に基づく一様参照文字列モデルにおける 3SAT 用 NIZK 証明システム。
  - setup_R = n。ここで n は節の最大数を指定する。
  - setup_P = setup_V = ある関数 size(n,k) による長さ N = size(n,k) の一様ランダム文字列。
- pairing-product equations に対する Groth-Sahai 証明。
  - setup_R = 言語を定義する双線形群の記述。
  - setup_P = setup_V = setup_R 内の双線形群の記述と追加の群要素を含む共通参照文字列。
- Pinocchio など、QAP に対する SNARK。
  - setup_R = 有限体 F と多項式を含む QAP 仕様。
  - setup_P = setup_V = 同じ有限体上で定義された双線形群といくつかの群要素を含む共通参照文字列。
  - 証明者と検証者は、共通参照文字列中の同じ群要素を使うわけではない。効率上の理由から、証明者が使う群要素の部分集合を setup_P とし、検証者が使う別の（はるかに小さい）部分集合を setup_V としてもよい。
- Cramer-Shoup ハッシュ証明システム。
  - setup_R = 素数位数の有限巡回群を指定する。
  - setup_P = 巡回群といくつかの群要素。
  - setup_V = 巡回群といくつかの離散対数。

Setup がどのように実行されるかは具体的設定に依存する。場合によっては、信頼された第三者がアルゴリズムを実行してセットアップを生成する。他の場合には、Setup は、一定数の腐敗した不正直な当事者に対する耐性を提供するマルチパーティ計算であり得る（補助出力は、敵対的当事者が MPC プロトコルから得る副情報を表すことがある）。さらに別の可能性はプレーン・モデルであり、そこではセットアップはセキュリティ・パラメータをコピーするだけである。例: setup_P = setup_V = k。

マルチ証明者証明システムや commit-and-prove システムなど、証明システムには変種がある。本書では標準的なシステムのみを扱う。

共通参照文字列。セットアップ情報が公開され、すべての人に知られる場合、その証明システムは共通参照文字列モデルにあるという。たとえばセットアップが setup_R = setup_P = setup_V を指定する場合、それを共通参照文字列 CRS と呼ぶ。

非対話型証明システム。相互作用が証明者から検証者への単一メッセージのみで構成される場合、証明システムは非対話型である。証明者のメッセージ π（証明と呼ばれる）を受け取った後、検証者は受理または拒否を返す。

公開検証可能性と指定検証者。非対話型証明システムにおいて、setup_V が公開情報（たとえば CRS モデル）であり複数の当事者に知られている場合、彼らはすべて証明 p を検証できる。この場合、証明は移転可能であり、証明者は一度だけ証明を作れば、その後それを多数の検証者へコピーして転送できる。他方、setup_V が秘密である場合、それを指定検証者証明システムと呼ぶ。

公開コイン。対話型証明システムにおいて、検証者のメッセージが一様ランダムで、証明者のメッセージから独立である場合、そのシステムを公開コインであるという。

1.6 定義と性質

関係 R に対する証明システム (Setup, Prove, Verify) は完全かつ健全でなければならない。さらに、知識の証明であることやゼロ知識であることなど、望ましい追加のセキュリティ性質を持ち得る。

1.6.1 完全性

直観的には、有効な証人 w を持つ正直な証明者が命題 x ∈ L について、命題が真であることを正直な検証者に納得させられるなら、証明システムは完全である。証明システムの完全な仕様は、この直観を捉える完全性の正確な定義を含まなければならない。以下では、証明者が開始する証明システムについての定義例を示す。

次の実験における完全性攻撃者 Adversary を考える。

1. Setup(params) -> (setup_R, setup_P, setup_V, aux_i) を実行する。
2. 攻撃者に最悪ケースのインスタンスと証人を選ばせる。Adversary(params, setup_R, setup_P, setup_V, aux_i) -> (x, w)。
3. 証明者が error を返すか、検証者が受理または拒否するまで、Prove と Verify の相互作用を実行する。プロトコルが終了しない場合は result = error とする。<Prove(setup_P, x, w, start); Verify(setup_V, x)> -> result。
4. (setup_R, x, w) ∈ R かつ result が accept でない場合、攻撃者は勝つ。

攻撃者の優位性をパラメータの関数として Advantage(params) = Pr[Adversary wins] と定義する。

params 上で実行される R に対する証明システムは、有意な優位性を持つ効率的攻撃者を誰も構成できない場合、完全である。

何が効率的攻撃者であるか（計算設備、実行時間、メモリ消費、利用期間、インセンティブなど）、どの程度の優位性を許容できるかは応用に依存する。特別に強い場合として、任意の攻撃者の勝率が小さい統計的完全性（無条件完全性とも呼ばれる）と、任意の攻撃者について優位性が正確に 0 である完全完全性がある。

1.6.2 健全性

直観的には、不正な証明者が偽の命題を真であると正直な検証者に納得させる可能性がほとんどない、または全くない場合、証明システムは健全である。証明システムの完全な仕様は、この直観を捉える健全性の正確な定義を含まなければならない。以下に定義例を示す。

次の実験における健全性攻撃者 Adversary を考える。

1. Setup(params) -> (setup_R, setup_P, setup_V, aux_i) を実行する。
2. （状態を持つ）攻撃者にインスタンスを選ばせる。Adversary(params, setup_R, setup_P, setup_V, aux_i) -> x。
3. 攻撃者を検証者と相互作用させ、検証者の出力を result とする（プロトコルが終了しない場合は result = reject とする）。<Adversary; Verify(setup_V, x)> -> result。
4. (setup_R, x) ∉ L かつ result が accept である場合、攻撃者は勝つ。

攻撃者の優位性をパラメータの関数として Advantage(params) = Pr[Adversary wins] と定義する。

params 上で実行される R に対する証明システムは、有意な優位性を持つ効率的攻撃者を誰も構成できない場合、健全である。

FF1.5 今後の図案提案: 健全性ゲームの図示。

何が効率的攻撃者であるか（計算設備、実行時間、メモリ消費、利用期間など）、どの程度の優位性を許容できるかは応用に依存する。特別に強い健全性概念として、任意の攻撃者の勝つ確率が小さい統計的健全性（無条件健全性とも呼ばれる）と、任意の攻撃者について優位性が正確に 0 である完全健全性がある。

1.6.3 知識の証明

直観的には、証明システムが健全であるだけでなく、正直な検証者を納得させることが証明者が証人を「知っている」ことを意味するなら、その証明システムは知識の証明である。証明者による証人の「知識」は、理想的なシミュレーション世界において、成功した証明者から証人を抽出できる能力として定義できる。ある証明システムが知識の証明であると主張されるなら、その ZKP プロトコルのセキュリティ分析は、この直観を捉える知識健全性の正確な定義を含まなければならない。

貢献募集: ここには、知識の証明という形式的概念に必要な抽出器のゲーム定義を含めるべきである。このセキュリティ性質は、理想世界で証明者から証人を直接受け取る理想 ZKP 機能の文脈における、理想／現実シミュレーション・パラダイムでも自然に現れる。

FF1.6 今後の図案提案: PoK のゲーム、および横に並べた PoK の理想機能。

1.6.4 ゼロ知識

直観的には、証明システムが、攻撃者が他の情報源から証人について既に知っていることを超える情報を、証明者の証人について漏えいしないなら、その証明システムはゼロ知識である。ゼロ知識性は、証人へのアクセスなしに正規のものに見える証明を生成できる効率的シミュレータの仕様によって定義される。証明システムがゼロ知識であると主張される場合、完全な仕様は、この直観を捉えるゼロ知識の正確な定義を含まなければならない。以下に定義例を示す。

FF1.7 今後の図案提案: 検証者と相互作用するシミュレータを図示する。

設計者が追加の効率的アルゴリズム `SimSetup` と `SimProve` を提供し、現実的な攻撃者が以下のゲームにおいて小さい優位性しか持たない場合、証明システムはゼロ知識である。攻撃者 `Adversary` を次の実験で考える。

1. ビット `b` を `0,1` から一様ランダムに選ぶ。
2. `b = 0` なら、`Setup(params) -> (setup_R, setup_P, setup_V, aux_i)` を実行する。
3. `b = 1` なら、`SimSetup(params) -> (setup_R, setup_P, setup_V, aux_i, trapdoor)` を実行する。
4. (状態を持つ)攻撃者にインスタンスと証人を選ばせる。`Adversary(params, setup_R, setup_P, setup_V, aux_i) -> (x, w)`。
5. `(setup_R, x, w) ∉ R` なら `guess = 0` を返す。
6. `b = 0` なら、攻撃者を証明者と相互作用させ、推測を出力させる（プロトコルが終了しない場合は `guess = 0` とする）。`<Prove(setup_P, x, w); Adversary> -> guess`。
7. `b = 1` なら、攻撃者をシミュレートされた証明者と相互作用させ、推測を出力させる（プロトコルが終了しない場合は `guess = 0` とする）。`<SimProve(setup_P, x, trapdoor); Adversary> -> guess`。
8. `guess = b` なら攻撃者は勝つ。

攻撃者の優位性をパラメータの関数として `Advantage(params) = |Pr[Adversary wins] - 1/2|` と定義する。

特別に強い概念として、任意の攻撃者が小さい優位性しか持たない統計的 ZK（無条件 ZK とも呼ばれる）と、任意の攻撃者について優位性が正確に 0 である完全 ZK がある。

`params` 上で実行される `R` に対する証明システムは、有意な優位性を持つ効率的攻撃者が存在しない場合、ZK である。効率的攻撃者の概念（計算設備、実行時間、メモリ消費、利用期間など）および許容できる優位性の大きさは応用に依存する。

検証者の知識からの独立性。証明の ZK 性は、検証者が証明されている証人について何かを知っているかどうかには依存しない。ただし、ある証明は、検証者も証人（または何らかの有効な証人）を知っている特定の場合に合わせて設計されることがあり、その場合、より効率的な証明生成（ある対話を通じて）または検証が可能になることがある。

多定理ゼロ知識。ゼロ知識の定義では、攻撃者は単一インスタンス上で証明者またはシミュレータと相互作用する。複数インスタンスの証明を見る攻撃者に対しても保護するよう、ゼロ知識定義を強化することが可能である。

正直な検証者に対するゼロ知識。より弱いプライバシー概念として、正直な検証者に対するゼロ知識 (HVZK) がある。ここでは攻撃者がプロトコルに正直に従う、すなわち定義の手順 6 および 7 で検証アル

ゴリズムを実行する。よくある設計手法は、まず HVZK 証明システムを構成し、その後、効率的な標準変換を用いて完全なゼロ知識を持つ証明システムを得ることである。

証人識別不能性と証人隠蔽。ときにはゼロ知識より弱いプライバシー概念で十分な場合がある。証人識別不能な証明システムでは、複数の可能な証人のうち証明者がどれを持つかを攻撃者が識別することが実行不可能である。証人隠蔽型の証明システムは、正直な証明者との相互作用が攻撃者による証人計算を助けないことを保証する。

1.6.5 高度なセキュリティ性質

文献には、証明システムが持ち得る多くの高度なセキュリティ概念が記述されている。これには、並行合成下のセキュリティ、中間者攻撃に対する非可鍛性、敵対者が物理アクセスを持つ設定におけるリセット攻撃への耐性、洗練されたセキュリティ証明を支援するシミュレーション健全性およびシミュレーション抽出可能性、ならびにユニバーサル合成可能性が含まれる。

ユニバーサル合成可能性。 UC フレームワークは、任意の環境において理想機能を実現するなら、プロトコルは安全であると定義する。理想的なゼロ知識機能は、証明者から入力 (x, w) を受け取り、かつそのときに限り (x, w) ∈ R であれば検証者にメッセージ (x, accept) を送るものとして考えられる。この理想機能は、有効な証人を持たない命題が受理されることはないため完全に健全であり、証明が単なる accept メッセージであるため完全にゼロ知識である。証明システムは、実世界でのシステム実行が理想証明システム機能の実行と「セキュリティ上等価」であるなら、UC 安全である。通常、証明システムが UC 安全であることを示すにはより多くの作業が必要だが、その一方で、このフレームワークは証明システムが他の暗号プロトコルと合成されるときに強いセキュリティ保証を提供する。

1.6.6 移転可能性と否認可能性

ゼロ知識の伝統的概念では、ZKP システムは、検証者が正当な証明実行に参加したことを説得力を持って宣伝することすらできないようにする。言い換えると、検証者は証明された命題について得た確信を他者へ移転できない。この性質は、しばしば否認可能性または非移転可能性と呼ばれる。なぜなら、正当な証明者として証明に参加した証明者は、元の検証者がトランスクリプトを公開したとしても、後にその参加をもっともらしく否認できるからである。

否認可能性はしばしば望ましい性質であるが、その反対である移転可能性、すなわち公開検証可能性も特徴として考えられ得る。公開検証可能な ZKP の正当な実行における検証者は、対応する証明済み命題が真であること（または知識の命題の場合、何らかの証明者が実際に主張された知識を有していたこと）を外部当事者に納得させることができる。移転可能性は、場合によっては相互作用のトランスクリプト（交換メッセージおよび検証者の内部状態からなる検証者のビュー）を単に共有することで達成できる。

推奨事項 1.1 - 否認可能性と移転可能性を特徴づける。 ZKP を用いる応用の適切なセキュリティ分析のためには、その ZKP が否認可能性と移転可能性のどちら（またはそれらの微妙な変種）を持つ必要があるかを特徴づけることが重要である。これは、他の応用との合成時のセキュリティ／プライバシーを評価するために関連する。

1.6.7 セットアップと信頼の例

セキュリティ定義は信頼されたセットアップを仮定する。セットアップがどのような形を取るか、そこにどの程度の信頼が置かれるかにはいくつかの変種がある。

- **セットアップなし、または信頼不要セットアップ**。セットアップが単にセキュリティ・パラメータ k のコピーである場合や、誰もがセットアップが正しいことを直接検証できる場合など、信頼が不要な場合。
- **一様ランダム文字列**。すべての当事者が一様ランダム文字列 URS = setup_R = setup_P = setup_V にアクセスできる。より軽い信頼の場合、当事者は単に一様にサンプルされた文字列を得る必要があり、それはたとえば太陽黒点活動のような信頼された共通乱数源から得られるかもしれない。より強い信頼の場合、ゼロ知識性はシミュレーション・トラップドアとともに URS 生成をシミュレートできる能力に依拠する。
- **共通参照文字列**。 URS モデルは CRS モデルの特殊例である。しかし CRS モデルでは、共通セットアップが非一様分布からサンプルされることもあり、その場合、信頼された共通乱数源への容易なアクセスは排除され得る。CRS が検証可能な構造を持つ、すなわち整形式であることを容易に検証できるか、それとも完全な信頼が必要かを区別できる。
- **指定検証者セットアップ**。セットアップが相関した setup_P と setup_V を生成し、setup_V が指定検証者専用である場合、セットアップ・アルゴリズムに関する信頼が必要である。これはたとえば

Cramer-Shoup 公開鍵暗号において、選択暗号文攻撃下のセキュリティを提供するために指定検証者 NIZK 証明が用いられる場合に当てはまる。この場合、セットアップは鍵生成過程の一部として生成され、受信者は暗号方式を安全にすることが自身の利益であるため、これを正直に行うと信頼できる。

- **ランダムオラクル・モデル**。共通セットアップは SHA-256 などの暗号学的ハッシュ関数を記述する。ランダムオラクル・モデルでは、ハッシュ関数は、まだ見ていない入力に対する問い合わせにランダム値を返すランダムオラクルのように振る舞うとヒューリスティックに仮定される。現実のハッシュ関数が決定的関数であるという事実を悪用してランダムオラクル・モデルが失敗する理論的例はあるが、実務上このヒューリスティックは良い効率を与え、現在のところ「自然な」証明システムについて既知の弱点はない。
- **閾値安全性および／またはサブバージョン耐性**。 CRS 生成に安全なマルチパーティ計算を用いる、多数の CRS があり安全性がその過半数の正直な生成にのみ依拠するマルチストリング・モデルを用いる、悪意ある生成 CRS に対してもゼロ知識性が成立するサブバージョン耐性 CRS を用いるなど、セットアップへの信頼を減らすための複数の提案がある。

1.7 仮定

証明システムの完全な仕様は、そのシステムがセキュリティ定義を満たすための仮定を述べ、その仮定が主張されたセキュリティ性質を含意することを示さなければならない。

セキュリティ分析は、数学的な還元証明の形を取り得る。これは、あるセキュリティ性質に対して有意な優位性を得る現実的な敵対者が存在すると、基礎となる仮定の 1 つに対して有意な優位性を得る現実的な敵対者を構成できることを示すものである。

例として、健全性が衝突困難ハッシュ関数に依拠すると仮定する。この事実の証明は、健全性攻撃者 `Adversary` を数回サブルーチンとして呼び出す、単純で効率的なアルゴリズム `Reduction` を記述する形を取り得る。さらに、この証明は、`Reduction` が衝突を見つける優位性が、任意の `Adversary` が健全性に対して持つ優位性と密接に関連していることを確立し得る。たとえば、`Advantage_soundness(params) <= 8 × Advantage_collision(params)` のようにである。

証明システムが、パラメータの一部として SHA-256 ハッシュ関数で具体化できるよう設計されているとする。予算 `\$1,000,000` の攻撃者が 5 年以内に SHA-256 の衝突を見つけるリスクが `2^-128` 未満であると仮定すれば、この還元は、同程度の能力を持つ敵対者が健全性を破るリスクが `2^-125` 未満であることを示す。

暗号学的仮定。暗号学的仮定、すなわち困難性仮定は、証明システム設計者が現実的攻撃者には計算できないと信じるものを指定する。あるセキュリティ性質が暗号学的仮定に全く依拠しない場合があり、その場合そのセキュリティは無条件であるという。たとえば、ある証明システムが無条件健全性または無条件ゼロ知識性を持つと言うことがある。しかし通常、健全性またはゼロ知識性のいずれかは困難性仮定に基づく。仮定の選択は、設計者のリスク許容度と、防御したい敵対者の種類に依存する。

もっともらしさ。破られた困難性仮定は用いられるべきではない。基礎となる暗号学的仮定が偽であることが示された場合に容易に更新できるよう、柔軟でモジュール的な証明システムを設計することを推奨する。

仮定のもっともらしさの順序を確立できる場合もあれば、できない場合もある。たとえば、ある群で離散対数問題を破ることができれば、同じ群で計算 Diffie-Hellman 問題も破れることが知られているが、その逆は必ずしも成り立たない。これは、離散対数仮定の方が計算 Diffie-Hellman 仮定よりもっともらしく、したがってセキュリティの観点から望ましいことを意味する。

ポスト量子耐性。今後数十年以内に量子コンピュータが開発される可能性がある。量子コンピュータは、離散対数問題など、いくつかの暗号学的仮定を効率的に破ることができる。証明システムの期待寿命が量子コンピュータの出現を超えるなら、量子コンピュータに耐えると考えられる困難性仮定に依拠する必要がある。異なるセキュリティ性質は異なる寿命を必要とし得る。たとえば、証明は直ちに検証されるためポスト量子健全性は不要である一方、攻撃者が証明トランスクリプトを収集・保存し、後でそこから何かを学ぼうとする可能性があるため、ポスト量子ゼロ知識性は必要である、という場合があり得る。

具体的パラメータ。暗号文献では、証明システムの背後にある理論を説明する際、「多項式時間敵対者の優位性は無視可能である」といった曖昧な表現を用いることが一般的である。しかし、現実世界での配備には具体的で精密なセキュリティが必要である。したがって証明システムは、具体的なパラメータ推奨と、それらが提供すると信じられるセキュリティ水準に関する記述を備えるべきである。

システム仮定。暗号学的仮定に加え、証明システムは、それが動作する機器または環境に関する仮定に依拠し得る。例として、証明システムが信頼されたセットアップに依拠するなら、どのような信頼が置かれ

るかを明確に述べるべきである。

セットアップ。証明者または検証者が確率的である場合、乱数を生成するためのエントロピー源が必要である。擬似乱数生成の不具合は他の種類の暗号システムに脆弱性を引き起こしてきたため、証明システムの完全な仕様は、そのエントロピー源の性質または品質についてどのような仮定を置くかを明示すべきである。

1.8 効率

証明システムの仕様は効率に関する主張を含み得る。その場合、測定単位は明確に述べられなければならない。関連する指標には以下が含まれ得る。

- **ラウンド複雑性:** 証明者と検証者の間の送信回数。通常、1 つの当事者から他方へのメッセージを 1 move として、move 数で測定される。重要な特殊例は、1 move 証明システム、すなわち非対話型証明システムである。そこでは検証者が証明者から証明を受け取り、受理するかどうかを直接判断する。非対話型証明は移転可能であり得る。すなわち、コピー、転送され、複数の検証者を納得させるために使用できる。
- **通信量:** 証明者と検証者の間で送信される総ビット数。
- **証明者計算量:** プロトコル期間中に証明者が費やす計算労力。システム依存性を避けるため支配的な暗号演算の数で測定される場合もあり、具体測定を行う場合には特定システム上の秒数で測定される場合もある。
- 意図された利用に応じて、メモリ消費、エネルギー消費、エントロピー消費、時間短縮のための並列化可能性、オフライン／オンライン計算のトレードオフなど、多くの他の指標が重要になり得る。
- **検証者計算量:** プロトコル期間中の検証者の計算労力。
- **セットアップ・コスト:** 共通参照文字列などのセットアップ・パラメータのサイズ、およびセットアップ作成の計算コスト。

証明システム仕様の読者は、効率測定に必要とする粒度が異なり得る。例として、情報理論的コアから構成され、その後暗号プリミティブでコンパイルされ完全なシステムとなる証明システムを考える。実装者は、情報理論的コアと暗号コンパイルの詳細な性能分析を望む可能性が高い。これはトレードオフと最適化の選択を導くためである。一方、利用者は高水準の性能分析と競合する証明システムとの同条件比較を望む可能性が高い。したがって、支配的な効率コストをすべて定量化する詳細分析と、妥当なパラメータ選択に対する性能を要約し最適な性能領域を特定する結論的分析の双方を提供することを推奨する。

1.8.1 セキュリティ性質の特徴づけ

ある技術のベンチマークでは、健全性とゼロ知識性など、異なるセキュリティ性質について達成または予想される異なるセキュリティ水準を明確にすべきである。各場合に、セキュリティの種類が無条件、統計的、または計算量的のいずれであるかも明確にすべきである。計算量的セキュリティを考える場合、事前計算がセキュリティ水準にどの程度影響し得るか、既知の攻撃が並列化可能かどうか、およびどのように並列化可能かを明確にすべきである。すべてのセキュリティ主張は、それが証明済みセキュリティ還元に基づくのか、ヒューリスティックな予想に基づくのかを明確に限定して述べるべきである。いずれの場合も、セキュリティ分析は、必要な計算量的仮定と実装要件を明確にすべきである。セキュリティ水準が古典的敵対者または量子的敵対者にどのように関係するかを明示すべきである。該当する場合、ベンチマークは量子的敵対者に対する技術のセキュリティ（不適合の可能性を含む）を特徴づけるべきである。

1.8.2 ベンチマークのための計算量的セキュリティ水準

各技術のベンチマークは、予想される計算量的セキュリティ水準 κ が約 128 ビット以上である少なくとも 1 つのパラメータ化を含まなければならない。各技術は、192 ビットまたは 256 ビットなど、少なくとも 1 つの追加のより高い計算量的セキュリティ水準でもベンチマークされるべきである。1 つだけの場合は後者が望ましい。複数水準でのベンチマークは、セキュリティ水準に応じて効率がどのように変化するかを理解するのに役立つ。256 ビットのような高いセキュリティ水準への関心は、たとえば意図された 128 ビットと比べた慎重な（かつヒューリスティックな）安全余裕と考えられる。これは、予想されたセキュリティ水準が後に過大評価であったと判明する可能性に対処することを意図している。128 ビット未満の計算量的セキュリティでの評価は、実行複雑性または時間がセキュリティ・パラメータとともにどのように変化するかを明らかにする目的では正当化され得るが、実用的セキュリティの推奨と解釈されるべきではない。

より低い計算量的セキュリティ・パラメータを許す例外。最大限の注意を払えば、128 ビット未満の計算量的セキュリティ水準が正当化される場合がある。以下はその例外である。一部の対話型 ZKP（2.6 節を

参照）では、プロトコル実行の一部期間だけ保持されればよい暗号学的性質がある場合があり、その期間はたとえば 1 分未満など固定時間未満であることが要求されることがある。たとえば、コインフリッピング・プロトコル中の一時的な隠蔽を可能にするコミットメント方式は、相手方が秘密値を明らかにするまでしか保持される必要がない場合がある。このような場合、その性質は 128 ビット未満のセキュリティで実装され得る。ただし、特に並行設定での合成に関して特別な注意を払い、効率差が実質的である場合に限る。このようなプロトコル構成要素の低いセキュリティ水準は、たとえば否認可能性（非移転可能性）を可能にするためにも有用であり得る。応用によっては、証明される知識の対象となる証人自体が、ZK インスタンスから 128 ビットセキュリティ相当より少ない計算資源で発見可能である場合、慎重な分析のもと他の例外も許容され得る。

1.8.3 ベンチマークのための統計的セキュリティ水準

一部の対話型 ZKP システムの健全性セキュリティは、検証者がチャレンジの新鮮性およびエントロピーを検証または信頼できることに基づく場合がある。たとえば、検証者が生成する nonce や、信頼されたランダムネス・ビーコンから得られる乱数である。そのような場合の一部では、統計的セキュリティ・パラメータ σ（例: 40 または 64 ビット）を、「一回限り」セキュリティを持つプロトコルの誤り確率（それぞれ 2^-40 または 2^-64 など）を指すために用いることがある。ここで、悪意ある証明者が有効な証人の知識なしに成功するには、チャレンジが何であるかを事前に推測する必要がある。失敗した証明試行の繰り返しを検出し防止できる仕組みがある場合、より低い統計的セキュリティ・パラメータが適している場合がある。

適切な最小パラメータは応用シナリオに依存し得るが、ベンチマークは、予想される統計的セキュリティ水準が少なくとも 64 ビットである少なくとも 1 つのパラメータ化で行わなければならない。統計的セキュリティ・パラメータの変化による効率差が大きい場合には、複数のセキュリティ水準でベンチマークすることを推奨する。40、64、80、128 ビットの場合が提案される。

多くの対話型プロトコルは、Fiat-Shamir 変換などを用いて、もともとは対話型の commit-challenge-response プロトコルのチャレンジを非対話的に計算することで、非対話型 (NI) 版へコンパイルできる。得られた NI 版では、証明者が証明の唯一の生成者であるため、応答を生成できるチャレンジを得るまで多数の試行を繰り返すことができる。元のプロトコルの計算コストが Õ(σ)、すなわち統計的セキュリティ・パラメータに対して線形を超えない場合、ベンチマークには、プロトコルが NI 版へ変換されたときにも意図された計算量的セキュリティ（少なくとも 128 ビット）を保持できる少なくとも 1 つの統計的パラメータを含めるべきである。必要な試行回数の期待値が 2^κ のオーダーであれば、計算量的セキュリティは保持される。

第 2 章 パラダイム

2.1 背景

文献には多数の種類のゼロ知識証明 (ZKP) システムがあり、通信コスト、計算コスト、基礎となる暗号学的仮定の間でさまざまなトレードオフを提供する。これらの証明の多くは、情報理論的 (IT) 証明システムと、その IT 証明システムを実際の ZKP へコンパイルする暗号コンパイラ (CC) に分解できる。

IT 証明システムの性質は、正当化を要しないものとして扱われるモデル内で考えられる。しかし、現実世界で有用な証明を得るには、IT 証明システムを、その理想的構成要素を除去する CC によって具体化する必要がある。現実世界における実際の証明実装には、命題、インスタンス、証人を何らかの具体的な仕方で表す算術化手続きについての合意も必要である。

IT/CC の分離はプロトコル設計を簡素化し、各部分を別々に分析、最適化、実装できるようにする。また、手元の課題に最適な解決策を探すとき、設計空間を体系的に探索しやすくする。簡潔性のため「ZK」という語が省略されることがあるが、暗黙の既定の焦点は ZKP システムにある。IT/CC の観点（2.2 節および 2.3 節で考える）は、本質的には NP 関係に対する一般証明システムに焦点を当てる。ZKP には IT/CC 分類の外で説明される他の種類と側面もあり、いくつかの特殊言語（2.4 節）、再帰的証明合成の側面（2.5 節）、対話性（2.6 節）が含まれる。

FF2.1 今後の図案提案: IT、CC、算術化という 3 つの主要概念を表す図。

2.1.1 情報理論的 (IT) 証明システム

IT 証明システムは、証明者と検証者が計算量的に無制限であっても、健全性と ZK 保証を提供する。しかし、これらは理想化された相互作用モデルを用いる。そのモデルでは、証明者または検証者の能力が、何らかの理想的な仕方で制限、構造化、または拡張されている。

一般的な理想化モデルの一種は、暗号コンパイラによって具体的に実現されるブラックボックス通信機能へのアクセスを提供する。たとえば、その理想化モデルでは、証明者が長い証明ベクトルを生成し、その後、検証者にこのベクトルへの制限されたアクセスだけを与えることがある。たとえばベクトルの少数の要素だけを問い合わせることを許し、残りを見ることはできないが、同時に証明者が問い合わせに応じて回答を変更することもできない、すなわちベクトル全体が事前にコミットされている、というモデルである。別の一般的例は理想コミットメントである。そこでは一方の当事者が値を信頼されたオラクルに送ることで値にコミットし（値は隠されたまま）、後で他方の当事者がその値を取得できるようにする（値が変更されていないという保証付き）。

FF2.2 今後の図案提案: 理想コミットメントの図示。

理想化モデルは当事者に制限を課すこともできる。たとえば、制限された証明者は、事前には協調できるがプロトコル実行中には互いに通信できない複数の別個の当事者から構成され得る。これにより、検証者は彼らに個別にチャレンジを送り、回答の一貫性を比較できる。このような場合、暗号コンパイラは、理想化された当事者を、人工的な制限を受けない（ただし計算量的には制限され得る）現実世界の当事者へ変換する方法を示す。

2.1.2 暗号コンパイラ (CC)

暗号コンパイラは、IT 証明システムを、証明者と検証者の直接的な相互作用を含む現実世界のプロトコルへ変換する。前述した相互作用モデルと当事者能力に関する理想化仮定は除去され、多くの場合、それらを強制することによって、もはや公理的に仮定する必要がないようにする。これは通常、より単純な暗号学的構成要素に依拠するプロトコルを用いて行われる。暗号コンパイラが用いるプリミティブは、ハッシュ関数やペアリング向き楕円曲線などの（場合によってはヒューリスティックな）具体的インスタンスによって実現される。これにより、現実世界の応用で実装・利用できる具体的スキームが得られる。

基礎となる構成要素は、衝突困難ハッシュ関数や準同型コミットメント方式などの暗号プリミティブであり得る。この場合、IT 証明システムとコンパイラの組合せのセキュリティは、これらのプリミティブの実現における暗号学的仮定に依存し、通常は計算量的に制限された当事者に対してのみ適用される。

あるいは、コンパイラは IT 証明システムが仮定する高水準の理想モデルを除去するが、より細かい局所的仮定を捉えるための新しい理想化プリミティブ（たとえばランダムオラクルや一般双線形群）を導入する場合がある。この場合、そのセキュリティは何らかの理想モデル仮定（たとえばランダムオラクルへの問い合わせ数の制限）の下で成り立ち、現実世界での実現には、さらなるコンパイラ（たとえば十分に定義された仮定の下でより細かいモデルの性質を達成するため）を用いるか、ヒューリスティック（直観的にはもっともらしいが形式的に証明されていない）であることになる。これらも通常、計算量的に制限さ

れた当事者に対してのみもっともらしい。暗号コンパイラは、相互作用の除去、一部メッセージサイズの縮小、あるいは ZK 機能を持たない IT 証明システムへの ZK 機能の追加など、望ましい追加性質を提供することもある。

2.1.3 算術化

各 IT 証明システムには、証明すべき命題を受け取るための何らかのネイティブ表現がある。一般的なものには、多項式方程式、ブール回路、算術回路が含まれる。これに対し応用側は、有限オートマトン、CPU+メモリモデル、ルックアップ表や反復要素で拡張された回路など、より高水準で簡潔な表現を好むことが多い。

高水準表現からネイティブ表現への変換は、証明スキームそのものとは独立したフロントエンドによって完全に処理され得る。しかし、多くの証明システムは、提示および実装の段階ですでに、自然な形式からより低水準のネイティブ形式（たとえば多項式方程式）への内部変換を含んでいる。この変換を算術化と呼び、以下の手続きからなる。

- **命題還元:** ネイティブな命題表現（たとえば検査対象の多項式）を生成する。
- **インスタンス還元:** ネイティブなインスタンス表現（たとえば多項式が評価されるインスタンス変数への割当て）を生成する。
- **証人還元:** ネイティブな証人表現（たとえば証人変数への割当て）を生成する。

2.1.4 複雑性と特徴

IT 証明システムと暗号コンパイラを組み合わせて生じる具体的な ZKP スキーム、およびその他の特殊 ZKP は、さまざまな特性を持ち得る。これらの証明に関する重要な複雑性指標には、証明長、ラウンド複雑性、クエリ複雑性（クエリ数および各クエリのサイズ）、体（アルファベット）サイズ、検証者の決定述語の複雑性（算術回路として表現した場合など）が含まれる。

FF2.3 今後の図案提案: 関心のある各種指標を含む図。

証明の簡潔性。証明長に関して、関心のあるカテゴリがいくつかある。

1. **完全簡潔:** 命題サイズから独立、すなわち `O(1)` 個の暗号要素。
2. **polylog 簡潔:** 暗号要素数が回路サイズの polylogarithmic。
3. **平方根簡潔:** 回路サイズの平方根に比例。
4. **深さ簡潔:** 命題を表す検証回路の深さに依存。
5. **非簡潔:** 証明長が回路サイズに対して劣線形でない。

SNARK という語は、succinct non-interactive argument of knowledge（簡潔な非対話型知識アーギュメント）を表す [BCCGLRT17]。

複雑性の参照基準。証明の簡潔性などの複雑性指標は、適切に文脈化されるべきである。たとえば、複雑性指標が公開情報（命題およびインスタンスなど）だけを考慮するのか、秘密部分（証人など）にも依存するのかは重要である。また、証明長の複雑性（線形、二次、平方根など）が命題サイズに関して述べられる場合、その命題サイズが算術化前か算術化後かも重要である。異なる研究では異なる参照基準が用いられる。

その他の特徴。さらに関心のある特徴づけの側面には以下が含まれる。

1. **セットアップ信頼**。すべての非対話型ゼロ知識証明には、すべての当事者が合意するグローバル公開パラメータ集合が必要である。グローバル公開パラメータを、たとえば有名な引用文をハッシュすることでランダムに選べるなら、その証明システムは公開コイン・セットアップ、または信頼不要セットアップを持つという。可能な場合、これは最も理想的なセットアップである。

   グローバル公開パラメータをランダムに選ぶことができず、信頼された当事者またはマルチパーティ計算を用いて選ばれる場合、その証明システムは秘密コイン・セットアップ、または信頼されたセットアップを持つという。この場合、信頼された当事者またはマルチパーティ計算に関与するすべての当事者の連合は、証明を偽造できる。信頼されたセットアップは、ユニバーサルでも言語依存でもあり得る。ユニバーサル・セットアップは同じセットアップを任意の言語に使用できるため、言語依存セットアップより高い柔軟性を提供する。

2. **プリミティブの種類**。対称鍵暗号（ブロック暗号など）、衝突困難ハッシュ関数、ランダムオラクル、公開鍵暗号など。

2.2 情報理論的 (IT) 証明システム

本節は、理想化モデルで設計された各種 IT 証明システムのカテゴリを概観する。これらは暗号学的仮定に依拠しない証明を可能にし、しばしば優雅で理解しやすい構成をもたらす。これらの証明は直接には現実世界で利用できないことが多いが、暗号コンパイラとの組合せには有用である。

図 2.1: ZKP のためのさまざまな IT 証明システム（内容の説明）

- PCP: 古典的な 3-coloring proof、PCP 定理からの構成。
- Linear PCP: Hadamard ベース、QAP/QSP/SSP からの構成、Line-Point、Fully Linear PCP。
- MPC in the head。
- IOP: Interactive PCP、Fast RS IOPP、Linear IOP。
- Linear IOP: IP ベース、Polynomial IOP、Fully Linear IOP。
- ILC: Ideal Linear Commitment。

凡例: Fast RS IOPP（FRI、Fast Reed-Solomon IOP of Proximity）、ILC（Ideal Linear Commitment）、IOP（Interactive Oracle Proof）、IP（Interactive Proof）、IT（Information Theoretic）、MPC（Secure Multi-Party Computation）、PCP（Probabilistically Checkable Proof）、QAP（Quadratic Arithmetic Program）、QSP（Quadratic Span Program）、SSP（Square Span Program）、ZKP（Zero-Knowledge Proof）。

記法。検証者が証明 Π に「点クエリ」を行うとは、検証者が証明オラクル O_Π へアクセスでき、このオラクルが入力として添字 i を取り、証明の第 i 記号 Π(i) を出力することをいう。検証者が証明 Π ∈ F^m（有限体 F 上）に「内積クエリ」を行うとは、証明オラクルが入力としてベクトル q ∈ F^m を取り、値 <Π, q> ∈ F を返すことをいう。検証者が証明 Π ∈ F^{m×k} に「行列ベクトル・クエリ」を行うとは、証明オラクルが入力としてベクトル q ∈ F^k を取り、行列ベクトル積 Π·q ∈ F^m を返すことをいう。

2.2.1 要約

古典的な命題 x の証明では、証明者は検証者が完全に読むための証明文字列 π を送り、検証者はその証明が命題の真実性について効率的で絶対的な検証可能性を伝える場合に限り受理する。たとえば、知識の古典的証明の自明な（ZK ではない）例は、既知の証人 w を直接送ることである。

その後、証明検査の分野は、対話型証明（単一の文字列の分析ではなく相互作用から納得が生じる）とアーギュメント（確実性は絶対ではないが圧倒的である）を含むように発展し、しばしば ZK 機能も伴う。

本節では、理想的（実現不可能）な構成要素を利用することが多く、必ずしも ZK ではない IT 証明システムを説明する。しかし、これらは後に簡潔かつ ZK である実際の証明へコンパイルすることを可能にする。図 2.1 は、次の主要な IT 証明システムの種類を考える。

1. **Probabilistically Checkable Proof (PCP):** PCP 証明では、証明者が（場合によっては非常に長い）証明文字列 π を検証者へ送る。検証者は証明に点クエリを行い、命題 x 全体を読み、受理または拒否する。
2. **Linear PCP:** Linear PCP 証明では、証明者がベクトル空間 F^m 内にある（場合によっては非常に長い）証明文字列 π を検証者へ送る。検証者は証明に線形クエリをいくつか行い、命題 x 全体を読み、受理または拒否する。
3. **MPC-in-the-head:** 証明は、シミュレートされた半正直セキュリティを持つ安全なマルチパーティ計算 (MPC) 実行の部分ビューから構成できる。証人はシミュレートされた当事者間で秘密分散され、その当事者たちが（シミュレーションにより）ZKP 述語を安全に計算する。部分トランスクリプトは、健全性と ZK を同時に保証する。
4. **Interactive Oracle Proof (IOP):** IOP は PCP の一般化であり、対話型設定を考える。各通信ラウンドで、検証者はチャレンジ文字列 c_i を証明者へ送り、証明者は PCP 証明 π_i で応答する。検証者はこれに点クエリを行える。複数ラウンドの相互作用後、検証者は受理または拒否する。
5. **Linear IOP:** Linear IOP は Linear PCP を対話型設定へ一般化したものである（上記 IOP 参照）。ここでは証明者が各ラウンドで証明ベクトル π_i を送り、検証者は線形（内積）クエリを行える。
6. **Ideal Linear Commitment (ILC):** ILC モデルは Linear IOP と似ているが、各ラウンドで証明者が証明ベクトルではなく証明行列を送る点が異なる。検証者は証明行列とクエリベクトルの積を学ぶ。各 ILC 証明行列は、入力と検証者メッセージの任意関数の出力でよく、線形関数に限定されない。

FF2.4 今後の図案提案: さまざまな IT 証明システムを絵で比較する 1 つの表。

2.2.2 Probabilistically Checkable Proof (PCP)

Probabilistically checkable proof は、証明者と検証者の間の相互作用を仮定する証明パラダイムである。証明者は最初のメッセージを送り、有効な証人が存在する場合にのみ以後のメッセージで検証者を納得させられる。本節ではまず、3-coloring 問題を証明する古典的 PCP 例を提示する。次に、PCP 情報理論的パラダイムと PCP 定理について、より正確な説明を与える。

2.2.2.1 古典的 3-coloring 証明　グラフ 3-coloring 問題は、NP に対する最初の ZKP システム [GMW91] の基礎であった。この問題のインスタンスは無向グラフ G = (V, E) である。命題、すなわち証明の目標は、グラフが 3-coloring を持つこと、すなわち頂点集合 V 内の各ノード i ∈ V に 3 色のいずれか c(i) ∈ {1,2,3} を割り当てることができ、任意の辺 (i,j) ∈ E、すなわち接続された任意のノード対が異なる色の 2 頂点を結ぶこと、である。証人は、各ノードを 3 色のいずれかに写す彩色割当て c : V -> {1,2,3} である。

R_3COL を、与えられたグラフ G に対して c が良い 3-coloring であるかを検査する関係とする。すなわち、(G, c) ∈ R_3COL は、#(c) = 3 かつすべての (u,v) ∈ E について c(u) != c(v) であることと同値である。関係 R_3COL は NP 完全であるため、グラフが 3 彩色可能であることを証明する ZKP を持つことは、任意の他の NP 言語（たとえば関係 R に関連付けられたもの）におけるメンバーシップを証明する ZKP を持つことを意味する。これは、証明者と検証者がそれぞれの R に対する入力を R_3COL の入力へ変換する多項式時間還元を局所的に適用することで行われる。

プロトコル。 R_3COL に対する古典的 ZKP は次のように進む。ここでは理想コミットメント機能を提供する IT システムを考える。

- 証明者は入力 (G, c) に対して、色のランダムシャッフルを定義するランダム置換 ϕ : {1,2,3} -> {1,2,3} を選び、シャッフルされた彩色を c'(v) = c(ϕ(v)) とする。
- 各ノード v ∈ V について、証明者はシャッフル後の色 c'(v) にコミットする。現実世界では、証明者は検証者へコミットメントを送る、またはコミットメントを提供するために相互作用する。
- 検証者は、証明者に、一様に選ばれた辺 (u,v) ∈ E の色を開くよう求める。
- 証明者はコミットメントを開き、c'(u) と c'(v) を検証者に明らかにする。
- 検証者は、証明者が 2 つのコミットメントを正しく開き、それらが {1,2,3} 内の異なる値を含む場合に限り、G を 3 彩色可能グラフとして受理する。

FF2.5 今後の図案提案: 古典的 R_3COL ZKP プロトコルを図示する。

セキュリティ分析。上記プロトコルは必要な 3 性質を満たす。

- **完全性:** 彩色 c が有効なら、その置換版 c' も有効である。したがって正直な証明者は常に c' 中の 2 つの異なる色を正しく開ける。
- **ZK:** 明らかになるのは 2 つのシャッフルされた色 c'(u), c'(v) だけであり、これは任意のランダムな異なる色のペアを選ぶことでシミュレートできる（1.6.4 節を参照）。これは、証明者が u と v が有効な辺を形成するかを検査する限り、悪意ある検証者が u と v を任意に選ぶ場合でも成り立つ。
- **健全性:** G が有効な彩色を許さない場合、悪意ある証明者がどのような色 c* にコミットしても、少なくとも 1 つの辺 (u,v) ∈ E が c*(u) = c*(v) を満たす。したがって検証者は少なくとも 1/|E| の確率で証明を拒否する。この健全性水準は低いが、ゼロ知識性を保ちながら、プロトコルを独立に多数回逐次実行することで増幅できる（たとえば、誤り確率をおよそ 2.7^{-k} とするために κ·|E| 回）。並列反復は ZK を保たないことに注意する。

2.2.2.2 PCP 定理から　一般に、NP 関係 R に対する PCP では、証明者は命題・証人ペア (x, w) から証明文字列 π ∈ Σ^m（有限アルファベット Σ 上の長さ m の文字列）への確率的多項式時間写像を計算する。検証者は π の記号のランダムに選ばれた部分集合を問い合わせ、その後、受理または拒否を決定する。重要なのは、π の内容が、検証者がどの π の添字を問い合わせるかを証明者が知らない状態で生成されることである。より正確には、検証者アルゴリズムは、インスタンス x から問い合わせ記号集合 Q ⊂ [m] へのランダム化写像、および問い合わせられた記号の内容に基づき受理または拒否を決める決定述語 D(x, Q, π_Q) によって指定される。PCP 定理 [Mic00; Kil92] は概略として以下を述べる。

定理 2.2.1 - PCP 定理。 NP のすべての言語に対して、対数通信複雑性を持つ効率的 PCP が存在する。

ゼロ知識 PCP (zk-PCP) はさらにゼロ知識性（1.6.4 節参照）を満たす。ここでゼロ知識シミュレータは、既定では、x のみを与えられて正直な検証者のビュー (Q, π_Q) を完全にサンプルすることが求められる。与えられた入力 x 上で正直な検証者によってある集合 Q が生成され得るかどうかを効率的に認識できると仮定する。文献では、非 zk PCP はしばしば小さいクエリ複雑性を持つことが要求される。しかし、3 彩色の例はゼロ知識であり、小さいクエリ複雑性を持たないことに注意されたい。

例: PCP としての 3-coloring。上記 2.2.2.1 節の 3-coloring 例では、π は Σ = {1,2,3} 上の長さ m = |V| の文字列であり、各 v ∈ V について c'(v) の値から構成される。ランダム性は、c を c' に変換するための置換 φ に由来する。ここで Q は、E 内の辺に対応するサイズ 2 の集合上で一様に分布し、D は 2 つの問い合わせ記号が異なる場合に受理する。R_3COL に対する zk-PCP の基本版は、1 - 1/|E| という高い健全性誤りを持つ。健全性は、（正直な検証者に対する）ゼロ知識性を尊重しつつ、独立に生成された証明 π^{(i)} を連結し、それらを独立に問い合わせることで増幅できる。同じ π へ複数回問い合わせるとゼロ知識性が損なわれることに注意する。

議論。上記の zk-PCP 概念は、健全性と（正直な検証者に対する）ゼロ知識性がともに計算量的に無制限な当事者に関して成り立つという意味で、情報理論的である。追加の暗号学的機構なしには、これをそのままゼロ知識証明プロトコルとして直接利用できない理由は、ゼロ知識性に必要な π への制限アクセスを検証者にだけ許しつつ、悪意ある証明者が健全性の依拠する π と Q の独立性仮定を破れないようにする方法が明らかでないからである。言い換えると、Q を先に証明者へ送ると悪意ある証明者は健全性を破れるし、π 全体を検証者へ送ると検証者が正直であってもゼロ知識性は損なわれる。暗号コンパイラの役割は、任意の zk-PCP（場合によっては構文上の制限付き）をゼロ知識証明プロトコルへ変換することである。

2.2.3 Linear PCP

PCP モデルおよび関連する PCP 定理は驚くほど強力であるが、高いコストを伴う。証明者は長い証明を明示的に書き出す必要があり、その大部分は読まれない。このコストは、理想化モデルを変更して、異なる、より表現力のある種類のクエリを許すことで低減できる。直観的には、証明者は小さい証明を生成し、それをはるかに大きいクエリ空間を通じて問い合わせられるようにすることで、クエリ手続きが負担の一部を担う。

FF2.6 今後の図案提案: 一般的な Linear PCP を図示する。

この単純かつ有用な例が Linear PCP (LPCP) モデルである。標準 PCP では、検証者は証明 π の個々の記号を読む有界個の点クエリを行える。一方 Linear PCP では、各クエリが π の各成分の任意の線形結合を取ることを許す。より具体的には、有限体 F 上の Linear PCP では証明はベクトル π ∈ F^m であり、各クエリ q_i ∈ F^m は内積 <π, q_i> を返す。既定では、クエリ q_i は入力非依存であり、x とは独立に選べることを要求する。任意の NP 関係 R に対して、多項式証明サイズ m、健全性誤り O(1/|F|)、定数個のクエリを持つそのような LPCP を構成できる。このより強力な PCP モデルの主な代償は、対応する暗号コンパイラが通常、「公開鍵暗号」の世界に属するより強いプリミティブに依拠する必要があり、さらに基礎となる LPCP の効率特徴を完全に尊重するためには強いセットアップ形態を必要とすることである。

LPCP は、単純な低通信証明システムを達成する方法として導入された [IKO07]。初期の動機は、複雑な PCP 機構を、より強い暗号を使う代償でより単純なものに置き換えることであった。

しかし当時すでに、単純性を超えて、この新しいアプローチが従来の PCP ベース手法では不可能だった効率特徴へ導き得ることが観察されていた。すなわち、R の複雑性から独立した定数個の「暗号文」または群要素だけを含む証明者から検証者への通信である。

以下では、Hadamard ベースと Quadratic Arithmetic Program (QAP) という 2 種類の Linear PCP を説明する。これらは現在の文献で最も一般的に用いられる Linear PCP である。本書で扱わない Linear PCP の種類には、Quadratic Span Program (QSP)、Square Span Program (SSP)、Line Point ベースなどがある。

2.2.3.1 Hadamard ベース　Hadamard ベース LPCP を提示するには、NP 関係を、体の加算と乗算を行うゲートを持つ有限体 F 上の算術回路として考えるのが便利である。具体的には、証明者は公開既知の算術回路 C と公開命題 x ∈ F^n に対して、C(x, w) = 0 となる証人 w ∈ F^m が存在することを検証者に納得させたい。

C と x はまず、s 個の変数 Y_1, ..., Y_s に関する次数高々 2 の多変数多項式 Q_i による、Q_i(Y) = 0 の形の s 個の二次方程式系へ変換される。C(x, w) = 0 を満たす w ∈ F^m が存在することと、すべての i について Q_i(y) = 0 を満たす y ∈ F^s が存在することが同値となる。

Q_i(Y) の充足可能性を証明する Hadamard LPCP では、証明者は入力 (x, w) に対して、まず入力 (x, w) 上の C の全 s ゲートの値 y ∈ F^s を計算する。次に、成分 yhat_{jk} = y_j y_k からなる二次サイズのベクトル yhat ∈ F^{s^2} を計算し、証明 π = (y, yhat) を出力する。y に関する任意の次数 2 多項式は、検証者が π へ単一の線形クエリを行うことで得られることに注意する。

検証者のクエリは 2 つの目標を達成する。(1) π の 2 部分がテンソル関係と整合していることを確認する。(2) すべての Q_i 関係の充足可能性を確認する。目標 (1) は、y の成分のランダム線形結合 (Σ_{j=1}^s r_j y_j)^2 を計算する 2 つの方法、すなわち直接問い合わせる方法と、対応する r_j r_k y_j y_k の組合せを

問い合わせる方法を比較することで実現される。目標 (2) は、Q_i(y) を問い合わせ、Q_i(y) = 0 であることを検証することで実現される。健全性は Schwartz-Zippel 補題 [Mos10] から従う。

高速検証。第一の特徴は、入力非依存の前処理が与えられた場合の高速検証である。クエリ q_i の生成コストは検証回路 C のサイズとともに増加するが、検証者は入力 x が知られる前にクエリを生成し、後で使用するために r_1, ..., r_n だけを保持できる。x が知られると、検証者は O(n) 算術演算だけで α = Σ_{i=1}^n r_i x_i を計算できる。最後に、LPCP の回答 a_1, a_2, a_3 が得られると、検証者は a_2 = a_1^2 および a_3 = α を確認することで受理するかどうかを決定でき、これは O(1) 個の体演算だけで済む。この高速検証機能は、同じクエリを複数命題の証明に再利用するときに特に有用である。

再利用可能健全性。Hadamard LPCP の別の特徴は、同じクエリを複数証明の検証に再利用しても成立する強い健全性である。任意の x と証明 π* について、証明者は x, π* だけに基づき、検証者が受理するかどうかを予測できる。ただし、検証者の未知のランダムクエリに関して O(1/|F|) の誤り確率を除く。これは、F が十分大きい（たとえば |F| >= 2^80）場合、検証者が π* を受理したかどうかを知っても、クエリ q_j について無視できる量の情報しか明らかにならないことを意味する。この強い健全性性質は、各インスタンスについて検証者が受理したかどうかを証明者が学べる場合でも、LPCP ベース SNARG の SRS を再利用可能にするために有用である。これは、古典的 (zk-)PCP が、複数証明に再利用された場合に秘密クエリ集合を悪意ある証明者が徐々に学べる「選択的失敗」攻撃に本質的に弱い [CDIKLOV19] ことと対照的である。この攻撃は、正直に生成された証明を少し乱し、各乱された証明インスタンスを検証者が受理したかどうかを学ぶことで実行できる。

2.2.3.2 QAP (R1CS) から　口語的には制約系を「回路」と呼ぶことが多いが、制約系はブール回路や算術回路とは異なる形も取り得る。いくつかのスキームは算術回路を直接ゼロ知識証明へコンパイルする [CD98] が、Quadratic Arithmetic Program (QAP) を扱う方が便利な場合がある。QAP は有限体上の双線形制約系 (BCS) であり、各制約は 2 つの入力の二次結合を許す。すなわち、スカラー倍、加算、乗算である。

正確には、ここで考える BCS は [BCGTV13, Appendix E] で定義される「rank-1 quadratic equations の系」と同じであり、記法に小さな変更がある。ここでは w ベクトルは証人だけを表す。一方、後者では w はインスタンスと証人の連結を表し、したがって w とインスタンス x の間の対応する整合性検査を追加している。

これらは、libsnark [libsnark] の「Rank 1 Constraint System (R1CS)」とも本質的に同一である。同値な定式化は、QAP（[GGPR13; PHGR13; BCGTV13; BCTV14b] などに従う）として、多項式の割り切りを直接推論する形で表現できる。

双線形制約系は 2 つのベクトルを扱う。n_x 個の要素からなり x = (x_1, ..., x_{n_x}) と表されるインスタンスと、n_w 個の要素からなり w = (w_1, ..., w_{n_w}) と表される証人である。制約系は、両者がいくつかの制約 n_c によって関係づけられることを述べる。各制約は特定の形の二次方程式である。すべての要素と演算は大きな素体 F_p 上で行われ、ここでは大きな素数 p を法とする整数として表す。特定の素数および体要素表現は、QAP ベース zkSNARK 構成で用いられる楕円曲線に関連する。

定義 2.2.1。n_x, n_w, n_c を正整数とし、n_z = n_x + n_w + 1 とする。F_q 上の双線形制約系は、S = ((a_j, b_j, c_j)_{j=1}^{n_c}, n_x) というタプルであり、a_j, b_j, c_j ∈ F_q^{n_z} である。このような系 S は入力 x ∈ F_q^{n_x} に対して、以下が真であるとき充足可能である。

S = ((a_j, b_j, c_j)_{j=1}^{n_c}, n_x) および x ∈ F_q^{n_x} に対し、証人 w ∈ F_q^{n_w} が存在し、z = (1, x, w) ∈ F_q^{n_z} としたとき、すべての j ∈ [n_c] について <a_j, z> · <b_j, z> = <c_j, z> が成り立つ。

ここで <·, ·> は F_q 上のベクトルの内積を表し、· は F_q における乗算を表す。

たとえば、双線形制約系

S = (((a_1=(0,3,0,0), b_1=(1,1,0,0), c_1=(4,0,5,0)), (a_2=(0,1,2,0), b_2=(6,0,1,0), c_2=(7,0,0,1))), 1)

は、z = (1, x_1, w_1, w_2) の下で、以下の 2 つの双線形制約を表す。

- <(0,3,0,0), z> · <(1,1,0,0), z> = <(4,0,5,0), z>
- <(0,1,2,0), z> · <(6,0,1,0), z> = <(7,0,0,1), z>

あるいは単に、

- 3x_1 · (1 + x_1) = 5w_1 + 4
- (x_1 + 2w_1) · (w_1 + 6) = w_2 + 7

である。定義により、この系 S は、入力 x_1 について、証人 w_1, w_2 が存在し、上記 2 式を満たすとき充足可能である。

zk-SNARK を用いてこの NP 命題を証明する場合、インスタンスは検証者に見える公開入力であり、証人は証明者のみが知り、証明構成に用いられるが、そのプライバシーは ZK 性によって保護される追加データである。

zk-SNARK 証明の生成コスト（時間とメモリ）は、主として制約数 n_c に依存する。この点を踏まえると、変数同士の乗算は「高価」である（各双線形乗算には新たな制約が必要であり n_c が増える）が、線形結合は各制約内で「無料」で付いてくることに注意する。

QAP は、すべての i について (Σ_j A_{ij} z_j) × (Σ_j B_{ij} z_j) = Σ_j C_{ij} z_j となる行列方程式集合を考える。ここで A, B, C は公開行列、z は制約系への公開入力と秘密入力を含むベクトルである。これを多項式系として符号化するため、まず体上の相異なる点 κ_i の集合を選ぶ。κ_i 評価点は効率上の理由からしばしば 1 の冪根に設定される。次に、すべての j について u_j(κ_i) = A_{i,j}, v_j(κ_i) = B_{i,j}, w_j(κ_i) = C_{i,j} となる多項式 u_j(X), v_j(X), w_j(X) を定義する。これらの多項式は線形補間技術で求められる。このとき、式が満たされることと、すべての κ_i について次が成り立つことが同値である。

(Σ_j u_j(κ_i) z_j)(Σ_j v_j(κ_i) z_j) = Σ_j w_j(κ_i) z_j

多項式 f(X) が f(α)=0 を満たすことと、(X-α) が f(X) を割り切ることは同値である。したがって、我々の方程式は、すべての κ_i について (X - κ_i) が

(Σ_j u_j(X)z_j)(Σ_j v_j(X)z_j) - (Σ_j w_j(X)z_j)

を割り切る場合に限り成り立つ。特に、これは、式が満たされることと、ある多項式 h(X) および t(X) = Π_i (X - κ_i) について

(Σ_j u_j(X)z_j)(Σ_j v_j(X)z_j) - (Σ_j w_j(X)z_j) = h(X)t(X)

が成り立つことと同値である。

IOP 証明者は、a(κ_j) = z_j であり、h(X) が上記のように定義される証明 π = a(X), h(X) を送る。検証者はランダムな体要素 r を選び、

(Σ_j u_j(r)a(κ_j))(Σ_j v_j(r)a(κ_j)) - (Σ_j w_j(r)a(κ_j)) = h(r)t(r)

が成り立つかを問い合わせる。Schwartz-Zippel 補題により、証明者が有効な証人を持たない限り、後者が偶然成り立つ確率は 1/|F_q| である。

2.2.3.3 Line-Point　貢献募集。

2.2.3.4 Fully Linear PCP　Referencesparadigms:IT:linear-IOP:fully-linear-IOP（2.2.6.3 節）を参照。

2.2.4 MPC in the head

安全なマルチパーティ計算 (MPC) プロトコル [Lin20] は、n 当事者が、n 個の局所入力を n 個の局所出力へ写す与えられた機能 f を、入力を「隠し」ながら計算できるようにする。「MPC-in-the-head」パラダイム [IKOS09] は、（完全正当性保証を持つ）単純な種類の MPC プロトコルを zk-PCP へコンパイルすることを可能にする。

FF2.7 今後の図案提案: MPC in the head と、それがどのように ZKP になるかを図示する。

MPC プロトコルから zk-PCP へ。 NP 関係 R(x, w) が与えられたとき、n 当事者 MPC 機能 f(x; w_1, ..., w_n) = R(x, w_1 ⊕ w_2 ⊕ ... ⊕ w_n) を定義する。各当事者 P_i の入力は公開命題 x と秘密入力 w_i からなり、R(x, w) は (x,w) ∈ R のとき 1、そうでなければ 0 である。直観的には、w_i は w の秘密分散シェアと考えられる。f の出力は全当事者に届けられる。Π_f を f を実現する MPC プロトコルとする。

zk-PCP 証明者は、入力 (x, w) に対して次のように証明 π を生成する。まず、w = w_1 ⊕ w_2 ⊕ ... ⊕ w_n となるように、w を n 個のランダムな加法シェア w_i にランダム分割する。次に、証明者は「頭の中で」、共通入力 x と秘密入力 (w_1, ..., w_n) 上の Π_f の仮想実行を行う。これには、各当事者 P_i の秘密ランダム入力 r_i を選び、すべての当事者が出力を得て終了するまで交換されるメッセージを計算することが含まれる。そして証明は π = (V_1, ..., V_n) となる。ここで V_i は、Π_f の仮想実行における P_i の完全なビューである。V_i は w_i、r_i、および P_i が受信したすべてのメッセージからなる。zk-PCP 検証者は、ランダムな記号ペア V_i, V_j を問い合わせ、(1) 2 つのビューが整合的であること、すなわち一方のビュー内の受信メッセージが他方のビューに暗黙に含まれる送信メッセージと一致すること、および (2)

ビューから暗黙に得られる P_i と P_j の出力がともに 1 であることを確認する。検証者は両条件が成り立つ場合に受理し、そうでなければ拒否する。Ishai らは設計上の他の柔軟性も議論している。たとえば、健全性を高めるために能動的 MPC プロトコルを使う、または複数の当事者ペアを検査することができる。

セキュリティ。上記 zk-PCP 構成は分析がかなり容易である。

- **完全性:** f の定義と Π_f の（完全）正当性から従う。
- **ゼロ知識性:** (i) 証人シェアのペア w_i, w_j が w について何も明らかにしないこと、および (ii) MPC プロトコル Π_f が任意の 2 当事者の結託に対してプライベートであること、から従う。
- **健全性:** R(x,w)=1 となる w が存在しない場合、分析すべき 2 つの場合がある。
  1. ビュー V_i が何らかの入力 (w_1^*, ..., w_n^*) 上の Π_f の有効な実行に対応する場合、すべてのビューの出力は 0 となり、検証者は常に拒否する。
  2. ビューが Π_f の任意の有効な実行と整合しない場合、少なくとも 1 つの不整合なビューのペアが存在しなければならない。検証者は確率 1 / binom(n,2) でそれを選び、拒否する。これは O(k) 回の反復により 2^{-k} へ低減できる。

効率。 MPC in the head は、とくに問題サイズが小さい場合や、証明者の計算コストが主なボトルネックである設定で競争力がある。基本的な対称鍵プリミティブで具体化できる。MPC プロトコルは、単一の実体が完全な情報を持たない分散設定のために設計されている。一方、証明システムの通常設定では、証明者が完全な情報を利用できる。これが、MPC ベース zk-PCP が他のシステムの漸近的な簡潔性水準に到達できない理由を説明する（2.2.5 節参照）。

適用可能性。上記プロトコルでは、zk-PCP は、f に対する任意の MPC プロトコル Π_f に基づき得る。ただし、安全な点対点チャネル上で、2 つの「半正直」当事者に対するセキュリティを提供する必要がある。すなわち、すべての当事者がプロトコルに従う場合、任意の 2 当事者の共同ビューは、それらの入力と f の出力だけからシミュレートでき、他の当事者の入力から独立である必要がある。この種の単純なプロトコルは、n >= 5 当事者について無条件に存在する [BGW88; CCD88; Mau06]。この構成は有用な仕方で修正・拡張できる。たとえば、(i) MPC プロトコルの完全正当性は、zk-PCP を対話型にする代償で緩和できる（2.2.5.1 節参照）、(ii) MPC モデルは入力非依存前処理で拡張できる [KKW18]、(iii) 多数の当事者に対する MPC プロトコルを用いることで、悪意ある当事者に対するセキュリティを持つ MPC プロトコルにより、無視可能な健全性誤りを達成できる。

IT コンパイラは、効率的 MPC の領域に存在する多数の技術を ZKP の領域へ移植するために使用できる。たとえば、代数幾何符号に基づく MPC プロトコル [CC06] は、ブール回路充足可能性に対する（統計的に健全な）ZKP プロトコルへ変換できる。そのプロトコルでは、通信複雑性と回路サイズの比が定数であり、健全性誤りは回路サイズにおいて無視可能である [IKOS09]。同様に、一般環 [CFIK03] や他の代数構造 [CDIKMRR13] をブラックボックス的に用いる MPC プロトコル、線形代数問題 [CD01] のための MPC プロトコルなど、多くの技術を活用できる。

2.2.5 Interactive Oracle Proof (IOP)

PCP システムと比べ、IOP システムは、証明者と検証者の間に追加の相互作用を許すため、より強力である。証明者は、回答すべき（ランダムな）チャレンジを制御できないことに注意する。それらは検証者によって選ばれるからである。追加の相互作用は、より効率的な証明者を設計するのに役立つ。対話型 zk-PCP は、MPC から zk-PCP への非対話型コンパイラにおいて、悪意ある証明者が MPC 当事者の乱数を MPC 正当性、ひいては zk-PCP 健全性を破るように選べるという問題を移すことによって、統計的正当性保証を達成する。

IOP は Interactive PCP の概念を一般化し、Probabilistically Checkable Interactive Proof の概念と一致する。

FF2.8 今後の図案提案: 一般的 IOP を図示する。

2.2.5.1 Interactive PCP　初期の応用。定数深さ回路で計算される NP 関係 R に対して、Interactive PCP [KR08] は驚くほど良いパラメータを持ち得る。低通信の古典的 PCP では証明サイズが R の回路サイズより大きい（おそらく本質的 [FS11]）のに対し、対話型の場合、証明サイズを概ね証人長まで削減でき、それでも低通信である。これはその後、定数深さ R から低深さ R（たとえば polylogarithmic 深さ）へ一般化された [GKR08]。空間制限計算に対する定数ラウンド変種も可能である [RRR16]。

Interactive Oracle Proofs。前述の「Interactive PCP」モデルでは、単一の証明 π があり、その記号を検証者が問い合わせることができ、さらに検証者と証明者の間に追加の（低通信）相互作用がある。PCP における相互作用を完全に一般化したものが Interactive Oracle Proof (IOP) モデル [BCS16; RRR16] であり、証明者が複数の証明 π_i を逐次的に生成し、検証者が問い合わせることを許す。より具体的には、

IOP モデルの公開コイン変種では、各 π_i は予測不能なランダムチャレンジ r_i への応答として現れる。検証者の証明 π_i へのクエリは相互作用の最後に行うことができる。IOP は、すべての証明 π_i から検証者が読むビット総数がインスタンスサイズの polylogarithmic である場合、完全簡潔である。IOP モデルが許す追加の相互作用により、このモデルにおける完全簡潔な証明の設計は、古典的 PCP モデルや上記の Interactive PCP モデルより単純になる。この単純性は顕著な効率改善にもつながる。

2.2.5.2 Fast RS IOPP　貢献募集。

2.2.6 Linear IOP

古典的 PCP モデルには、相互に直交する 2 つの緩和がある。相互作用を追加することと、点クエリではなく線形クエリを用いることである。Linear IOP (LIOP) モデル [BBCGI19] は、この 2 つの緩和を自然な形で組み合わせる。IOP モデルと同様、証明者は複数ラウンドで証明ベクトル π_i ∈ F^{m_i} の列を生成する。ここで各 π_i はランダムチャレンジ r_i への応答である。この相互作用の最後に、検証者は LPCP モデルと同様、各 π_i に線形クエリを行える。LIOP モデルは、以前の Interactive Linear Commitment (ILC) [BCGGHJ17] と密接に関連している。

FF2.9 今後の図案提案: 一般的 Linear IOP を図示する。

2.2.6.1 IP ベース　GKR プロトコル。古典的 sum-check プロトコル [LFKN92]（[Tha20] も参照）に基づく「二重効率的」GKR 対話型証明プロトコル [GKR08]、すなわち効率的な証明者と効率的な検証者を持つプロトコルは、次の特徴を持つ NP 用 LIOP として捉えられる。

- R(x,w) が、サイズ s、深さ d、入力数 m の階層化された算術回路で F 上で計算される場合、約 d ラウンドで証明 π_i はそれぞれサイズ約 log s の F 上のベクトルであり、単一ラウンドで証明ベクトルのサイズが約 m である。ここで ≈ は s の polylogarithmic 以下の乗法因子を隠す。
- 検証者は各証明に定数個の線形クエリを行い、次数 2 の決定述語を適用する。健全性誤りは約 d/|F| である。さらに、回路が適切な種類の簡潔な記述を持つ場合、検証者の実行時間は約 d + m にできる。

具体的には、回路は log-space uniform であるべきである。すなわち、ゲートラベルが与えられたとき、ゲートに関する関連局所情報を対数空間で計算できる必要がある。この一様性の特徴は、GKR プロトコルの元の文脈、すなわち証人 w を伴わない浅い多項式サイズ決定性回路の高速検証において重要であった。しかし、NP に対する証明システムの文脈（ゼロ知識の有無にかかわらず）では、上記のような LIOP は、簡潔な記述を持たない一般的な検証回路に対しても意味を持つ。命題非依存の前処理を用いれば、非一様な場合でも高速オンライン検証を達成できる。

空間制限計算に適用可能な、より洗練された関連プロトコルも、NP に対する（定数ラウンド）LIOP として捉えられる [RRR16; Gol18]。

最適化により、GKR プロトコルは具体的効率が良く、後続の改良 [CMT12; VSBW13; Tha13; XZZPS19] で達成されている。大きな体上の階層化算術回路について、Libra [XZZPS19] は算術設定で証明者側の定数計算オーバーヘッドを得た。Virgo++ [ZLWZSXZ21] は GKR プロトコルを任意の算術回路へ一般化し、階層化回路用プロトコルと同じ証明者の定数計算オーバーヘッドを得た。

これは、任意の構造、深さ、証人長の一般的検証回路に適用されるが、簡潔性を全く提供しない、上で議論した単純な定数オーバーヘッド zk-LPCP for NP と比較できる。低深さ決定性計算を効率的に検証する場合、すなわち ZK も証人も伴わない場合、GKR 変種は暗号コンパイラなしで、つまり実際の相互作用として用いることができ、通信が十分安価なら実用的である [WJBSTWW17]。

2.2.6.2 Polynomial IOP　Polynomial IOP (PIOP) は標準 IOP（2.2.5 節）の一般化であり、一部のラウンドにおいて、証明者が多項式を検証者へ「送る」ことを許され、検証者が少数の評価点でその多項式を問い合わせることを許される [BFS20; CHMMVW20]。PIOP は多項式コミットメント方式（2.3.5.2 節）を用いて簡潔なアーギュメントへコンパイルできる。本質的には、PIOP で証明者が送る各多項式 p は、多項式コミットメント方式によるその多項式への簡潔なコミットメントで置き換えられ、検証者が p へ行う評価クエリは、多項式コミットメント方式が提供する評価証明によって答えられる。

文献中の多くのプロトコルは、もともとこの言語で表現されていなくても PIOP と見なせる。たとえば vSQL [ZGKPP17] は、回路評価用 GKR 対話型証明 [GKR08] を回路充足可能性のための PIOP へ暗黙に適応している。概略として、与えられた回路 C について、C(x,w)=y となる証人 w が存在することを証明するため、PIOP 証明者はまず証人 w の低次数拡張多項式を送り、その後 GKR 対話型証明を適用して C(x,w)=y を確立する。関連する PIOP はさまざま存在する [ZXZS20; ZLWZSXZ21; WTTW18; XZZPS19]。別の例として [BTVW14] では、回路充足可能性に対する 2 証明者対話型証明が PIOP をもたらし得る。証明者はま

ず回路の「正しいトランスクリプト」の拡張多項式を送り、次に sum-check プロトコル [LFKN92] の単一呼び出しを用いてトランスクリプトが正しいことを確認する。Spartan [Set20] は、この PIOP を回路充足可能性から R1CS へ拡張した。Marlin [CHMMVW20]、Aurora [BCRSVW19]、Fractal [COS20] は、「単変量 sum-check プロトコル」に基づく R1CS 用 PIOP を与える。

2.2.6.3 Fully Linear IOP　一部の設定では、検証者は入力命題 x への完全アクセスを持たない。たとえば入力は銀行や病院など 2 つ以上の当事者間で分割され得る。このような分散設定でも、入力 x に対して実行されるクエリが、各当事者の保持するシェアへの局所クエリとして実装できるなら、証明システムを使用できる。たとえば、x が線形秘密分散方式、または線形準同型暗号／コミットメント方式を用いて秘密分散されているなら、各シェアに局所的に線形クエリを適用することで、x への線形クエリを効率的に行うことができる。

上記設定は、入力 x ∈ F^n と証明 π ∈ F^m に共同で線形クエリを適用する LPCP および LIOP モデルのより強い変種を自然に要請する。ここで検証者は、そのようなクエリを通じて以外には入力へ直接アクセスできない。この種の IT 証明システムは [BBCGI19] において fully linear proof systems の名で研究された。この概念は ZK 版へ拡張され、検証者は x ∈ L である事実以外、x と w について何も学ばない（通常の定義で w だけを隠すのと対照的）。これは、シミュレータが入力 x へアクセスしない ZK 定義によって形式化される。これにより、低クエリ複雑性を持つ fully linear proof system は、多項式時間言語に対しても、また P = NP であっても意味を持つことに注意する。これは proximity proofs [BGHSV04] に類似するが、後者は入力が L に近いという弱い保証しか与えない。また、検証者が入力の適切な符号化を問い合わせられる holographic proofs [BFLS91] とも密接に関連する。

Fully linear proof systems は、入力命題 x が証明者には知られているが複数検証者間に分散されている、効率的な分散ゼロ知識証明を目標とすることから動機づけられる。これは、複数マシンにまたがる並列計算を許す他の種類の分散ゼロ知識証明（たとえば [WZCPS18]）と混同すべきではない。

したがって自然な目標は、証明長とクエリ複雑性が小さい、特に入力長に対して劣線形である fully linear PCP (FLPCP) および fully linear IOP (FLIOP) を設計することである。この目標は、単純な言語、たとえば F^n 内の Booleanity check 言語 {0,1}^n、または F^{n/2} のベクトル対の連結で内積が 0 である内積言語についても十分に動機づけられる。上で議論した具体的な LPCP と LIOP はどちらも、同じクエリ数で fully linear にできることが分かっている。しかし、Hadamard LPCP および GGPR の LPCP の証明長は少なくとも n に線形であるのに対し、GKR プロトコル [GKR08] は、上記例について総証明長と回答長が回路深さに匹敵する、すなわち O(log^2 n) である FLIOP を与え、ラウンド数も同程度である。同様に、RRR プロトコル [RRR16] は低空間計算に対する定数ラウンド劣線形 FLIOP を含意する。

上記例のように、低次数多項式によって認識される言語に対しては、より単純で効率的な fully linear proof systems [BBCGI19] がある。たとえば、x が単一の次数 2 方程式を満たすことを証明する非対話型 zk-FLPCP は、サイズ O(√n) の証明と同程度のクエリ数で存在し、これは最適である [Kla03]。このプロトコルは、通信複雑性の上界 [AW09] の ZK 変種である。定数次数方程式系によって認識される言語（上記 2 例を含む）に対しては、O(log n) ラウンドの相互作用を持つ zk-FLIOP に甘んじる代償で、証明サイズとクエリ複雑性を O(log n) へ削減できる。これらの構成は、反復構造を利用する GGPR 型 zk-LPCP の一般化から始まり、その後バランシングと再帰によって効率を改善する。

2.2.7 Ideal Linear Commitment (ILC)

Ideal Linear Commitment (ILC) モデルは、LIOP の別の観点を提供する [BCGGHJ17]。ILC モデルにおける証明は、各証明ベクトル π_i が行列 Π ∈ F^{n_i×m_i} としてパースされ、各 π_i へのクエリがベクトル q ∈ F^{m_i} によって指定され、行列ベクトル積 Πq を返す LIOP と見ることができる。あるいは、基礎となる体をベクトル空間で置き換えれば、ILC は標準 LIOP と見ることもできる。

ILC モデルは Linear Interactive Proofs (LIP) モデル [BCIOP13] を緩和する。なぜなら、各 ILC 証明行列は入力と検証者メッセージの任意関数の出力でよい一方、各 LIP 証明行列は検証者メッセージの線形関数でなければならないからである。

FF2.10 今後の図案提案: 一般的 ILC を図示する。

線形時間で符号化可能な誤り訂正符号 [Spi96] および線形時間で計算可能なハッシュ関数 [AHIKV17] によって具体化されると、コンパイラは基礎となる ILC 証明の漸近的計算複雑性を保つ。

また、算術回路充足可能性に対する定数オーバーヘッドの「平方根簡潔」zk-ILC の構成も示されている。具体的には、サイズ s の F 上の算術回路によって計算される NP 関係 R(x,w) に対して、通信量は約 √s、ラウンド複雑性は O(log log s)、両当事者は O(s) 体演算が支配する実行時間を持つ RAM プログラムとして実装でき、健全性誤りは log |F| において無視可能である。この ILC と線形時間の暗号コンパイラ具体

化を組み合わせると、もっともらしい暗号学的仮定の下で、定数計算オーバーヘッドを持つ算術回路充足可能性に対する平方根簡潔なゼロ知識アーギュメントが得られる。

この ILC アプローチは、定数計算オーバーヘッドを持つ ZKP システムを得る。単純な LPCP ベース・アプローチより漸近的簡潔性が良い。GKR ベース・アプローチとは比較不能である。通常、簡潔性は GKR より悪い（回路が比較的深い場合を除く）が、任意の回路と証人長に対して定数オーバーヘッドを達成する。他の 2 つのアプローチと異なり、ILC ベース・システムの隠れた乗法定数はかなり大きいようであり、具体的効率において競争力を持つよう最適化できるかは未解決である。

ゼロ知識に対して定数計算オーバーヘッドを持つ 3 つのアプローチ（GKR、ILC、LPCP）はいずれも、超多項式サイズの体上の算術計算に対してのみ無視可能な健全性誤りを達成できることに注意する。並列 RAM モデルでは、プロセッサ数をわずかに増やす代償で（並列）証明者計算オーバーヘッドを償却するアプローチが SPARKS システム [EFKP20] で提案された。しかし、ブール回路サイズまたは RAM マシン上の逐次実行時間という標準指標を考えると、定数オーバーヘッドのゼロ知識は依然として広く未解決であり、いかなる暗号学的仮定の下でも候補構成は知られていない。

2.3 暗号コンパイラ (CC)

本節では、2.2 節で説明したいくつかの IT システムに対する暗号コンパイラ (CC) を説明する。CC は IT 証明システムを、証明者と検証者の直接的相互作用を含む現実世界のプロトコルへ変換する。CC が用いるプリミティブは、具体的ハッシュ関数やペアリング向き楕円曲線など、（場合によってはヒューリスティックな）具体的インスタンスで実現される。これにより、現実世界の応用で実装・利用できる具体的スキームが得られる。CC はまた、相互作用の除去、一部メッセージサイズの縮小、あるいは ZK 機能を持たない IT 証明システムへの ZK 機能の追加など、望ましい追加性質を提供することもある。

IT 証明システムと暗号コンパイラの分離は有用だが、常に厳密なわけではない。一部の具体的 ZKP スキームでは、それらをそのような構成要素へきれいに分解し、再び組み合わせると元のスキームが得られる、という方法が存在しない。したがって、ここで提示される抽象化と分類は、有用な教育的・分類学的道具と考えられる。一方で、具体的な ZKP スキームについては、工夫や最適化の余地が大きく、具体的性能とパラメータに差異が生じる。

次の小節では、zk-PCP (§2.3.1)、LPCP (§2.3.2)、MPC-in-the-Head (§2.3.3)、IOP (§2.3.4)、LIOP (§2.3.5)、ILC (§2.3.6) に対する暗号コンパイラを説明する。

FF2.11 今後の図案提案: 各主要 IT 証明システム種別のラベルを含む、一般的な暗号コンパイラの図示。

2.3.1 zk-PCP のための CC

例: 3-coloring。上記 2.2.2.1 節の 3-coloring 例 `R_3COL` では、コミットメントが利用可能であると仮定した。このプロトコルを現実世界で実現するため、計算量的仮定の下で安全な暗号学的コミットメント方式に依拠する暗号コンパイラを用いる。このコンパイラは次のように進む。`(x,w)` が与えられると、証明者は zk-PCP 証明者を用いて証明 `π ∈ Σ^m` を生成し、基礎となるコミットメント方式を用いて `m` 個の記号それぞれに独立にコミットする。検証者は zk-PCP 検証者を用いて記号の部分集合 `Q` を選び、チャレンジとして証明者へ送る。証明者は、`Q` が有効（正直な検証者によって生成され得る）であることを確認した後、記号 `π_Q` を開く。検証者は zk-PCP の決定述語 `D` を用いて受理するかどうかを決める。

標準的暗号学的仮定を用いる場合。上記コンパイラは単純かつ自然だが、標準的な計算量的秘匿コミットメント方式 [Gol01] を用いる場合の分析は見かけより繊細である。特に、効率的シミュレーションには、`Q` の分布が多項式サイズの台を持つことが必要である。これは `R_3COL` に対する zk-PCP の基本版には適用されるが、健全性を増幅したものには適用されない。その結果、このコンパイラは健全性の低い（定数ラウンド）ゼロ知識プロトコルを与え、健全性は逐次反復によって増幅できる。多項式サイズ台の制限を避ける別の暗号コンパイラは、統計的秘匿コミットメント方式を用い、さらに繊細な分析を必要とするもので、NP に対する定数ラウンド・ゼロ知識証明の研究 [GK96] に暗黙に含まれる。上記 2 つのコンパイラはいずれも、通信複雑性が `π` を通信するコストより大きいゼロ知識証明プロトコルを与える。理想的には、通信を `π_Q` だけを通信するコストに近づけたい。後に見るように、これは大きな差を生み得る。

統計的束縛性（かつ計算量的秘匿性）を持つコミットメントで具体化すると、上記コンパイラは計算量的健全なアーギュメントではなく統計的健全な証明を与える。この場合、硬い NP 関係に対する証明システムにおいて、証明者から検証者への通信が証人サイズより大幅に小さくなり得ないという強い複雑性理論的証拠 [GH97; GVW02] があるため、高い通信コストは本質的に見える。一方で、別の暗号コンパイラ [IMSX15] は、任意の衝突困難ハッシュ関数を用いて、通信複雑性が `π_Q` のサイズに近いゼロ知識アーギュメントを得ることができる。一部の zk-PCP（後に議論される）では、これは証人サイズよりはるかに小さくなり得る。この種の最初のコンパイラ（Kilian の研究 [Kil92] に暗黙に含まれる）は、任意の PCP、す

なわち ZK 要件を持たない zk-PCP から類似の ZK アーギュメントを得ることができる。ただし、zk-PCP ベースのコンパイラと異なり、Kilian のコンパイラは基礎となる暗号プリミティブを非ブラックボックス的に用いる [RTV04] ため、実用上は非効率である。

ランダムオラクル・モデルにおける実用的 NIZK コンパイラ。これまでのブラックボックス・コンパイラはいずれも、基礎となる「対称」暗号プリミティブをランダムオラクルとして抽象化すれば、無条件に分析できる。しかし実際にはさらに進んで、Fiat-Shamir 変換 [FS87] により、これらの対話型公開コイン・プロトコルを非対話型にできる。2 段階を組み合わせると、ランダムオラクル・モデルで任意の zk-PCP から NIZK への具体的に効率的なコンパイラが得られる。後者は、たとえば SHA-256 や AES に基づく実用的ハッシュ関数でヒューリスティックに具体化される。この方法論のさらなる議論については背景部分を参照されたい。興味深いことに、ランダムオラクル・モデルであっても、NIZK は基礎となる対話型プロトコルを完全に支配するわけではない。相互作用の除去には代償が伴うことがあるからである。たとえば、健全性誤り 2^{-σ} を持つ対話型プロトコルを考える。σ は 1.8.3 節で議論したように 40 または 64 のオーダーであり得る。Fiat-Shamir 変換で得られる NIZK では、証明者は約 2^σ 個のランダム・トランスクリプトを生成し、検証者が受理するものを見つけるまで試すことで、偽の命題について確実に検証者を納得させられる。これは、非対話型変種が、かなり小さい健全性誤り、すなわち計算量的セキュリティ・パラメータ κ に等しいより高い σ を持つ zk-PCP に依拠すべきであることを意味する。これにより具体的な通信・計算コストは増加するが、その増加は小さな因子（約 κ/σ）である。

標準暗号学的仮定からの NIZK。最近の一連の研究は、Fiat-Shamir 変換で得られた NIZK 証明におけるランダムオラクルを、標準的暗号学的仮定の下で具体化する方法を示している。これらの研究は、特殊な種類の相関困難ハッシュ関数 [CGH04] と、特殊な種類の Σ-protocols [Dam10] に依拠する。後者は、ある追加性質を満たす 3 メッセージの正直な検証者に対する（計算量的）ゼロ知識証明システムである。さらにこれは、Blum のグラフ Hamiltonicity プロトコル [Blu87] で具体化できるが、より体系的な探索が残されている特殊な zk-PCP に暗黙に依拠する。これら最近の NIZK プロトコルと対照的に、標準的暗号学的仮定に基づく NP 用 NIZK の古典的アプローチ [FLS99] は、hidden bits model におけるゼロ知識証明と呼ばれる非常に異なる種類の情報理論的証明システムを用いる。この種類の既知の証明は、zk-PCP 対応物より複雑で非効率である。しかし hidden bits model から NIZK への暗号コンパイラは、素因数分解の困難性、またはより一般に適切な種類のトラップドア置換 [CL18] に依拠できる。同様の暗号コンパイラを zk-PCP から得られるかどうかは興味深い問題である。

2.3.2 Linear PCP (LPCP) のための CC

Linear PCP 用の暗号コンパイラは、高価だが再利用可能な信頼されたセットアップと、「効率的に反証可能ではない」暗号学的仮定 [Nao03] を代償として可能である。これらのコンパイラは、非常に短い証明（約 1000 ビット、ある設定ではそれ未満）を持つ zk-SNARK を生み得る。この種のコンパイラの使用は初期スキーム [Gro10; Lip12] に暗黙に含まれ、後に明示的に考慮された [BCIOP13; GGPR13]。

まず、LPCP を再利用可能な構造化参照文字列 (SRS) を持つ指定検証者 SNARG へコンパイルするための自然な試みを考える。検証者は LPCP クエリ q_1, ..., q_d（たとえば Hadamard LPCP の場合 d=3）を生成し、SRS σ に検証者クエリの（各成分の）線形準同型暗号化 E(q_1), ..., E(q_d) を含める。すなわち σ = (E(q_1), ..., E(q_d)) である。検証者は復号に用いる秘密鍵 k_V を保持する。

入力 (x,w) と σ が与えられると、証明者は LPCP 証明ベクトル π を計算し、E の線形準同型性を用いて、d 個の暗号文 π_hat = (E(<π,q_1>), ..., E(<π,q_d>)) からなる短い SNARG 証明を計算し、検証者へ送る。検証者は秘密鍵 k_V を用いて LPCP 回答 a_1 = <π,q_1>, ..., a_d = <π,q_d> を復号し、LPCP 決定述語を適用して受理するかどうかを決める。

上記の準同型暗号の内側に LPCP を実装する試みは、明らかに完全性要件を満たす。さらに、暗号化がクエリを隠すため、証明 π はクエリから独立であり、したがって健全性も成立すると考えたくなる。しかし、この単純な健全性議論には 2 つの理由で欠陥がある。

第一に、標準的な線形準同型暗号方式 E は暗号化された入力上で線形関数を計算することをサポートするが、線形関数だけが計算できるという保証は提供しない。実際、線形準同型暗号は完全準同型になり得る [Gen09]。その場合、健全性は完全に破綻する。この問題の解決は、本質的には「仮定によって排除する」ことであり、暗号化入力上で線形計算しかサポートしないと予想される暗号方式 E に依拠することである。あるいはこれは、定数項を含むアフィン計算へ拡張できる。この強い「linear-only encryption」概念 [BCIOP13] は、以下でより詳細に議論される。

第二の問題は、上記解法には、悪意ある証明者が各暗号化クエリ E(q_i) に対して異なる証明ベクトル π_i* を用いることを妨げるものがないことである。これは LPCP モデルにおける悪意ある証明者の能力を超えており、健全性を破り得る。この問題への解決は、検証者が LPCP に追加クエリとして、元のクエリのランダム線形結合 q_{d+1} = ρ_1 q_1 + ... + ρ_d q_d を加え、このクエリへの回答が a_{d+1} = ρ_1 a_1 +

... + ρ_d a_d を満たすことを確認することである。これは、LPCP から 1 ラウンド Linear Interactive Proof (LIP) [BCIOP13] への単純な IT コンパイラと見なせる。LIP は標準的対話型証明を、証明者（正直・悪意の双方）が検証者メッセージの線形関数しか計算できないよう制限する、より強い情報理論的証明システムである。

要約すると、修正コンパイラは 2 段階で進む。第一に、情報理論的コンパイラを適用して LPCP を LIP へ変換する。LPCP の証明サイズが m、クエリ数が d である場合、上記単純コンパイラで得られる LIP は、検証者メッセージが (d+1)·m 個の体要素からなり、証明者メッセージが d+1 個の体要素からなる。第二に、linear-only encryption を用いて LIP を指定検証者 SNARG へコンパイルする。SRS は (d+1)·m 個の暗号文からなり、証明は d+1 個の暗号文からなる。この変換は、Hadamard LPCP の文脈で議論した LPCP の追加特徴、すなわちゼロ知識性、高速検証、再利用可能健全性をすべて保つ。後者は、得られる SNARG の SRS を複数証明に安全に再利用できることを意味する。最後に、LPCP が「知識の証明」でもあり（自然な構成ではすべてそうである）、linear-only encryption が「抽出可能」である（具体的インスタンスに対してもっともらしい仮定）場合、得られる (zk)-SNARG も知識の証明であり、すなわち (zk)-SNARK である。

残る問題が 1 つある。上記アプローチは、指定検証者設定に本質的に限定されるように見える。暗号化された LIP 回答を復号できるのは検証者だけだからである。これは一部の応用には十分だが、SNARG の多くの応用は公開検証を必要とする。解決策（Groth の研究 [Gro10] に初めて暗黙に含まれた）は、LPCP の特殊性質に再び依拠することであり、その性質は LIP への変換でも保たれる。LIP 検証者の決定述語が次の意味で二次であると仮定する。受理するかどうかを決めるため、検証者は p_x(u,a)=0 の形の等式を検査する。ここで p_x は入力命題 x によって決まる次数 2 多項式、ベクトル u は LIP クエリによって決まる状態情報、ベクトル a は LIP 回答を含む。実際、これは Hadamard ベース LIP の場合に当てはまる。すると、復号鍵を知らなくても暗号化入力上でそのような二次検査を実行できる暗号方式を用いることで、公開検証を達成できる。幸い、この種の機能はペアリングベース暗号によってサポートされる。SRS σ が、双線形群を用いて実装できる「pairing-friendly encryption」による LIP クエリおよび状態情報 u の暗号化を含むなら、証明者は入力 (x,w) に対して LIP 回答 a の暗号化を計算でき、その後、誰もが暗号化された u,a が x で定義される二次関係を満たすことを確認できる。

いくつかの QAP ベース構成（たとえば [GGPR13; Gro16]）は、おおまかには Linear PCP モデルにおける IT 証明システムとして再構成でき、その後 [BCIOP13] などで暗号学的にコンパイルされる。結果として得られる証明システムは、定数因子で異なる簡潔性を持つ。

2.3.3 MPC-in-the-Head のための CC

貢献募集: 貢献が必要である。

2.3.4 IOP のための CC

一方向関数に基づく暗号コンパイラ [BGGHKMR90] を用いると、2.2.5.1 節の Interactive PCP は、通信複雑性が証人長に匹敵する低深さ（または空間制限）NP 関係に対する統計的健全 ZKP プロトコルをもたらす。

古典的 PCP の場合と同様、衝突困難ハッシュ関数（または古典的、あるいは量子的ランダムオラクル）[CMS19] を用いて、完全簡潔 IOP を NP に対する対話型（または非対話型かつ透明）完全簡潔アーギュメントへ変換する暗号コンパイラがある。これらのコンパイラは、古典的 PCP 用のものを自然に拡張する。対話型変種では、証明者が Merkle 木を用いて各証明へ簡潔にコミットし、その後、検証者が次の証明のためのランダムチャレンジを生成して送る。最後に、検証者は IOP 検証者が問い合わせたい証明記号の部分集合を開くよう証明者にチャレンジし、証明者は Merkle 木に関する少量の関連情報を明らかにして応答する。非対話型変種は、Fiat-Shamir ヒューリスティックにより対話型変種から得られる。ただし、ランダムオラクル・モデルにおける分析は対話型の場合かなり難しく、基礎となる IOP にいくつかの追加仮定を必要とする。

実用面では、STARK [BBHR18b]、Aurora [BCRSVW19]、Fractal [COS20] などの透明 SNARK は、具体的に効率的な IOP に依拠する。これらは次に議論する群ベース SNARK ほど簡潔ではないが（典型的な証明サイズは前者で 100-200 kB、後者で 1-2 kB）、透明であり、ポスト量子安全性がもっともらしく、公開鍵暗号の使用を完全に避けるという利点を持つ。最後の点は、より高速な証明者を可能にするため有用である。これらの実用的 IOP の重要技術要素は FRI プロトコル [BBHR18a] である。FRI は、オラクルが Reed-Solomon 符号に近いかを検査する対話型テストである。FRI プロトコルの有用な特徴は、厳密に線形個の算術演算を用いて実現できることである。しかし、その上に構築される IOP は、証明者側で依然として準線形時間を必要とする。

理論面では、IOP モデルは古典的 PCP で達成されることが知られていない漸近効率特徴を生み出す。IOP の証明サイズを最小化する最近の進展については [BCGGRS19; RR20] を参照。IOP の証明サイズは、それ

が証明者実行時間の下界としてのみ機能するため、IOP の暗号応用において主要関心パラメータではない。しかし、IOP 構成の新技術は、IOP ベース証明システムの具体的効率に関する進歩につながる可能性が高い。IOP の具体的効率により直接関連するのは、健全性誤り分析の改善に関する研究 [BKS18; BGKS20; BCIKS20] である。

2.3.4.1 Interactive PCP のための CC　貢献募集。

2.3.5 Linear IOP のための CC

貢献募集: Linear IOP（fully linear 以外）/ IP ベースのための CC。

2.3.5.1 Fully Linear IOP のための CC　証明者が各証明ベクトルを秘密分散することで、fully linear proof system は [BBCGI19] により、IT 証明者が生成する証明全体の長さ（すなわち線形クエリされる π）および線形クエリへの回答長によって通信複雑性が支配される分散ゼロ知識証明へコンパイルできる。分散ゼロ知識用のコンパイラは情報理論的であり、多項式 IOP が提供する追加構造や検証者の決定述語の低い代数次数を利用しない。fully linear IOP と異なり、plain LIOP では支配的通信コストが IT 証明者の証明長である可能性は低いことに注意する。たとえば証明長は、証明をハッシュまたは準同型暗号で「圧縮」できるため、通信に影響しない場合がある。

2.3.5.2 PIOP のための CC: 多項式コミットメント　より良い暗号コンパイラの鍵は、GKR ベース LIOP の次の追加特徴である。各証明ベクトル π_i は少数変数の多項式の係数を定義するものとして見なせ、π_i への各線形クエリは単一点でその多項式を評価する。より具体的には、GKR ベース LIOP の長い証明ベクトルは、(x,w) を符号化する log m 変数の多重線形多項式を定義する。

これは、証明を有界次数多項式の係数ベクトルとして解釈し、通常は高々対数個の少数変数において、クエリを評価点として解釈する、より洗練された LIOP の概念で捉えられる。この LIOP の洗練は Polynomial IOP (PIOP) [BFS20] と呼ばれ、追加構造を利用する暗号コンパイラの可能性から動機づけられる。密接に関連するモデルの別名に Algebraic Holographic Proof [CHMMVW20] がある。PIOP の暗号コンパイラは、多項式コミットメントの効率的実現に依拠する。これは、与えられた点における評価の効率的証明をサポートする仕方で、証明者が多項式へコミットできる機能的コミットメント方式である。

GKR プロトコルと多項式コミットメントを組み合わせた最初のシステムは Hyrax [WTTW18] と (zk)-vSQL [ZGKPP17] であった。Hyrax は、証明サイズが約 d + √m となる代償で、離散対数に基づく透明な多項式コミットメント実装を用いた。vSQL システムは、ペアリング向き群に依拠し、信頼されたセットアップを必要とする広く用いられる多項式コミットメント方式 [KZG10] の変種を用いることで、√m の加法項を除去できた。その後の Libra システム [XZZPS19] は、類似の多項式コミットメントと、改良された GKR ベース PIOP のゼロ知識変種を組み合わせた。Virgo [ZXZS20] は Libra の変種であり、対称暗号のみに基づく効率的で透明な多項式コミットメントを用い、FRI プロトコルに基づく IOP と m = O(log s) の GKR ベース LIOP を組み合わせる。類似の多項式コミットメントは RedShift [KPV19] でも提案された。別の提案 [BFS20] は、未知位数群を用いて、はるかに良い簡潔性を持つ透明な多項式コミットメント方式を達成するが、具体的証明者複雑性が高く、ポスト量子安全ではないという代償がある。その後の提案 [Lee20] は、類似の多項式コミットメント方式を与え、クラス群の使用を避け（対称外部 Diffie-Hellman 仮定に基づく）、高い証明者コストを緩和した。PIOP と多項式コミットメントを組み合わせて簡潔なアーギュメントを得る追加研究には、以下が含まれる。Kopis と Xiphos [SL20] は、Spartan [Set20] に暗黙に含まれる R1CS 用 PIOP を多項式コミットメント Dory [Lee20] およびその変種と組み合わせる。Cerebrus [LSTW21] は Ligero [AHIV17] で与えられた IOP から多項式コミットメント方式を抽出し、同じ Spartan [Set20] の PIOP と組み合わせる。

2.3.6 ILC のための CC

衝突困難ハッシュ関数を用いると、ILC 証明システムは、通信複雑性がおおよそ Σ_i (n_i + m_i) である、プレーン・モデルにおける公開コイン・アーギュメントへコンパイルできる [BCGGHJ17]。

2.4 特殊化された ZK 証明

本節は特殊目的言語、すなわち NP 全体を扱わない証明に対する ZKP を考える。例には、離散対数の知識や RSA モジュラス N の因数分解の知識の証明が含まれる。

貢献募集。

2.5 証明合成

再帰的合成により、「証明の証明の証明」などが存在することを証明できる。これは、効率改善、機能向上、機械計算を証明する能力 [BCTV14a] など、複数の利点を持ち得る。たとえば、再帰的ゼロ知識証明を用いれば、不特定サイズの while ループを持つ機械計算が満たされることを理論的に証明でき、さらに計算の進行に応じて証明を逐次更新できる。これは incrementally-verifiable computation [Val08] の概念である。再帰的証明は、進行中の分散計算の完全性、すなわち proof-carrying data [CT10; CTV15] の概念も証明できる。これにより、中間ノードおよび最終検証者は、他の当事者によって行われた先行する計算部分がすべて正しいことを保証される。

FF2.12 今後の図案提案: 再帰的 ZK 証明の図示。

ブロックチェーン領域では、これらの技術はいくつかの応用を見出した。第一に、ブロックチェーンの現在状態の妥当性は、これを incrementally-verifiable computation と見なすことで簡潔に証明できる。したがって、システムへの新規参加者は、Mina（旧 Coda）[BMRS20] のように、1 つの再帰的証明を検査するだけで状態が正しいことを確信できる。第二に、単一取引内であっても、複数の証明を再帰的に合成することで、取引サイズと検証時間の簡潔性を達成できる。これは Zcash Orchard [HGBNL21] で用いられている。

再帰的証明を構成するアプローチには、zk-SNARK の再帰的合成 [Val08; CT10; BCCT13; BCTV14a]、および accumulation / split-accumulation schemes [BGH19; BCMS20; COS20; BDFG20] への一般化が含まれる。

貢献募集: 構成アプローチおよび追加応用について、より詳細な説明が必要である。

2.6 対話性

2.2 節で説明された証明システムのいくつかは対話型であり、古典的対話型証明 (IP)、IOP、Linear IOP が含まれる。これは、検証者が複数のチャレンジ・メッセージを証明者へ送り、証明者はチャレンジ i+1 を受け取る前にチャレンジ i へ応答することを意味する。健全性は、証明者がチャレンジ i へ応答するとき、チャレンジ i+1 を予測できないことに依拠する。他の証明システム、すなわち古典的 PCP と Linear PCP は非対話型である。これらすべての証明システムは、暗号コンパイラ（2.3 節参照）と組み合わせることでアーギュメント・システムを得られ、そのシステムが対話型かどうかはコンパイラに依存する。

2.6.1 対話型証明・アーギュメント・システムの利点

1. **効率と単純性**。対話型証明システムは、非対話型のものより単純または効率的であり得る。例として、研究者は IOP モデル [BCS16; RRR16] を導入したが、これは部分的には、対話性によって最先端 PCP 構成 [BCGT13] における効率ボトルネックを回避できたためである。別の例として、IP から派生した一部のアーギュメント・システム [WTTW18; XZZPS19] は、最先端 PCP [BCGT13] または Linear PCP [GGPR13; Gro16] より、証明者の空間複雑性（重要なスケーラビリティ・ボトルネック）が大幅に良い。

   それでも、対話型プロトコルが公開コインであれば、多くの設定では Fiat-Shamir 変換（2.3 節参照）により、効率をほとんど失うことなく非対話型かつ公開検証可能にできる。これは、プロトコル設計者が、プロトコル設計の単純化、効率改善、信頼されたセットアップの弱化または除去などのため、対話性を「資源」として活用する自由を持ち、それでも Fiat-Shamir 変換を用いて非対話型アーギュメントを得る選択肢を持つことを意味する。

   対話型プロトコルに Fiat-Shamir ヒューリスティックを適用して非対話型アーギュメントを得ると、健全性誤りが増えることがあり、統計的セキュリティを計算量的セキュリティへ変換することがある（1.8.3 節参照）。しかし最近の研究 [BCS16; CCHLRRW19] は、実用的および理論的に関心のある特定の IP、IOP、Linear IOP プロトコルにこの変換を適用すると、健全性誤りの増加は相互作用ラウンド数の多項式にすぎないことを示している。

2. **セットアップ**。 Linear PCP 用の暗号コンパイラは現在、構造化参照文字列 (SRS) を必要とする（3.6.2 節参照）。ここで SRS は、セットアップ段階中に信頼された第三者によって生成されなければならない構造化文字列であり、健全性には、この信頼されたセットアップ中に使用されたトラップドアが明らかにされてはならないことが必要である。対照的に、IP、IOP（および PCP）、Linear IP に適用される一部のコンパイラは、証明者と検証者が一様ランダム文字列 (URS) にアクセスするだけでよいアーギュメントを与える。URS は共通乱数源から取得できる。このようなセットアップは文献で透明セットアップと呼ばれる。

3. **暗号プリミティブ**。IP、IOP、Linear IOP から派生したアーギュメント・システムは、より望ましい暗号プリミティブに依拠する場合もある。たとえば IP そのものは情報理論的に安全であり、暗号学的仮定に依拠しない。また、Linear PCP から派生するアーギュメントと対照的に、IOP から派生するものは対称鍵暗号プリミティブのみに依拠する（例 [BCS16]）。衝突困難ハッシュ族のような反証可能仮定に基づき、プレーン・モデルで簡潔な対話型アーギュメントを得る方法も知られている [Kil95] が、簡潔な非対話型アーギュメントについてはこれは知られていない。
4. **非移転可能性**。一部の応用では、証明が否認可能または非移転可能であること、すなわち検証者が第三者に命題の妥当性を納得させられないことが不可欠である（1.6.6 節参照）。これらの性質は対話型プロトコルに固有ではないが、対話は証明を非移転可能にする自然な方法を提供する（詳細は 2.6.3 節参照）。
5. **対話性は敵対者の能力を制限し得る**。対話型プロトコルは、より少ないセキュリティ・ビット数で実行でき、したがってより効率的になり得る。たとえば対話型設定では、プロトコル終了に時間制限を強制し、攻撃者の実行時間を制限できる場合がある。あるいは対話型設定では、敵対者がプロトコルを攻撃する試行を 1 回だけにすることが可能な場合がある。一方、多くの非対話型設定ではこれは不可能である。詳細は 1.8.2 節を参照。
6. **対話性が応用に本質的であり得る**。多くの応用は本質的に対話的である。たとえば現実世界のネットワーク・プロトコルは、接続を開始するだけでも複数メッセージを伴う。さらにゼロ知識プロトコルは、応用においてしばしば oblivious transfer など他の暗号プリミティブと組み合わされる。他のプリミティブが対話型であれば、ゼロ知識プロトコルが非対話型かどうかにかかわらず、最終的な暗号プロトコルは対話型になる。応用が本質的に対話型であるなら、対話を資源として活用してプロトコルを単純化、効率化できる場合、それは合理的であり得る。

2.6.2 対話型証明・アーギュメント・システムの欠点

1. **対話型プロトコルはオンラインで行われなければならない**。対話型プロトコルでは、非対話型プロトコルのように、証明を単に公開または投稿して、後で検証者の都合で検査することはできない。
2. **公開検証可能性**。多くの応用では、任意の当事者が任意の時点で証明を検証できることが必要である。対話型プロトコルで公開検証可能性を達成することは難しい場合がある。これは、対話型プロトコルの健全性が、証明者が次に受け取るチャレンジを予測できないことに依拠するからである。公開的に信頼された予測不能乱数源（ランダムネス・ビーコンなど）と、証明者がメッセージにタイムスタンプを付ける方法がない限り、チャレンジを送る当事者以外の者が、チャレンジが適切に生成され、証明者がチャレンジ i+1 を知る前にチャレンジ i へ応答したことを納得する方法は明らかでない。詳細は下記 2.6.3 節を参照。
3. **ネットワーク遅延は対話型プロトコルを遅くし得る**。対話型プロトコルがネットワーク越しに送られる多数のメッセージから構成される場合、ネットワーク遅延がプロトコル全体の実行時間に大きく寄与することがある。
4. **タイミング攻撃またはサイドチャネル攻撃**。対話型プロトコルでは証明者が複数メッセージを送る必要があるため、非対話型プロトコルに比べてサイドチャネル攻撃やタイミング攻撃への脆弱性が増える可能性がある。公開コイン・プロトコルでは、検証者のメッセージが単なるランダムコインであり、タイミング攻撃がこの場合に証明者へ情報を漏らすべきではないため、タイミング攻撃は健全性ではなくゼロ知識性にのみ影響する。秘密コイン・プロトコルでは、これらの攻撃がゼロ知識性と健全性の双方に影響し得る。
5. **並行セキュリティ**。対話型プロトコルが単独で使われるのではなく、複数の対話型プロトコルが並行実行され得る環境で使われる場合、そのプロトコルが安全に保たれるよう相当な注意が必要である。たとえば [Gol13, Section 2.1] およびその参照文献を参照。並行実行セキュリティの問題は、非対話型プロトコルでは大きく緩和される [GOS06]。
6. **証明長**。現在知られている最短の証明を持つゼロ知識プロトコルは、非対話型である Linear PCP に基づく。これらの証明は数個の群要素にすぎない。公開コインのゼロ知識プロトコルは Fiat-Shamir ヒューリスティックで非対話型にできるが、IP または IOP に基づくものは現在より長い証明を生成する。より長い証明は一部の応用（たとえば公開ブロックチェーン）には不適切となり得るが、他の応用（エンタープライズ・ブロックチェーンなど関連するものを含む）には適切であり得る。

2.6.3 移転可能性と対話性の微妙な関係

対話性と移転可能性／否認可能性の関係は単純ではない。以下の段落は、いくつかの可能な組合せを示す。

非対話型かつ否認可能。非対話型 ZKP は非移転可能であり得る。これは、たとえばそれ自体が否認可能であるローカル CRS のようなセットアップ仮定に基づくことがある。この場合、悪意ある検証者は、外部当事者に対して、その CRS が実際のプロトコル実行で用いられたものだったことを証明できない。そのため外部当事者は、検証者が CRS をシミュレートし、正当な証明者の実際の参加なしにプロトコル実行トランスクリプトをシミュレートできるようにしたのではないかと合理的に疑う。非移転可能性の別例は、ZKP が (i) ある主張（メンバーシップまたは知識）を証明することを意図しているが、実際にはそれと (ii) 指定検証者の秘密鍵の知識との選言を証明する場合である。これは、たとえば公開鍵基盤 (PKI) を仮定して実現できる。これは、元の検証者に対して、初期命題 (i) が真であることを納得させるのに十分である。なぜなら検証者は、証明者が実際には秘密鍵 (ii) を知らないことを知っているからである。言い換えれば、対話型証明の成功は初期主張 (i) の真実性に由来する。しかし外部当事者にとっては、証明トランスクリプトは元の指定検証者が秘密鍵 (ii) の知識を用いて生成したものかもしれない。その意味で、指定検証者は正当な証明のトランスクリプトが検証者によってシミュレートされたものではないと他者に納得させられない。

非対話型かつ移転可能。移転可能性が特徴として意図されるなら、非対話型プロトコルは公開（否認不能）CRS で達成できる。たとえば CRS が信頼されたランダムネス・ビーコンによって生成され、健全性が証明者が CRS を制御できないことから従うなら、任意の外部当事者（証明生成時に証明者と関与していなかった者でさえ）は、後で、証明トランスクリプトが正当な証明者によってのみ生成され得たことを検証できる。

対話型かつ否認可能。単独実行設定（並行実行なし）で否認可能性を得る古典的例は、巻き戻しを用いることに基づき安全性が証明された対話型 ZKP プロトコルから来る。ここで否認可能性は、任意の悪意ある検証者に対してトランスクリプトをシミュレート可能であることから従う。各対話ステップで、シミュレータは悪意あり得る検証者が発行したチャレンジを学び、その後、証明者の先行メッセージを再選択するために巻き戻すことで、その後のチャレンジに答えられるようにする。いくつかの技術はコミットメントまたはトラップドアの使用を必要とし、適切な信頼されたセットアップがあれば、巻き戻しなしの直線的シミュレーションでもこの性質を可能にすることがある。

対話型かつ移転可能。ある設定では、対話型 ZKP プロトコル実行からでも、移転可能な証明を構成するトランスクリプトを生成できる。通常、移転可能性は、（悪意あり得る）検証者が、プロトコル実行中に選ばれたチャレンジが証明者の先行メッセージ決定時点で予測不能であったことを外部当事者に説得力を持って示せる場合に達成できる。移転可能な証明トランスクリプトは、証明者が送ったメッセージと、悪意ある検証者の内部状態からの追加情報、チャレンジ生成の詳細を含んで構成される。たとえば、検証者が以前のメッセージの暗号学的ハッシュ出力（または鍵付き疑似ランダム関数）として生成したチャレンジは、後で、正当な証明者だけが有効な後続メッセージ（応答）を生成できたという保証を提供するために用いられ得る。別例として、対話型 ZKP プロトコルが、証明者が送信メッセージすべてを認証する通信プロトコル（たとえば PKI 内で署名され、信頼されたサービスによってタイムスタンプ付けされる）と合成される場合、その認証メッセージ列全体は検証者の手元で移転可能な証明となる。さらに、移転可能なトランスクリプトから、実際の移転も対話的に実行できる。すなわち、トランスクリプトを保持する検証者が、移転可能なトランスクリプトの知識の移転可能 ZKP における証明者として振る舞い、外部検証者に新しい移転可能トランスクリプトを移転する。

2.6.3.1 ゼロ知識証明の（非）移転可能性／否認可能性　ZK 証明のオフライン非移転可能性（否認可能性）。ゼロ知識証明は一般に対話型である。セットアップがない場合、相互作用は本質的である。実際、Goldreich と Oren は、非自明な言語についてゼロ知識証明には少なくとも 3 ラウンドが必要であることを示した。

セットアップがない場合、ZK 性はオフライン非移転可能性、否認可能性としても知られる性質を保証する。検証者は、シミュレータを実行することで同等のトランスクリプトを常に計算できることに注意する。これは、検証者が証明者から受理される証明を受け取ったという証拠を得ないため、受け取った証明を他者へ移転する上で優位性を持たないことを意味する。

ZK 証明のオンライン非移転可能性。オンライン非移転可能性の場合、状況はより複雑である。この場合、悪意ある検証者は正直な証明者とゼロ知識証明システムを実行しながら、同時に、受け取っている証明を移転しようとして他者とやり取りする。したがって非移転可能性は中間者攻撃に対するセキュリティの一形態である。そのような攻撃に対するセキュリティは、同じゼロ知識証明システムが敵対者によって正直な検証者へ証明を移転するために用いられる場合、通常は非可鍛性と呼ばれる。敵対者の活動の一部として異なるプロトコルが関与する場合、そのような攻撃の下でのセキュリティをモデル化するには、より強い概念（たとえばユニバーサル合成可能性）が必要となる。

NIZK 証明の移転可能性: 公開検証可能 ZK。何らかの形のセットアップが考慮され、ゼロ知識証明がそれを本質的に利用する場合、ゼロ知識証明の移転可能性は避けられなくなり得る。実際、共通参照文字列モ

デルとプログラム可能ランダムオラクル・モデルの双方で、非対話型ゼロ知識証明を構成できることに注意する。そのような証明は、同じセットアップまたは同じランダムオラクル具体化を用いて検証者がシミュレートすることはできない。より具体的には、非対話型ゼロ知識証明は検証者からの寄与なしに構成されるため、検証者間で自然に移転可能な公開検証可能証明である。

指定検証者 NIZK 証明。より洗練されたセットアップでは、他の選択肢が可能になる。たとえば、公開鍵によって実装された公開アイデンティティを持つ検証者を考える。この場合、証明者は検証者の公開鍵を本質的に用いる非対話型ゼロ知識証明を計算でき、対応する秘密鍵を用いる検証者も識別不能な証明を計算できる。この場合、その証明は非対話型指定検証者ゼロ知識証明であり、非移転可能である。なぜなら、その証明を受け取った検証者は同等の証明を自分自身で計算できたため、その証明が正直な証明者から来たという事実について他者と共有できる証拠がないからである。

対話型 ZK 証明の移転可能性。公開鍵で実装されたアイデンティティの使用は、対話型の場合にも影響を与え得る。たとえば、移転可能性の風味を持つ対話型 ZKP システムを設計できる。証明者がトランスクリプトに署名すれば、検証者は、元の証明者が正直であったと信じる者に対して、その「証明」を移転できる。

第 3 章 実装

3.1 概要

ZKP の実装に関する標準またはフレームワークを持つことにより、プラットフォームは新しい構成や新しいスキームへより容易に適応できるようになる。そのような構成やスキームは、効率、セキュリティ、または応用固有の変化により、より適している場合がある。アプリケーション開発者と新しい証明システムの設計者は、ともに、さまざまな応用で呼び出される異なる ZKP 構成の性能とセキュリティのトレードオフを理解したい。本トラックは、アプリケーション開発者が ZKP 証明システムと相互作用するために使用できる標準インターフェースを構築することに焦点を当てる。これは、相互運用性、柔軟性、性能比較を促進するための取り組みである。このようなインターフェースを実現する最初の取り組みでは、R1CS/QAP 型の制約系表現を用いる一般命題 (NP) 用の非対話型証明システム (NIZK) に焦点を置く。これには、現在配備されている多くの（ただしすべてではない）実用的な汎用 ZKP スキームが含まれる。この焦点により、相互運用性のための具体的な形式を定義できる。一方で、算術回路やブール回路など、追加の制約系表現スタイルが利用されており、継続中の取り組みの範囲内にあることを認識している。また、本番ソフトウェアにおけるこれらの証明システム配備のベストプラクティスを確立することも目指す。

FF3.1 今後の図案提案: フロントエンド、バックエンド、中間表現 (IR) の一般的な実装パラダイムを示し、データ／入力／出力の流れを矢印で示す図を追加する。

3.1.1 本章が扱わないこと

- ZKP アプリケーションの構築方法に関する唯一の説明。
- ZKP システム構築に必要なセキュリティ要件の網羅的一覧。
- フロントエンド・ツールの比較。
- あるユースケースを別のユースケースより優先する表示。

3.2 バックエンド: 暗号システム実装

ZK 証明実装のバックエンドとは、低水準の暗号プロトコルの実装を含むソフトウェア部分である。バックエンドは、インスタンスと証人が変数割当てとして表現され、関係が低水準言語（算術回路、ブール回路、R1CS/QAP 制約系、算術制約充足問題など）を通じて表現される命題を証明する。

FF3.2 今後の図案提案: バックエンドの各コンポーネントを「表階層」として示す図を追加する。

バックエンドは通常、提案された ZK 証明システム（多くの場合、論文出版物の擬似コードで指定される）の具体的実装と、必要な算術演算、シリアライゼーション形式、テスト、ベンチマークなどを支えるコードからなる。バックエンドは多数存在し、その多くは学術研究プロトタイプとして始まり、オープンソース・プロジェクトとして利用可能である。バックエンドの提供機能と特徴は急速に進化するため、整理された分類表 [ZKP.Science] を参照すると有用である。

バックエンド選択の考慮事項には以下が含まれる。

- バックエンドが実装する ZK 証明システム、およびそれに関連するセキュリティ、仮定、漸近性能（セキュリティ・トラック文書で議論）。
- 具体的性能（ベンチマーク節参照）。
- プログラミング言語および API スタイル（将来の ZK 証明標準への準拠により満たされ得る。API およびファイル形式の節を参照）。
- プラットフォーム・サポート。
- オープンソースとしての利用可能性。
- 保守者および利用者の活発なコミュニティ。
- 実装の正確性と堅牢性（監査や形式検証などで判断）。
- 応用（使いやすさと精査の証拠として）。

3.3 フロントエンド: 制約系の構築

ZK 証明システム実装のフロントエンドは、命題を便利な言語で表現し、それらを低水準表現へコンパイルして適切な ZK バックエンドを呼び出すことにより、ゼロ知識で証明する手段を提供する。

フロントエンドは以下からなる。

- 命題を表現する高水準言語の仕様。

- 高水準言語で表現された関係を、何らかのバックエンドに適した低水準関係へ変換するコンパイラ。たとえば R1CS 制約系を生成し得る。
- インスタンス還元: 高水準命題のインスタンスを低水準インスタンスへ変換すること（たとえば R1CS インスタンス変数への割当て）。
- 証人還元: 高水準命題の証人を低水準証人へ変換すること（たとえば証人変数への割当て）。
- 通常、命題の有用で手作業により最適化された構成要素からなる「ガジェット」ライブラリ。

FF3.3 今後の図案提案: どの入力が秘密／公開（証人 vs インスタンス）であるか、およびガジェットをどのように合成できるかを示す、例示的または抽象的回路図を追加する。

これまでフロントエンドで実装されてきた命題表現言語には、汎用言語用のコードライブラリ、ドメイン固有言語、ZKP 用に適応された汎用高水準言語、仮想 CPU 用のアセンブリ言語が含まれる。

フロントエンドのコンパイラおよびガジェット・ライブラリは、しばしば制約系コスト（制約数や変数数など）を削減する最適化を実装する。これには、R1CS における「無料の線形結合」の利用、証人変数として与えられる非決定性アドバイスの利用（整数算術やランダムアクセスメモリなど）、冗長性除去、与えられた代数設定に合わせた暗号スキームの使用（ハッシュ関数について Jubjub 曲線上の Pedersen ハッシュや MiMC、デジタル署名について RSA 検証など）、その他多くの技術が含まれる。たとえば Zcon0 Circuit Optimisation handout [Hop18] におけるさらなる議論を参照。

実装済みフロントエンドは多数存在し、その中には同じ基礎バックエンドを呼び出す代替的な方法を提供するものもある。多くは学術研究プロトタイプとして始まり、オープンソース・プロジェクトとして利用可能である。フロントエンドの提供機能と特徴は急速に進化するため、整理された分類表 [ZKP.Science] を参照すると有用である。

3.4 API およびファイル形式

我々の主要な目標は、証明システムと、証明システム実装のフロントエンド利用者との相互運用性を改善することである。実装の標準インターフェースを構築するため、2 つのアプローチに焦点を当てた。

1. 証明システムが、その能力をフロントエンドへ公開するための共通 API の開発を目指す。この API は、基礎実装の詳細からできるだけ独立であるべきである。
2. 証明システム実装とフロントエンドが、言語および API の障壁を越えて相互作用できるよう、一般的な制約系形式（具体的には R1CS）およびその割当てを符号化するファイル形式の開発を目指す。

同じ抽象スキームを実装するバックエンド同士の相互運用性、たとえば証明のシリアライゼーション形式の標準を開発することは目指さなかった（さらなる議論は拡張制約系相互運用性の節を参照）。

3.4.1 汎用 API

証明システム実装の性能と使いやすさのトレードオフを比較しやすくするため、フロントエンド・アプリケーション開発者は、基礎となる証明システムと汎用インターフェースを通じて相互作用したい場合がある。そうすれば、証明システムを入れ替え、トレードオフを実践上観察できる。これは学術的な分析・比較にも役立つ。

図 3.1 は、NIZK に関与する当事者とオブジェクトの抽象化を示す。内容としては、`Gen` が公開パラメータを生成し、証明者が証人とインスタンスおよび言語に基づいて証明を作成し、検証者がインスタンスと言語に基づいて受理または拒否を出力する流れが示されている。

汎用 API 設計は完了していないが、将来的価値があり得る多数のトレードオフおよび設計アプローチを調査した。

API とインターフェースは、ユニバーサル NIZK 設定と非ユニバーサル NIZK 設定に分ける。ユニバーサル設定では、NIZK の CRS 生成は関係から独立である（すなわち 1 つの CRS が任意の NP 命題の証明を可能にする）。非ユニバーサル設定では、CRS 生成は関係（制約系として表現される）に依存し、与えられた CRS はその特定関係に関する任意のインスタンスに対応する命題の証明を可能にする。

表 3.1: ユニバーサル性と前処理の種類ごとの API・インターフェース（要約）

- **非ユニバーサル（Generate が制約系を入力として必要）かつ前処理型**: QAP ベース [PHGR13], [GGPR13], [BCGTV13]。
- **非ユニバーサルかつ非前処理型**: 未整理または不明。
- **ユニバーサル（Generate がサイズ上界だけを必要）かつ前処理型**: vnTinyRAM、vRAM、Bulletproofs（明示的 CRH）。
- **ユニバーサルかつ非前処理型**: Bulletproofs（PRG ベースの CRH 生成）。

- **ユニバーサルかつスケーラブル（Generate がセキュリティ・パラメータ以外を必要としない）かつ前処理型**: 不可能。
- **ユニバーサルかつスケーラブルかつ非前処理型**: PCD（再帰的合成）に基づく「完全スケーラブル」SNARK。

証明システムは、汎用インターフェースを用いて以下の能力を表現する必要があり得る。

1. 入力言語またはサイズ上界が与えられたとき、証明パラメータと検証パラメータの形で CRS オブジェクトを作成する。
2. CRS オブジェクトを具体的符号化へシリアライズする。
3. 算術制約のための体のサイズと標数など、証明システムに関するメタデータ。
4. 証明者だけが知る秘密入力を含む Witness オブジェクト、および証明者と検証者に知られる公開入力を含む Instance オブジェクト。
5. 証明パラメータ、Instance、Witness が与えられたとき Proof オブジェクトを作成する。
6. 検証パラメータと Instance が与えられたとき Proof オブジェクトを検証する。

今後の作業: 再帰的合成、証明および検証操作のバッチ化などの概念を符号化する追加作業とともに、我々の暫定モデルを活用する具体的 API 設計が望まれる。

3.4.2 R1CS ファイル形式

制約系を構築するフロントエンドは多数あり、制約系（および変数割当て）を消費して証明を作成または検証するバックエンドも多数ある。したがって、フロントエンドとバックエンドの双方が制約系と変数割当てを通信するために使用できるファイル形式 [ZKP-FF] は有用である。目標には、単純性、実装の容易さ、コンパクト性、ハードコードされた制限の回避が含まれる。

ZKP で一般的な制約系は R1CS (Rank-1 Constraint System) である。これは、関係を双線形制約系、すなわち rank 1 quadratic constraint system として指定するための NP 完全言語である [BCGTV13, extended version Appendix E]。これは QAP (Quadratic Arithmetic Program) [PHGR13] をより直観的に再定式化したものである。R1CS は、多くの ZK 証明構成、および運用配備されている多くの ZK 証明応用のネイティブ制約系言語である。

FF3.4 今後の図案提案: R1CS 制約系を例示する（行列、QAP 多項式などを可視化する）。

本節の残りでは、R1CS の空間効率的な符号化でよく使われる可変長整数を多用する形式を提案する。`VarInt` は可変長符号なし整数を表し、`SignedVarInt` は可変長符号付き整数を表す。`VarInt` は通常、長さやバージョン番号に用いられ、`SignedVarInt` は体要素定数に用いられる。`VarInt` の実際の記述はまだ指定されていない。

扱うプリミティブな変数添字は以下の形である。

- `ConstantVar <- SignedVarInt(0)`
- `InstanceVar(i) <- SignedVarInt(-(i + 1))`
- `WitnessVar(i) <- SignedVarInt(i + 1)`
- `VariableIndex <- ConstantVar / InstanceVar(i) / WitnessVar(i)`

`ConstantVar` は体内の添字付き定数を表し、通常は 1 に割り当てられる。`InstanceVar` はインスタンス、すなわち公開入力の添字付き変数を表し、負の添字でシリアライズされる。`WitnessVar` は証人、すなわち秘密／補助入力の添字付き変数を表し、正の添字でシリアライズされる。`VariableIndex` はこれらの可能な変数添字のいずれかを表す。

扱うプリミティブな式は以下の形である。

- `Coefficient <- SignedVarInt`
- `Sequence(Entry) <- | length: VarInt | length * Entry |`
- `LinearCombination <- Sequence(| VariableIndex | Coefficient |)`

係数は非ゼロでなければならない。エントリは型、次に添字でソートされるべきである。順序は `ConstantVar`、ソート済み `InstanceVar`、ソート済み `WitnessVar` である。

制約は次の形で表される。

- `Constraint <- | A: LinearCombination | B: LinearCombination | C: LinearCombination |`

係数（線形結合内の定数）は `SignedVarInt` で表す。正準形にはまだ制約がない。これらはゼロであってはならない。`LinearCombination` は、`VariableIndex` と `Coefficient` のペアの列として表現する。線形結合は

VariableIndex の型、次に添字でソートされるべきである。すなわち ConstantVar が最初、InstanceVar が 2 番目（昇順）、WitnessVar が最後（昇順）に現れるべきである。

制約は 3 つの LinearCombination オブジェクト A, B, C として表現し、符号化された制約は A * B = C を表す。

ファイル形式は、バックエンド実装またはパースにとって重要な制約系の詳細を含むヘッダを持つ。

- Header(version, vals) <- | version: VarInt | vals: Sequence(SignedVarInt) |

ヘッダの vals 要素は以下のような情報を含む。

- P <- 体の標数。
- D <- 拡大次数。
- N_X <- インスタンス変数数。
- N_W <- 証人変数数。

拡大体要素の表現は現在指定されていないため、D は 1 であるべきである。

ファイル形式は、マジックバイト列 R1CSstmt、ヘッダ、制約列を以下のように含む。

R1CSFile <- | "R1CSstmt" | Header(0, [P, D, N_X, N_W, ...]) | Sequence(Constraint) |

バージョン 0 におけるヘッダのさらなる値は本仕様では未定義であり、無視されるべきである。R1CS 回路にはファイル拡張子 .r1cs が用いられる。

さらなる作業: 証明生成においてバックエンドが使用する割当てを表現する形式が望まれる。この目的のため、マジック R1CSasig とファイル拡張子 .assignments を予約する。また、デバッグ用のシンボル表を表現する形式も望まれる。この目的のため、マジック R1CSsymb とファイル拡張子 .r1cssym を予約する。

将来的には、いくつかの証明システムがより自然に消費できる他種類の制約系と言語も指定したい。

3.5 ベンチマーク

ゼロ知識証明システムの多様性と応用の複雑性が増すにつれ、利用者が自分の応用に最適な証明システムを理解することはますます難しくなっている。その理由の 1 つは、トレードオフ空間が高次元であることにある。もう 1 つの理由は、良質で統一されたベンチマーク指針が欠如していることである。我々は、先行研究との公平で偏りのない比較を可能にしつつ、科学者がトレードオフ空間全体を探索し、新しいシナリオにおいて微妙な差異を含むベンチマークを設定し、より多くの応用を可能にするような自由度を与える、ベンチマーク手続きを定義することを目指す。

ベンチマーク標準化は、厳格な要求事項ではなく、ベストプラクティスを文書化することを意味する。これは、新しい汎用証明システム、および既存スキームの実装に特に推奨される。さらに長期目標は、標準化されたハードウェア上で独立したベンチマークを可能にすることである。

3.5.1 どの指標と構成要素を測定するか

任意のベンチマークについて、主要指標として、すべての構成要素の実行時間（単一スレッド）と通信複雑性（非対話型証明システムの場合は証明サイズ）を測定し報告することを推奨する。測定対象構成要素には、少なくとも証明者と検証者を含めるべきである。セットアップが重要である場合、それも測定すべきであり、パラメータ読込みやラウンド数（対話型証明システムの場合）などの追加システム構成要素も推奨される。

さらに以下の指標が提案される。

- 並列化可能性。
- バッチ化。
- メモリ消費（正確な測定値または上界）。
- 演算数（体演算、多重指数演算、FFT など）およびそのサイズ。
- ディスク使用量／ストレージ要件。
- クロスオーバー・ポイント: 検証が計算を実行するより速くなる点。
- 与えられたシステムで扱える最大インスタンス。
- 証人生成（これは高水準コンパイラと応用に依存する）。
- 任意の指標間のトレードオフ。

3.5.2 ベンチマークの実行方法

ベンチマークは解析的性質のものでも計算的性質のものでもよい。システムによって、どちらかがより適切な場合があり、あるいは相互に補完できる。解析的ベンチマークは、漸近解析および特定指標（たとえば証明サイズ）に関する具体式からなる。理想的には、解析的ベンチマークはセキュリティ水準によってパラメータ化されるべきである。そうでない場合には、ベンチマークが行われるセキュリティ水準と使用される仮定を報告すべきである。

計算的ベンチマークは、一貫した市販機械上で実行されるべきである。クラウド提供者の使用は推奨される。これにより安価な再現性が可能になるからである。機械仕様は、それに課される追加制限（スロットリング、スレッド数、提供メモリなど）とともに報告されるべきである。ベンチマーク機械は一般に以下のカテゴリのいずれかに属するべきであり、機械説明はカテゴリを示すべきである。ソフトウェア実装が特定のアーキテクチャ仮定（特殊なハードウェア命令の使用など）を置く場合、それは明確に示されるべきである。

- バッテリ駆動のモバイルデバイス。
- ラップトップなどのパーソナルコンピュータ。
- 多数のコアと大容量メモリを持つサーバ型機械。
- 複数機械を用いるサーバクラスタ。
- カスタムハードウェア（ソフトウェア実装との比較に使うべきではない）。

ほとんどの実行は単一スレッド機械で実行し、並列化可能性は任意測定指標とすることを推奨する。ベンチマークは、1.8.2 節および 1.8.3 節の推奨に従い、できれば複数のセキュリティ水準で取得されるべきである。

より良い比較を可能にするため、他の証明システム／実装の指標も同じ機械上で実行し報告することを推奨する。ベンチマークとして報告される任意のインスタンスを簡単に実行できる方法を提供する責任はライブラリ開発者にある。これは結果の再現性にも役立つ。実装へのリンクは zkp.science に集められ、ライブラリ開発者には自分のライブラリが適切に参照されるようにすることが奨励される。さらに、実装が報告される場合、科学出版会場がソースコード提出を要求することを奨励する。理想的には、これらの会場は再現性を検査し、結果が再現できたかどうかを示すべきである。

3.5.3 どのベンチマークを実行するか

現在のゼロ知識証明の応用、および証明システム間の差異に基づき、ベンチマーク集合を提案する。この一覧は決して完全ではなく、新しい応用が現れ、新しい性質を持つシステムが開発されるにつれて修正・更新されるべきである。

ゼロ知識証明システムは既存応用にブラックボックス的に使用できるが、証明システムを念頭に置いて応用を設計すると大きな効率向上が得られることが多い。両方のシナリオを扱うため、一般的に使用されるプリミティブ（SHA-256 など）を含むベンチマーク集合と、機能だけが指定されプリミティブは指定されないもの（128 ビット古典セキュリティの衝突困難ハッシュ関数など）を提案する。

FF3.5 今後の図案提案: ベンチマーク実行対象となるユースケースの異なる層・水準を示す図またはマインドマップを追加する（例: zkproof3 の benchmarks proposal paper を参照）。

一般的に使用されるプリミティブ。以下はマイクロベンチマークとして機能し、独立にも関心があるプリミティブ一覧である。ライブラリ開発者は、与えられたプリミティブを自分のライブラリがどのように実行するかを自由に選べるが、一般的ファイル形式（R1CS など）で回路記述を利用可能にしておくことで、その過程を支援できる。

推奨:

1. SHA-256。
2. AES。
3. 異なるサイズにおける単純なベクトルまたは行列積。

さらなる提案:

1. Zcash Sapling の「spend」関係。
2. RC4（RAM メモリアクセス用）。
3. Scrypt。
4. メモリサイズ `s` で `n` ステップ実行する TinyRAM。
5. 数論変換（係数から点へ）: 小さい体、大きい体、パターン照合。

反復:

1. 上記の関係を n 個並列に配置して並列化したもの。

機能。以下は、アプリケーション開発者にとって特に関心のある暗号機能の例である。プリミティブの実現方法は、必要なセキュリティ性質を達成する限り二次的であり得る。NIZK バックエンドに合わせて調整された、これらのプリミティブの実現に対する制約系実装のベンチマークを提供することは有用である。

以下すべてにおいて、ZKP 命題の基礎となるプリミティブは、128 ビット以上の水準で与えられ、NIZK 証明システムのセキュリティと整合すべきである。

- 非対称暗号:
  - 署名検証。
  - 公開鍵暗号。
  - 128 ビットセキュリティを持つ任意の群上の Diffie-Hellman 鍵交換。
- 対称暗号およびハッシュ:
  - 1024 バイトメッセージ上の衝突困難ハッシュ関数。
  - サイズ `2^20` の集合における集合メンバーシップ（たとえば Merkle 認証木を用いる）。
  - MAC。
  - AEAD。
- 再帰的合成（Proof-Carrying Data）のため、整合するパラメータを持つスキーム自身の検証回路。
- 範囲証明 [自由に選ばれたコミットメント方式]:
  - 数が `[0, 2^64)` に含まれることの証明。
  - 数が正であることの証明。
- 置換の証明（2 つのコミットされたリストが同じ要素を含むことの証明）。

3.6 正確性と信頼

本節では、証明システムの実装を信頼できるものにするための要件を検討する。数学的スキームが主張された性質を満たすとしても、すなわち必要な意味で安全であることが証明され、その仮定が成り立ち、セキュリティ・パラメータが慎重に選ばれているとしても、その後の実装では多くの問題が起こり得る。コードのバグ、構造化参照文字列のサブバージョン、配備時の侵害、サイドチャネル、改ざん攻撃などである。本節は、そのようなリスクを強調し、実務者に考慮事項を提供することを目指す。

3.6.1 考慮事項

高水準プロトコルと命題の設計。 ZK 証明システムを呼び出す高水準プロトコルの仕様、とくにゼロ知識で証明すべき NP 命題は、意図されたドメイン固有のセキュリティ性質を達成できない場合がある。

これらのプロトコルを指定・検証する方法論はまだ初期段階にあり、実務上は手動レビューと証明スケッチに依拠することが多い。保証を得る可能な方法には、査読付き学術出版物への依拠（たとえば Zerocash [BCGGMTV14] や Cinderella [DFKP16]）、高水準ガジェットの再利用（4.4 節参照）、訓練を受けた暗号研究者によるプロトコル性質の慎重な手動仕様化と証明、形式検証の新興ツールが含まれる。

代数的簡約や、元の意図された命題で使われるアルゴリズムをより効率的な代替で置き換えるなど、命題に非自明な最適化が適用される場合、その最適化は適切な形式性水準の証明によって支えられるべきである。

暗号プリミティブの選択。一般に使われる伝統的暗号プリミティブ（ハッシュ関数、PRF など）は、ZK 証明システム用回路で実装されるときの効率を考えて設計されていない。近年、この設定で効率上の利点を持ち得る代替の「回路フレンドリー」プリミティブが提案されている（LowMC や MiMC など）。このようなプリミティブのセキュリティ評価には保守的アプローチを推奨し、それらを受け入れる基準は、より伝統的なプリミティブに対する基準と同じくらい厳密である必要があると助言する。

命題の実装。ZK 証明システムで証明される命題の具体的実装（ブール回路や R1CS など）は、高水準仕様を正しく捉えられない場合がある。このリスクは、命題が低い抽象化水準で実装される場合に高まる。低水準実装は誤りを起こしやすく、推論しにくいからである。

高水準仕様およびドメイン固有言語の使用（フロントエンド節参照）は、この誤りのリスクを下げられる。ただし、高水準仕様自体やコンパイル過程にも誤りが発生し得る。

さらに、リスクはしばしば、命題サイズ（回路サイズや R1CS 制約数など）を削減する最適化の文脈で生じる。

正しい命題意味論はセキュリティに不可欠であることに注意する。同じ高水準プロトコル、同じ制約系、互換バックエンドを用いる 2 つの実装でも、インスタンス還元（高水準命題からバックエンドに必要な低

水準入力への変換）が互換でない場合、正しく相互運用できないことがある。これは、完全性の失敗（証明が検証されない）または健全性の失敗（偽だが説得力のある証明を引き起こし、セキュリティ脆弱性となる）の双方を含む。

サイドチャネル。開発者は、サイドチャネル攻撃が有害となり得る異なる過程を認識し、サイドチャネルを最小化する措置を取るべきである。これには以下が含まれる。

- **SRS 生成:** 一部スキームでは、ランダムにサンプルされ破棄された要素が、もし露出すればシステムの健全性をサブバートするために利用され得る。
- **割当て生成／証明:** 秘密補助データが露出し、攻撃者が証明に使われた秘密データを理解できるようになる可能性がある。

監査。まず、回路設計者は低水準回路とともに、自分の回路および命題の高水準記述を提供し、両者のつながりを説明すべきである。

高水準記述は、実装されているプロトコルのセキュリティ性質の監査、およびそれらが設計者の意図または利用者が期待しそうな性質と一致するかの監査を容易にすべきである。

低水準記述がコードで直接表現されていない場合、コードと記述の対応関係は、手動またはツール支援によって監査過程で確認できるほど明確であるべきである。

回路実装の正確性とセキュリティの監査における主な焦点は、低水準記述が高水準記述と一致することの検証である。これには、ZK 証明システムのセキュリティ性質に対応するいくつかの側面がある。

- 低水準回路のインスタンスは、高水準命題のインスタンス以上の情報を明らかにしてはならない。これは、高水準インスタンスの正準符号化であることを保証することで最も容易に達成される。
- 高水準命題のインスタンスと証人に対応しない、低水準回路のインスタンスと証人を見つけられてはならない。

すべての抽象化水準で、操作される値の領域と表現を明確にするため型を用いることが有益である。通常、与えられた証明システムは、高水準命題に必要なすべての値の型を直接表現できるわけではない。代わりに、値はたとえば R1CS ベース証明システムでは体要素として符号化される。これらの要素に対して利用可能な演算は、表現対象の値に対する演算と異なる場合がある。たとえば、オーバーフローがある場合、体の加算は整数加算に対応しない。

意図されていない命題インスタンスを証明しようとする敵対者は、必ずしも意図された表現に対応するインスタンス変数と証人変数を用いるよう制約されない。したがって、制約系が意図されない表現を明示的に除外していることを保証するため、十分な注意が必要である。

証明システムのフロントエンドがこの問題に対処する方法には、広い設計トレードオフ空間がある。フロントエンドは、高水準命題を直接表現するのに適した豊かな型集合を提供するかもしれない。体要素だけを提供し、表現問題をフロントエンド利用者に任せるかもしれない。ユーザが新しい型を定義できる抽象化機構を提供するかもしれない。フロントエンドを用いて表現された命題の監査可能性は、この設計選択における主要な考慮事項であるべきである。

フロントエンドが命題要素の合成に「ガジェット」アプローチを取る場合、各ガジェットが入力変数および／または出力変数を必要な型に制約する責任を持つのかが明確でなければならない。

テスト。制約系をテストする方法には以下が含まれる。

- 失敗のテスト: 実装は受理すべきでない割当てを受理するか。
- 回路入力のファジング。
- 不足制約の発見。たとえばビットを表す変数に対するブール制約の欠落や、その他の型制約の欠落。
- 死んだ制約の発見と報告（最適化で除去するのではなく）。
- 意図しない非決定性の検出。たとえば部分的に固定された割当てが与えられたとき、残りを解き、解が 1 つだけであることを確認する。

証明システム実装は、テストおよびデバッグ目的で、与えられた割当てが制約を満たさなかった理由へのアクセスを提供することでテストを支援できる。また、通常利用では発生しないインスタンス変数および証人変数の値（たとえば正しい型の値を表さないため発生しない値）を注入できるようにすべきである。これらの機能は「ホワイトボックス・テスト」、すなわち回路実装が意図された理由でインスタンスと証人を拒否することをテストすることを容易にする。この支援がないと、失敗モードの十分なカバレッジを持つ正しいテストを書くことは難しい。

3.6.2 SRS 生成

重要な信頼問題は、秘密乱数を伴うパラメータ・セットアップ過程（構造化参照文字列）を必要とする証明システムで生じる。これらは、その過程が脆弱である、または安全に実行するのに高価であるシナリオに対処しなければならない場合がある。ここでは、エアギャップ、公開検証可能性、スケーラビリティ、中止の処理、参加者の評判、ランダムネス・ビーコンなど、これらのセットアップが直面しなければならない現実世界の社会的・技術的問題を検討する。

ZKP スキームは、健全性および／または ZK 性のために URS（一様参照文字列）または SRS（構造化参照文字列）を必要とする。これには適切な乱数源、および共通参照文字列の場合には安全に実行されたセットアップ・アルゴリズムが必要である。さらに、一部のプロトコルは応用間で再利用できる参照文字列を作成する。したがって、主要な ZKP スキーム族のセットアップ段階の実行、および共通資源の共有に関する考慮事項を求める。

本節は、実務者が CRS を安全に生成するための考慮事項を提供することを目的として、Implementation Track 参加者による公開討論を要約する。

SRS サブバージョンと失敗モード。単一機械で SRS を構築することが、あるシナリオに適合する場合がある。たとえば、検証者が単一実体であり、その検証者が SRS を生成するシナリオが含まれる。このシナリオで考慮すべき側面は、subversion zero-knowledge である。これは、SRS が検証者によって悪意を持って選ばれた場合でもゼロ知識性を維持できる証明スキームの性質である。

subversion zero knowledge の戦略には以下が含まれる。

- SRS 生成にマルチパーティ計算を用いる。
- [Gro16; PHGR13] のいずれかを適応する。
- 更新可能 SRS: SRS を一度安全な方法で生成し、その後、再生成の必要なく多くの異なる回路へ特殊化できる。

その他のサブバージョン考慮事項は 1.6.7 節で議論されている。

MPC を用いる SRS 生成。 SRS を生成する単一実体への信頼を減らすため、マルチパーティ計算を用いて SRS を生成することが可能である。この方法は、理想的には（独立した計算フェーズごとに）少なくとも 1 人の参加者が正直であれば安全であるべきである。

FF3.6 今後の図案提案: SRS プロトコルを表す MPC セレモニーの図を追加する。

MPC のセキュリティを強化する考慮事項には以下がある。

- 可能な限り多くの参加者を持つ。
  - 参加者の多様性。結託の可能性を下げる。
  - 実装の多様性（曲線、MPC コード、コンパイラ、オペレーティングシステム、言語）。
  - ハードウェアの多様性（CPU アーキテクチャ、周辺機器、RAM）。
    - 一回限り使用するコンピュータ。
    - GCP / EC2（エンタープライズ・セキュリティの活用）。
  - ハードウェア侵害を懸念する場合、サイドチャネル（電力、音声／無線、監視）を避ける。
    - ハードウェア除去: WiFi/Bluetooth チップを除去する、ウェブカメラ／マイク／スピーカーを切断する、不要ならハードディスクを除去するか swap を無効化する。
    - エアギャップ。
  - 決定的コンパイル。
  - 追記専用ログ。
  - トランスクリプトの公開検証可能性。
  - スケーラビリティ。
  - 中止の処理。
  - 評判。
- ハードウェアからの情報抽出を困難にする。
  - ハードウェア読み取り専用トグルを持つフラッシュドライブ。

一部のプロトコル（Powers of Tau など）は、予測不能な公開乱数のサンプリングも必要とする。そのような乱数は、proof of work ブロックチェーンや株式市場など他のエントロピー源から取得できる。Verifiable Delay Functions は、これらの源をバイアスする能力をさらに減らせる [BBBF18]。

SRS 再利用性。SRS を必要とするスキームでは、SRS の一部の再利用を可能にする SRS 生成過程を設計できる場合があり、これにより攻撃対象領域を減らせる。良い例は Groth16 構成 [Gro16] のための「Powers of Tau」方法 [BGM17] である。そこでは SRS の大部分を、特定の制約系へ特殊化する前に再利用できる。

指定検証者設定。検証者が事前に知られた単一実体である場合がある。この設定で優れたスキームが存在する。さらに、公開検証可能なスキームもこの設定へ特殊化できる。

3.6.3 緊急時対応計画

緊急時対応計画という概念は、将来さらに検討されるべきである。たとえば次にどのように対処するか。

- 証明システムが侵害された場合。
- 特定の回路にバグがある場合。
- ZKP プロトコルが破られた場合（無効な証人を持つ証明を特定するなど）。

発展可能なアイデアには以下がある。

- プロトコルにおけるスキーム・アジリティおよびプロトコル・アジリティ。システム設計時に、使用するプリミティブについて柔軟性を持たせる。
- コンバイナ（複数の証明システムを並列使用）。単一証明システムへの依存を減らすため、複数を使用する。
- ZKP プロトコルが破られたときにそれを特定する方法を議論する（無効な証人を持つ証明の特定など）。

3.7 拡張制約系相互運用性

以下は、実務者が望ましいと特定した、より強い形の相互運用性である。これらはさらに扱われるべきである。

3.7.1 命題および証人形式

R1CS ファイル形式の節および関連リソースでは、R1CS 制約系のためのファイル形式を定義している。インスタンスおよび証人を含め、この仕様を完成させる作業が残っている。これにより、利用者はフレームワーク（フロントエンドとバックエンド）を選択できるようになり、保存および通信のためのストリーミングが可能になり、これらの形式を受け入れる任意のバックエンドで実行できるベンチマーク・テストケースの作成が容易になる。

重要な点として、R1CS 以外の制約系言語についても類似の形式が望まれる。

3.7.2 命題意味論、変数表現およびマッピング

上記を超えて、異なる実装が制約系の命題（インスタンス）表現の意味論を調整する必要がある。たとえば、高水準プロトコルが命題の一部として RSA 署名を持つ場合、定数を法とする大整数が、より小さい体上の変数列としてどのように表現されるのか、またそれらの変数が実際の R1CS インスタンスのどの添字に配置されるのかに曖昧さが残る。

高水準抽象化の観点から命題意味論を正確に指定することは、インスタンス還元（高水準命題から ZKP 証明者および検証者が必要とする実際の入力への変換）の複数実装によって呼び出される制約系の相互運用性に必要である。さらに進んで、実際のインスタンス還元実装を再利用し、高水準かつ場合によってはドメイン固有の値表現（大整数など）を取り、低水準変数へ変換することを試みてもよい。これは、言語およびプラットフォームの非互換性、適切なモジュール化とパッケージングに関する問題を提起する。

正しい命題意味論はセキュリティに不可欠であることに注意する。同じ高水準プロトコル、同じ制約系、互換バックエンドを用いる 2 つの実装でも、インスタンス還元が互換でない場合、正しく相互運用できない可能性がある。これは完全性の失敗（証明が検証されない）および健全性の失敗（偽だが説得力のある証明を引き起こし、セキュリティ脆弱性となる）の双方を含む。さらに、意味論は検証の前提条件であり、デバッグにも役立つ。

一部のバックエンドは、制約系の一様性または規則性（反復パターンや代数構造など）を利用できるため、必要な情報を伝える形式や意味論から利益を得られる。

今日の制約系の典型的複雑性水準では、上記すべてを手動で処理し、非形式的仕様と先行実装の検査に基づき新規再実装することが受け入れられる場合が多い。しかし、応用複雑性が増すにつれて、これは維持困難で誤りを起こしやすくなると予想される。

3.7.3 証人還元

同様の考慮事項は、証人還元、すなわち高水準証人表現（与えられた命題に対するもの）を証人変数への割当てへ変換することにも生じる。たとえば、高水準プロトコルが特定深さと特定ハッシュ関数の Merkle 木を用い、高水準インスタンスが Merkle 認証パスを含む場合がある。証人還元は、これらを証人変数へ変換する必要がある。その証人変数には、Merkle 認証パスの全データ（特定の規約で体要素に符号化され、特定の順序で割り当てられる）に加え、ハッシュ関数を評価し、整合性とブール性を保証する制約で発生する多数の追加証人変数が含まれる。

証人還元は、制約系の特定実装に大きく依存する。相互運用性への可能なアプローチは、上記と同様、形式仕様、コード再利用、手動のアドホック互換性である。

3.7.4 ガジェット相互運用性

単一の制約系およびその割当てより細かい粒度で、サブ回路とガジェットの共有が必要である。たとえば libsnark は、高度に最適化された R1CS ガジェットの豊富なライブラリを提供しており、複数のフロントエンド・コンパイラ開発者が、自らの制約系構築フレームワークの文脈で再利用したいと望んでいる。

制約の塊をフレームワーク間で移植することは比較的容易であるが、ガジェットの外部可視変数の意味論を調整する点には課題がある。これは完全な制約系について上で述べた問題と類似するが、より難しい。ガジェットの外部可視変数の意味論を調整または再利用し、さらに、証人を内部変数への割当てへ変換するため、インポートされたガジェットの証人還元関数を調整または再利用する必要がある。

インスタンス意味論と同様、明確に定義されたガジェット意味論は、健全性、完全性、検証に不可欠であり、デバッグに役立つ。

3.7.5 手続き的相互運用性

インスタンスおよび証人還元に関する前述の必要性（制約系全体レベルおよびガジェット・レベルの双方）に対する魅力的なアプローチは、ある実装が別の実装のインスタンス／証人還元を、フレームワークやプログラミング言語を越えて呼び出せるようにすることである。これは単なるデータではなく、手続きコードの呼び出しの通信を必要とする。これに対する提案アプローチには、実行可能コード（`.so` ファイルや `.dll` など）へのリンク、関連するポータブル環境を持つ洗練された可搬高水準言語の使用、WebAssembly などの低水準可搬実行可能形式の使用が含まれる。これらすべてには、適切な呼び出し規約（体要素はどのように表現されるかなど）、利用指針、例が必要である。

相互運用性を超えて、有限体や楕円曲線算術などの低水準構成要素は多くの、またはすべての実装で必要であり、適切なライブラリを再利用できる。標準的なネイティブ・ライブラリによるコード再利用の慣行を用いて、これはかなりの程度すでに行われている。このような再利用ライブラリは、一貫した呼び出し規約のための便利な共通基盤も提供し得る。

3.7.6 証明相互運用性

別の望ましい目標は、異なる実装から来る証明者と検証者の間の相互運用性である。すなわち、一貫した判定を行う検証者を独立に書けること、および古い検証者を納得させる証明を生成し続ける証明者を再実装できることである。これは、ブロックチェーンの文脈（第 4 章参照）のように証明が公開投稿され、証明者と検証者の双方について複数の独立実装が望まれる応用で特に重要である。

このような相互運用性を達成するには、証明者と検証者は以下すべてについて合意しなければならない。

- ZK 証明システム（有限体や楕円曲線の選択など、すべての自由度を固定する）。
- インスタンス／命題および証人の形式（3.7.1 節参照）。
- 証明者パラメータおよび検証者パラメータの形式。
- 証明の形式。
- 制約系（R1CS など）および対応するインスタンス・証人還元の正確な仕様（3.7.3 節参照）。

代替として、正確に指定された決定的フロントエンド・コンパイルとともに、正確な高水準仕様を用いることもできる。

3.7.7 共通参照文字列

共通参照文字列 (CRS)、すなわち証明者パラメータと検証者パラメータに関する標準化も必要である。第一に、通信と保存のためのストリーミング形式について相互運用性が必要であり、これによりアプリケー

ション開発者は、異なるセキュリティおよび性能性質を持つ異なる実装を、必要に応じて容易に切り替えられる。さらに、構造化参照文字列 (SRS) については、ZK 証明システムとバックエンドによるその具体的実現に依存する非自明な意味論があり、一部スキーム（Powers of Tau プロトコルなど）では異なる回路間で SRS を部分的に再利用できる可能性がある。

3.8 将来目標

3.8.1 相互運用性

追加の ZK 証明システム・バックエンド、ならびに追加の通信・再利用シナリオを支えるため、相互運用性の多くの追加側面が分析され、標準でサポートされる必要がある。

追加の相互運用形態。上記の拡張制約系相互運用性の節で議論したように、R1CS 領域内であっても、単なる制約系と割当て表現を超えた多くの追加ニーズがある。これらはセキュリティ、機能、開発と再利用の容易さに影響する。

追加の関係スタイル。 R1CS 型制約系は、最初のワークショップにおける Implementation Track の議論で最も焦点が当てられ、その結果、それに適したファイル形式と API 仕様が導かれた。その他の制約系スタイルを議論することは重要な目標である。これらは、他の ZK 証明システムおよび対応するバックエンドによって用いられている。これには、算術回路およびブール回路、それらの変種で規則的／反復的要素を利用できるもの、ならびに算術制約充足問題が含まれる。

再帰的合成。証明の再帰的合成技術、および Proof-Carrying Data (PCD) [CT10; BCTV14a] としてのその抽象化は、多段階計算または大量データを扱う応用において ZK 証明システムの性能と機能を改善できる。これは追加オブジェクトおよび対応する相互運用性考慮事項を導入する。たとえば、PCD compliance predicates は、その意味論を決定する追加規約を持つ制約系であり、相互運用性のためにこれらの規約は正確な仕様を必要とする。

ベンチマーク。論文間のミリ秒単位比較や競争を可能にするため、具体的参照ベンチマークおよび参照プラットフォームの作成を目指す。

さまざまな明確に定義されたシナリオで異なる証明スキームを評価するため、十分に指定された評価基準を持つオープンなコンペティションの作成を目指す。

3.8.2 フロントエンドと DSL

ドメイン固有言語の領域、とくに相互運用性、正確性、効率（ソース間最適化を可能にすることさえ含む）の側面について議論を拡大したい。拡張制約系相互運用性の節におけるガジェット相互運用性の目標も、フロントエンドに関連する。

3.8.3 実装の検証

形式検証、監査、一貫性テストなど、ベストプラクティスに向けた指針を支援するため、将来のワークショップで以下の主題を議論したい。

第 4 章 応用

4.1 導入

本章は、ベストプラクティスのセキュリティ要件を満たすようプロトコルを設計することを含め、ZKP ベース応用を構築する既存技術を概観することを目指す。高水準応用と低水準応用を区別する。前者は特定ユースケース向けに設計されたプロトコルであり、後者は必要な基礎操作またはサブプロトコルである。各ユースケースは回路を認め、プロトコルのセキュリティと機能を保証するために必要なサブ回路を議論する。本章では、回路を述語、サブ回路をガジェットと呼ぶ。

- **述語:** 命題と証人が満たさなければならない関係または条件。回路として表現できる。
- **ガジェット:** 述語を構成するために必要な基礎道具。場合によっては、ガジェットをセキュリティ要件として解釈できる（たとえば、コミットメント検証ガジェットを使うことは、基礎データのプライバシーを保証することと同等である）。

1.5 節で説明した証明者と検証者間の証明システム構文を想起する。後に見るように、プロトコルは複数のユースケースを認めるよう抽象化・一般化できる。同様に、一般的に使われるプログラミング言語から必要なガジェットを生成するコンパイラも存在する。制約系の作成は ZKP 応用の基本的部分であり、そのため多様なフロントエンド・ソフトウェア選択肢が利用可能である。

機能性と性能。 ZKP の設計は、機能性と性能のトレードオフの対象である。利用者は、任意の述語に対して証明を構成できるという意味で強力な ZKP を望み、これはユニバーサル ZKP の必要性につながる。他方で、利用者は効率的な構成を望む。表 3.4.1 に従えば、ZKP は (i) ユニバーサルまたは非ユニバーサル、(ii) スケーラブルまたは非スケーラブル、(iii) 前処理型または非前処理型として分類できる。

項目 (i) は基礎 ZKP の機能性に関連し、項目 (ii) と (iii) は性能に関連する。zk-SNARK の利用により、非常に効率的な検証者を持つユニバーサル ZKP が可能になる。しかし、多くの提案は高価な前処理に依存しており、そのため一部ユースケースではそのようなシステムのスケールが難しい。Ref. [CT10] で最初に提案された Proof-Carrying Data (PCD) という技術は、既存 ZKP に対してモジュール的に再帰的合成を得ることを可能にする。これは、zk-SNARK を構成要素として用い、スケーラブルで非前処理型の解を構成できることを意味する。その結果、zk-SNARK における効率的検証者だけでなく、計算資源消費が効率的、とくにメモリ要件に関して効率的な証明者も得られる。これは Refs. [BCTV14a] および [BCCT13] で説明されている。

構成。 4.2 節は応用にとって関心のある異なる種類の検証可能性性質に言及する。4.3 節は先行研究を列挙する。4.4 節は多様な応用に有用な可能性のあるガジェットを説明する。その後の 3 節では 3 つの ZKP ユースケースを提示する。4.5 節はアイデンティティ管理に関連するユースケースを説明し、4.6 節は資産移転に関連する応用文脈を検討し、4.7 節は規制遵守に関連する 1 つのユースケースを例示する。

4.2 検証可能性の種類

検証可能性の種類。 ZK ベース応用を設計するとき、どの ZKP 機能が必要かを定義する次の 3 モデルのうち、どれが必要かを念頭に置く必要がある。

1. **公開型**。要件として公開検証可能であるもの。証明が移転可能であることがシステム要件であるスキーム／ユースケース。
2. **指定型**。セキュリティ機能として指定検証者を用いるもの。証明の意図された受信者だけが検証できるため、証明は非移転可能となる。この性質は対話型 ZKP と非対話型 ZKP の双方に適用できる。
3. **任意型**。移転可能である必要はないが、非移転可能性の要件もないもの。この性質は、対話型モデルと非対話型モデルの双方で適用可能である。

2.6.3 節では、移転可能性と否認可能性を議論している。これは、対話型および非対話型の双方の設定における公開検証可能性と指定検証可能性の側面に強く関連する。

ユースケース例として、ユーザープライバシーと規制管理の側面が関連する、ブロックチェーン通貨に関する応用を考える。

取引の妥当性が公開であるべき場合、公開検証可能な ZKP は適切であり得る。たとえば、誰もが何らかの資産の所有者が変わったことを知る必要がある一方で、その所有権を制御するブロックチェーン・アドレスの秘密鍵など、裏付けデータは秘密に保たれる必要がある場合である。

しかし、証明される命題そのものが検証者の範囲を超えて秘密であるべき場合もあり、その場合は非移転可能な証明を用いるべきである。これは、購入に十分な資金があることの証明、またはあるブロックチェーン・アドレスの秘密鍵を知っていることの証明などに適用され得る。Alice は、Bob が受け取った証明を使って Charley に Alice の主張を納得させることを防ぎたい。その目的のため、Alice は Bob と否認可

能な対話型証明を実行できる。あるいは、Alice は Bob を指定検証者として構築した（非対話型）証明トランスクリプトを Bob に送ることができる。ユースケースに応じて、公開検証可能性と指定検証可能性の双方が応用目標として意味を持ち得る。両者を区別することが重要である。

「検証者の指定」は、真正性とプライバシーの間の可能な衝突を解決できる [JSI96]。たとえば投票所は、Bob だけに、自分の投票が数えられたことを納得させたい。投票所は Bob を証明の妥当性によって納得させる対象として指定し、悪意ある強制者が Bob にどのように投票したかを証明するよう強制することを防ぐ。指定検証者証明は非移転可能であるため、Bob は望んだとしても証明を移転できない。

Alice が、命題 θ が真であることを Bob だけに納得させたいとする。その目的のため、Alice は「θ が真である、または私は Bob の秘密鍵を知っている」という選言を証明できる。Bob は自分の秘密鍵を知っているため、そのような証明を自分で生成できた。したがって、Bob から送られたそのような証明トランスクリプトを見ても、第三者 Charlie は θ が真であるとは納得しない。これは、Bob が自分の秘密鍵を Charlie に共有した場合、またはその鍵が公開漏えいした場合でさえ成り立つ。

指定証明は対話型設定と非対話型設定の双方で可能である。対話型設定では、たとえば undeniable signature の署名者であることを証明する場合、証明者は検証がいつ行われるかを制御できる。しかし一般には、指定検証者アプローチなしでは、証明者は誰が証明を検証できるかを制御できない可能性がある。すなわち、検証者が別の支配当事者へのリレーとして振る舞っている場合である。指定証明の利用は、この問題を解決する可能性を持つ。

4.3 先行研究

本節は、ZKP の世界に存在するいくつかの研究および応用の概観を含む。

貢献募集: さらに多くの参考文献を追加する必要がある。

匿名クレデンシャルのための ZKP プロトコルは、学術領域で広く研究されてきた [CKS10; BCDEK+14; CDD17; BCDLRSY17; NVV18]。Miracl、Val:ID、Sovrin [Sov18]、LibZmix [Mik19] などの製品は、プライバシー保護型アイデンティティ・フレームワークを実現する実用解を提供している。Zerocash は学術研究として始まり、その後匿名取引を保証する製品へ発展した [BCGGMTV14]。Baby ZoE は Ethereum 上の Zerocash を可能にする [zca18]。HAWK も zk-SNARKS を用いて取引プライバシーを持つスマートコントラクトを可能にする [KMSWP16]。

4.4 述語内のガジェット

これらのプロトコルのセキュリティを形式化することは非常に難しい。とくに、事前に定められた要求集合がないため、アドホックな過程となる。

ユースケースでは、プライバシー要件とセキュリティ保証を必ず区別しなければならない。その違いを説明するため、プライバシー保護型資産移転のユースケースを議論する。

安全な資産移転は、金融機関が送信者、受信者、資産、金額の身元・内容を知っているなら、複数の金融機関で可能である。プライバシー保護型資産移転では、送信者と受信者の身元は、移転を管理する実体からさえ隠され得る。成功した移転は、プライバシー要件を満たすだけでなく、セキュリティ保証も提供しなければならないことに注意することが重要である。

プライバシー要件には、送信者と受信者の匿名性、資産種別と資産額の秘匿が含まれ得る。セキュリティ保証には、送信者以外の誰も送信者の代わりに移転を開始できないこと、または資産の十分な保有がない送信者が当該資産種別の移転を実行できないことが含まれ得る。

ここでは、考慮すべき初期ガジェット集合を概説する。表 4.1 はガジェットの簡単な一覧である。この一覧は、継続的に、かつケースごとに拡張されるべきである。各ガジェットについて、秘密／証人が何であり、公開／命題が何であるかを指定する表現を書く。

非技術者向けの NP 命題の表現例: 公開されたチェス盤面構成 `A` と `B` について、私は秘密のチェスの手列 `S` を知っており、構成 `A` から開始して `S` を適用すると、すべての手が合法であり、最終構成が `B` となる。

一般形（Camenisch-Stadler 記法）: `Zk { (wit): P(wit, statement) }`

リング署名の例: `Zk { (sig): VerifySignature(P1, sig) or VerifySignature(P2, sig) }`

表 4.1: ガジェット一覧

#	ガジェット名	ZKP 追加前の直観的説明	例示表
G1	コミットメント	封筒	表 4.2
G2	署名	署名付き委任状	表 4.3
G3	暗号化	受信者印付き封筒	表 4.4
G4	分散復号	複数人が開封に必要な受信者印付き封筒	表 4.5
G5	ランダム関数	抽選機	表 4.6
G6	集合メンバーシップ	ホワイトリスト／ブラックリスト	表 4.7
G7	Mix-net	投票箱	表 4.8
G8	汎用回路、TM、RAM プログラム	一般計算	表 4.9

表 4.2: コミットメント・ガジェット (G1; 封筒)

- 強化ガジェット: 「この封筒の中に隠された値を知っている。しかもその値を変更することはできない。」
  ZKP 命題: コミット値の知識。証明者が知る証人は、値と乱数を含む opening `O=(v,r)`。公開インスタンスはコミットメント `C`。述語は `C = Comm(v,r)`。
- 強化ガジェット: 「これら 2 つの封筒の中に隠された値が等しいことを知っている。」
  ZKP 命題: コミット値の等価性。証人は `O1=(v,r1)` と `O2=(v,r2)`。公開インスタンスは `C1, C2`。述語は `C1=Comm(v,r1)` かつ `C2=Comm(v,r2)`。
- 強化ガジェット: 「これら 2 つの封筒の中に隠された値が、特定の関係にあることを知っている。」
  ZKP 命題: コミット値間の論理的、算術的等の関係。証人は `O1=(v1,r1)` と `O2=(v2,r2)`。公開インスタンスは `C1, C2, relation R`。述語は `C1=Comm(v1,r1)`, `C2=Comm(v2,r2)`, `R(v1,v2)=True`。
- 強化ガジェット: 「この封筒内の値は特定範囲内にある。」
  ZKP 命題: 範囲証明。証人は `O=(v,r)`。公開インスタンスはコミットメント `C` と区間 `I`。述語は `C=Comm(v,r)` かつ `v` が範囲 `I` 内にあること。

表 4.3: 署名ガジェット (G2; 署名付き委任状)

- 強化ガジェット: 共通に知られたメッセージ上の秘密の有効署名。
  ZKP 命題: 共通に知られたメッセージ `M` 上の秘密署名 σ の知識。証人は署名 σ。公開インスタンスは検証鍵 `VK` とメッセージ `M`。述語は `Verify(VK,M,σ)=True`。
- 強化ガジェット: コミットされたメッセージ上の秘密の有効署名。
  ZKP 命題: 秘密メッセージ `M` の共通に知られたコミットメント `C` 上の秘密署名 σ の知識。証人は opening `O` と署名 σ。公開インスタンスは検証鍵 `VK` とコミットメント `C`。述語は `C=Comm(M)` かつ `Verify(VK,M,σ)=True`。

表 4.4: 暗号化ガジェット (G3; 受信者印付き封筒)

- 強化ガジェット: 出力平文が公開暗号文に対応する。
  ZKP 命題: 秘密平文 `M` の知識。証人は秘密復号鍵 `SK`。公開インスタンスは暗号文 `C` と暗号化鍵 `PK`。述語は `Dec(SK,C)=M`。複数の `C` と `M` がある場合は成分ごとに成り立つ。

表 4.5: 分散復号ガジェット (G4; 複数人が開封に必要な受信者印付き封筒)

- 強化ガジェット: 出力平文が公開暗号文に対応する。
  ZKP 命題: 秘密平文 `M` の知識。証人は復号鍵 `SK` の秘密シェア `[SK_i]`。公開インスタンスは暗号文 `C` と暗号化鍵 `PK`。述語は `SK=Derive([SK_i])` かつ `Dec(SK,C)=M`。複数の `C` がある場合は成分ごとに成り立つ。

表 4.6: ランダム関数ガジェット (G5; 抽選機)

- 強化ガジェット: 検証可能ランダム関数 (VRF)。
  ZKP 命題: VRF が秘密シードと公開（または秘密）入力から計算されたこと。証人は秘密シード `W`。公開インスタンスは入力 `X` と出力 `Y`。述語は `Y = VRF(W,X)`。

表 4.7: 集合メンバーシップ・ガジェット (G6; ホワイトリスト／ブラックリスト)

- Accumulator: 集合包含。証人は秘密要素 `X`。公開インスタンスは公開集合 `S`。述語は `X ∈ S`。
- Universal accumulator: 集合非包含。証人は秘密要素 `X`。公開インスタンスは公開集合 `S`。述語は `X ∉ S`。
- Merkle Tree: 要素がベクトル内の特定位置を占める。証人は秘密要素 `X`。公開インスタンスは公開ベクトル `V`。述語はある `i` について `X = V[i]`。

表 4.8: Mix-net ガジェット (G7; 投票箱)

- Shuffle: 入力暗号文と出力暗号文に含まれる平文集合が同一である。証人は置換 π と復号鍵 SK。公開インスタンスは入力暗号文リスト C と出力暗号文リスト C'。述語はすべての j について Dec(SK,C'_j)=Dec(SK,π(C_j))。
- Shuffle and reveal: 入力暗号文に含まれる平文集合が出力平文集合と同一である。証人は置換 π と復号鍵 SK。公開インスタンスは入力暗号文リスト C と出力平文リスト P。述語はすべての j について P_j=Dec(SK,π(C_j))。

表 4.9: 汎用回路、TM、RAM プログラム・ガジェット (G8; 一般計算)

- 強化ガジェット: 「この計算を正しくする秘密入力が存在する。」
  ZKP 命題: 回路／チューリング機械／ RAM プログラム計算の正しさの ZK 証明。証人は秘密入力 w。公開インスタンスはプログラム C（回路、TM、RAM プログラムのいずれか）、公開入力 x、出力 y。述語は C(x,w)=y。
- 強化ガジェット: 「あるサブ計算が正しいことをすでに知っているという前提で、この計算は正しい。」
  ZKP 命題: 別出力の検証 + 後処理（合成）の ZK 証明。証人は秘密入力 w。公開インスタンスはサブルーチン C' を持つプログラム C、公開入力 x、出力 y、中間値 z=C'(x,w)、および z=C'(x,w) であることの zk 証明 π。述語は C(x,w)=y。

4.5 アイデンティティ・フレームワーク

4.5.1 概要

本節では、ゼロ知識証明を用いるアイデンティティ管理ソリューションを説明する。考え方は、あるユーザーが属性の集合を持ち、それらが発行者または複数発行者によって証明され、その証明がそれら属性またはその一部の検証に対応するというものである。証明後、この情報、ここではクレデンシャルと呼ぶものを用いて、それら属性に関するクレームを生成できる。たとえば、Alice が 18 歳を超えており、欧州連合に属する国に住んでいることを示したい場合を考える。Alice の年齢と居住国の証明について 2 人の発行者が責任を持つなら、Alice はゼロ知識証明を用いて自分がそれら属性を持つことを示せる。たとえば、年齢が 18 歳を超えていることを示すためにゼロ知識範囲証明を用い、欧州連合に属する国に住んでいることを証明するためにゼロ知識集合メンバーシップを用いることができる。この証明は、Alice が何らかのサービスを利用することを許可するために、その証明を検証しなければならない検証者へ提示できる。したがって関与する当事者は 3 つである。(i) クレデンシャル保持者、(ii) クレデンシャル発行者、(iii) 検証者。

4.5.2 アイデンティティとゼロ知識の動機

デジタル・アイデンティティは、インターネットの誕生以来、学術研究者と産業界実務者の双方にとって関心のある問題である。具体的には、個人、会社、資産が、ID カード、署名文書、ライセンスなどの物理的識別を生成する必要なくオンラインで識別されることを可能にする問題である。アイデンティティのデジタル化には、プライバシー喪失と、それに伴うアイデンティティ窃盗、監視、ソーシャル・エンジニアリング、その他の有害な試みにさらされるという固有のリスクがある。実際、この問題は、パスワード暗号化、公開鍵証明書、TLS のようなインターネット・プロトコル、その他多数の暗号学的道具の助けにより、適度なプライバシーを達成する形で部分的に解決されてきた。しかし、これらの解決策は、オンラインの利用者／アイデンティティのプライバシー需要を満たすには不十分な場合がある。暗号学的ゼロ知識証明は、デジタルに相互作用する能力をさらに高め、プライバシーとプロセスの正しさに必要な正当性の保証の双方を得られるようにする。

以下は、アイデンティティ・スキームの一般化バージョンの概観である。データ構造と当事者に使用する用語を定義し、含める機能と求めるプライバシー保証について詳述する。

4.5.3 用語／定義

データ構造。このプロトコルでは、当事者間で転送または交換される情報を表すために、複数の異なるデータ構造を用いる。既存のアイデンティティ標準と先行 ZKP 研究に適応しつつ、できるだけ一般化した定義を試みた。

- **属性:** システム内の保持者に関する最も基本的な情報（例: 年齢、国籍、学位、職業証明書、未払い債務など）。これらは事実に基づく性質であり、そこから特定の認可を導出できる。
- **（機密かつ匿名）クレデンシャル:** システム内の保持者に関する 1 つ以上の属性を含むデータ構造（例: クレジットカード明細、婚姻状態、年齢、住所など）。私的データを含むため、クレデンシャルは共有できない。

- **（検証可能）クレーム:** クレデンシャル（または複数のクレデンシャル）内の属性に関するゼロ知識述語。クレームはあるアイデンティティについて行われるべきであり、zk 述語によって定義された制約系に含まれる何らかの論理命題を含むべきである。
- **クレデンシャルの証明:** クレデンシャルによって証明されるクレームを検証するために用いられるゼロ知識証明。クレデンシャルは機密に保たれるため、それから導出される証明が、問題のクレームを証明する方法として提示される。

プロトコルの当事者。

- **保持者:** 属性が証明される当事者。保持者は自身の属性を含むクレデンシャルを保持し、それらに関する何らかのクレームを証明するためにゼロ知識証明を生成する。保持者は、あるクレームに対するクレデンシャルの証明を提示するという。
- **発行者:** 保持者の属性を証明する当事者。発行者は保持者にクレデンシャルを発行するという。
- **検証者:** クレームに対するクレデンシャルのゼロ知識証明を検証することにより、保持者に関する何らかのクレームを検証する当事者。

注記。このプロトコルと非 ZK ベースのアイデンティティ・プロトコルの主な違いは、後者では保持者がクレーム／認可の証拠としてクレデンシャル自体を提示するのに対し、本プロトコルでは保持者がクレデンシャルから計算されたゼロ知識証明を提示する点である。

4.5.4 プロトコル記述

機能。アイデンティティ・プロトコルの一部として考慮した興味深い機能は多数ある。最初から含めることにした基本機能は 4 つである。

1. 発行者によるクレデンシャル発行を通じた、第三者による匿名かつ機密な属性証明。
2. 保持者によるクレデンシャルの証明提示を通じた、ゼロ知識証明を用いるクレームの機密証明。
3. 検証者によるゼロ知識証明検証を通じたクレームの検証。
4. 発行者によるリンク不能なクレデンシャル失効。

このプロトコル・バージョンには含まれないが、探索する価値のある興味深い機能もある。これには、クレデンシャル移転、権限委譲、追跡監査可能性が含まれる。

プライバシー要件。システム内の各当事者について高いプライバシー水準を目指すべきであるが、プロトコルの正確性を損なってはならない。各当事者の匿名性、相互作用とデータ交換の機密性、公開データ（コミットされた形）におけるリンク不能性を考える。これらは通常、コミットメント非可鍛性、ランダムデータからの識別不能性、偽造不能性、アキュムレータ健全性などの暗号学的要求、または原像知識の証明、署名検証の証明などのゼロ知識命題として具体化できる。

- **保持者匿名性:** 保持者の基礎となる物理的アイデンティティは、一般公衆から隠されなければならず、必要に応じて発行者および検証者からも隠される。これには、保持者だけが知る秘密に結びついた識別子と呼ばれる擬似ランダム文字列を用いる。
- **発行者匿名性:** 特定のクレデンシャルをどの発行者が発行したかは保持者だけが知るべきである。
- **匿名クレデンシャル:** 保持者がクレデンシャルを提示するとき、検証者はどの発行者が証明書を発行したかを知ることはできない。検証者は、そのクレデンシャルが何らかの承認済み発行者によって発行されたことだけを知り得る。
- **保持者追跡不能性:** 保持者識別子とクレデンシャルは、時間を通じて保持者を追跡するために用いられてはならない。
- **機密性:** 保持者と発行者以外はクレデンシャル属性を知ってはならない。
- **識別子リンク可能性:** 保持者が証明を提示しない限り、誰も 2 つの識別子をリンクできてはならない。
- **クレデンシャル・リンク可能性:** 公開利用可能なデータから、誰も 2 つのクレデンシャルをリンクできてはならない。主として、保持者のデジタル・アイデンティティを用いることで、2 人の発行者が結託して 2 つのクレデンシャルを同一保持者にリンクできてはならない。

詳細な見方。スキームの具体的インスタンス化について、これらの要件を達成するさまざまな方法とそのトレードオフ（ペアワイズ識別子 vs 1 つの固定公開鍵、異なる失効機構など）を検討し、それぞれのプライバシーおよび効率性質を詳述する。

機能 vs プライバシーおよび堅牢性要件。インスタンス化方法、証明詳細、プライバシー／堅牢性のいくつかの側面は、4 つの機能／問題に関連する次の 4 つの表で説明される。保持者識別（表 4.10）、発行者識別（表 4.11）、クレデンシャル発行（表 4.12）、クレデンシャル失効（表 4.13）。

表 4.10: 保持者識別 - クレデンシャル保持者をどのように識別するか

- **連合領域における単一識別子:** PRF ベースの公開鍵 idPK が実体の物理 ID から導出され、連邦的機関によって証明／オンボードされる。
  証明詳細: 実体が最初に取得すべきクレデンシャルは、システム上のアイデンティティを証明するオンボーディング・クレデンシャルである。保持者が生成する任意のクレデンシャル証明は、idPK にオンボーディング・クレデンシャルが発行されたことの検証を含まなければならない。
  プライバシー／堅牢性: 物理的アイデンティティは隠されるが公開鍵に結びつく。発行者は結託して同一保持者の異なるクレデンシャルをリンクできる。実体はシステム内で 1 つのアイデンティティしか持てない。

- **自己主権領域における単一識別子:** PRF ベースの公開鍵 idPK を実体自身が導出する。
  証明詳細: 任意のクレデンシャル証明は、保持者が idPK の原像を知っており、クレデンシャルが当該 idPK に発行されたことを示さなければならない。
  プライバシー／堅牢性: 物理的アイデンティティは隠され、必ずしも公開鍵と接続される必要はない。発行者は結託して同一保持者の異なるクレデンシャルをリンクできる。実体は複数のアイデンティティを持て、必要に応じて「否定的クレデンシャル」の発行時にそれらを忘れることができる。

- **複数識別子:** 識別子によるペアワイズ識別。新しい相互作用ごとに保持者が新しい識別子を生成する。
  証明詳細: 保持者が以前の発行者へ再接続する必要があるたびに、新旧識別子のつながりを ZK で証明しなければならない。任意のクレデンシャル証明は、保持者がそのクレデンシャルが発行された識別子の秘密を知っていることを示さなければならない。
  プライバシー／堅牢性: 物理的アイデンティティは隠され、必ずしも公開鍵と接続される必要はない。結託する発行者集合は、同一保持者のクレデンシャルをリンクできない。実体は複数のアイデンティティを持て、必要に応じて「否定的クレデンシャル」の発行時にそれらを忘れることができる。

表 4.11: 発行者識別

- **連合的許可:** 更新可能な承認済み発行者リストがあり、中央機関またはノード集合によって更新される。
  証明詳細: クレデンシャルを受理するには、リスト内の 1 つに対して署名を検証しなければならない。発行者匿名性を維持するため、リング署名を使用できる。クレデンシャルの各証明について、保持者は、そのクレデンシャル内の署名が承認済みリスト内の発行者によるものであることを証明しなければならない。
  プライバシー／堅牢性: 検証者／公衆はクレデンシャル発行者が誰であるかは知らないが、承認済みであることを知る。

- **自由許可:** 誰でも発行者になれる。発行者は識別子を用いる。
  1 型は、署名検証鍵が公開利用可能な発行者である。2 型は、署名検証鍵が保持者／検証者とのペアワイズでのみ識別できる発行者である。
  証明詳細: 1 型発行者が発行したクレデンシャルは、無関係な当事者への証明に使用できる。2 型発行者が発行したクレデンシャルは、問題の発行者を知っている当事者への証明にのみ使用できる。
  プライバシー／堅牢性: リング署名が使用される場合、1 型発行者識別子は発行者のアイデンティティがクレデンシャルにリンクできることを意味せず、「鍵 K_a が会社 A に属する」ことだけを意味する。そうでなければ、匿名かつクレデンシャルへリンク不能なのは 2 型発行者だけである。

表 4.12: クレデンシャル発行

- **ブラインド署名:** 発行者は正しい証明の証明を見た後、自己証明されたクレデンシャルのコミットメントに署名する。システムには第 2 種類の証明が必要になる。
  証明詳細: 正しい証明の証明は、発行者が許可する構造、データ型、範囲、クレデンシャル型を含まなければならない。場合によっては、証明は属性自体の検証を含まなければならない（例: 住所がフロリダ州にあるが都市は知らない）。クレデンシャルの署名がゼロ知識内または何らかの公開検証の一部として検証されない場合、クレデンシャルの証明は受理されてはならない。
  プライバシー／堅牢性: 発行者のクレデンシャル署名は限定的な正当性しか追加しない。保持者は実在しない特定の値／属性を追加でき、発行者はそれを知らない可能性がある。発行者は保持者と結託して、発行者が責められずにブラインド署名を生成できる。

- **平文署名:** 発行者は証明を生成し、コミットメントに署名し、クレデンシャルを平文で保持者へ送る。
  証明詳細: クレデンシャルの署名がゼロ知識内または何らかの公開検証の一部として検証されない場合、クレデンシャルの証明は受理されてはならない。
  プライバシー／堅牢性: 発行者は保持者のデータを見ることができ、他者と共有する可能性があるため信頼されなければならない。発行者の署名は信頼でき、責任を発行者に割り当てることができる。

表 4.13: クレデンシャル失効

- **正のアキュムレータ失効:** 発行者はアキュムレータから要素を削除することでクレデンシャルを失効させる [BCDLRSY17]。
  証明詳細: 保持者は、クレデンシャルが発行され、同時に失効していないことを証明するため、クレデンシャルの集合メンバーシップを証明しなければならない。発行者は、クレデンシャルを表す要素をアキュムレータから削除することで失効させることができる。
  プライバシー／堅牢性: アキュムレータが中央機関によって維持される場合、失効を元の発行にリンクできるのはその機関だけであり、一般当事者によるタイミング攻撃（join-revoke linkability）を避けられる。アキュムレータが公開状態を通じて維持される場合、追加された値を追跡してメンバーシップをテストできるため、失効と発行のリンク可能性が生じ得る。
- **負のアキュムレータ失効:** 発行者はアキュムレータに要素を追加することで失効させる。
  証明詳細: 保持者は、クレデンシャルが発行されたことを証明するため、クレデンシャルの集合メンバーシップを証明しなければならない。発行者は、失効対象クレデンシャルに関連する失効秘密を負のアキュムレータに追加することで失効できる。保持者は、当該クレデンシャルに関連する失効秘密の集合非メンバーシップを証明しなければならない。検証者はクレーム検証のため、アキュムレータの最新バージョンを用いなければならない。
  プライバシー／堅牢性: アキュムレータが公開状態を通じて維持される場合でも、失効は発行へリンクできない。なぜなら 2 つのイベントは互いに独立だからである。

ガジェット。異なる機能を具体化する各方法は、ガジェット節で説明したガジェットの一部を用いる。任意の証明の述語には主に 3 つの部分がある。

1. 第一は、アイデンティティ、この場合は保持者、の真実性を証明する部分であり、以下のガジェットを使用できる／すべきである。
   - アイデンティティが正しく証明されたことを検査するためのコミットメント。
   - 識別子の原像が保持者によって知られていることを証明する PRF。
   - 新しい識別子が以前に使われた識別子または承認済み識別子とつながりを持つことを証明する文字列等価性。
2. 次に、クレデンシャルの正当性、すなわちそれが正しく発行され失効していないことを扱う制約系の部分がある。
   - クレデンシャルが正しくコミットされたことを検査するコミットメント。
   - 保持者がクレデンシャル情報、すなわちコミットメントの原像を知っていることを証明する PRF。
   - クレデンシャルが現在の識別子に接続された識別子へ発行されたことを証明する文字列等価性。
   - コミットメントが何らかの集合（通常はアキュムレータ）に存在することを証明するアキュムレータ（集合メンバーシップ／非メンバーシップ）。これは、正しく発行され失効していないことを含意する。
3. 最後に、属性自体に課される規則／制約を検証するための論理がある。この部分は「クレデンシャル」と呼ばれる汎用ガジェットと見ることができ、クレデンシャルに埋め込まれた特定属性を検証できる。クレデンシャル型に応じて、次の低水準ガジェットを用いる。
   - クレデンシャル内のデータが正しい型であることを検査するデータ型。
   - クレデンシャル内のデータがある範囲内にあることを検査する範囲証明。
   - インスタンス計算において算術演算が正しく行われたことを検証する算術演算（体算術、大整数など）。
   - インスタンス内の値とクレデンシャル内のデータ、またはそこから導出された計算を比較するための論理演算子（大小比較、等価性など）。

セキュリティ上の注意:

1. 発行者が検証者と結託すると、失効情報のリアルタイム共有がある場合、失効機構を用いて保持者に関する情報を明らかにできる。
2. さらに、クレデンシャルへのコミットメントと失効情報が公開追跡可能であり、そのイベントが互いに依存している場合（たとえばコミットメントを削除することで失効する場合）、発行と失効の間にリンク可能性が生じ得る。
3. 自己証明または発行者と保持者の結託の場合、データ完全性の保証は大幅に低くなる。ZKP への入力が偽装され、その証明が健全でなくなる可能性がある。
4. ブロックチェーンの利用は、外部状態に対する信頼されたオラクルへの依存を生む。他方で、ブロックチェーン内容レベルで保証されるプライバシーは、ネットワークレベルのトラフィック分析とは直交する。

4.5.5 クレデンシャル集約のユースケース例

ここでは特定のユースケース、すなわち accredited investors（適格投資家）に焦点を当てて説明する。このシナリオでは、クレデンシャル保持者は、そのクレームを証明するのに必要な以上の情報を明らかにせず、自分が適格であることを示すことができる。

ユースケース記述。上記プロトコルを説明するため、特定ユースケースを提示し、証明の述語を明示的に書く。主として、あるアイデンティティ Alice が、会社株式を取得するため、SEC 規則の下で自分が適格投資家であることを、ある会社 Bob Inc. に証明したい。Alice は証明者であり、IRS、AML エンティティ、銀行はすべて発行者であり、Bob Inc. は検証者である。

このユースケースへの適応における異なる過程は以下である。

1. Alice に 3 つの機密クレデンシャルが発行される。これは、ある実体が適格投資家であるために適用する規則を表す。SEC が、証明鍵と検証鍵を生成するための回路として、適格性規則の制約系を生成すると仮定する。現実のシナリオには、適格性に関する連邦規則 [ECFR] がある。
   - IRS は税務クレデンシャル `C0` を発行し、「2017 年 1 月 1 日から 2018 年 1 月 1 日まで、識別子 `X0` を持つ Alice は識別子 `Y` を持つ IRS に対し `0` ドルの債務を負う」というクレームを証明する。このクレデンシャルは 2 つの属性、Alice の純所得 `\$income` と、Alice が税を支払っていれば `b=1` となるビット `b` を持つ。
   - AML エンティティは KYC クレデンシャル `C1` を発行し、クレーム `T1 :=` 「識別子 `X1` を持つ `Alice` はブラックリスト組織（集合）と関係がない」を証明する。
   - 銀行は純資産クレデンシャル `C2` を発行し、クレーム `T2 :=` 「`Alice` は `V_Alice` の純資産を持つ」を証明する。
2. その後 Alice は Bob Inc. に対して以下を証明する。
   - 「Alice の識別子 `XBob` は、機密クレデンシャル `C_i` に接続された識別子 `X_i`（`i=0,1,2`）と関連している。」
   - 「私は、あるコミットメント `C_i` の原像であるクレデンシャルを知っており、それらは正当な発行者によって発行された。」
   - 「アキュムレータ `U` に存在する、あるコミットメント `C_i` の原像であるクレデンシャルは、3 つの命題 `T_i` を満たす。」

インスタンス化詳細。上記の表における選択肢に応じて、以下が用いられる。

- 保持者識別: 識別子を一意な匿名識別子（公開鍵）として具体化する。
- 発行者識別: 発行者のアイデンティティはすべての参加者に知られている。参加者は、回路にハードコードされた公開署名検証鍵を用いて、発行者が発行するクレデンシャル上の署名を公開検証できる。
- クレデンシャル発行: クレデンシャルは、正のアキュムレータに署名済みコミットメントを公開し、クレデンシャルを平文で Alice に共有することで発行される。
- クレデンシャル失効: 動的かつ正のアキュムレータからクレデンシャルのコミットメントを削除することで行われる。Alice は、自分のクレデンシャルが失効していないことを示すため、コミットメントのメンバーシップを証明しなければならない。
- クレデンシャル検証: Bob Inc. はインスタンスとともに暗号学的証明を検証する。

会社株式の移転および発行はこのユースケースの範囲外であることに注意する。ただし、本書の「資産移転」節を用いてその機能を提供できる。

また、証明鍵と検証鍵が SEC によって検証されたという事実は、証明検証が Alice が適格投資家であることを含意するという保証を Bob Inc. に与える。

述語:

- 青 = プロトコル／命題において公開可視。
- 赤 = 秘密証人。証明時に当事者間で共有される可能性がある。

定義／記法:

- 公開状態: 発行および失効のためのアキュムレータ。すべてのクレデンシャルへのコミットメントを含む。
- `ConfCred = Commitment to Cred = { Revoke, certificateType, publicKey, Attribute(s) }`。

ここで IRS、AML、銀行はいずれもよく知られた公開鍵を持つ権限主体である。Alice の `publicKey` は長期公開鍵であり、長期 ID が承認されない限り新しいクレデンシャルを作成できない。このスキームの目標は、保持者が適格投資家というクレームに対する機密集約クレデンシャルの新鮮な証明を作成することである。

- IRS は ConfCredIRS = Commitment(openIRS, revokeIRS, "IRS", myID, $Income, b), sigIRS を発行する。
- AML は ConfCredAML = Commitment(openAML, revokeAML, "AML", myID, "OK"), sigAML を発行する。
- 保持者は、一時的なブラインド集約クレデンシャルとして機能する新鮮な公開鍵 freshCred を生成し、以下の ZKP を生成する。

ZkPoK{ (witness: myID, ConfCredIRS, ConfCredAML, sigIRS, sigAML, $Income, mySig, openIRS, openAML; statement: freshCred, minIncomeAccredited) : Predicate: ... }

述語:

- ConfCredIRS は IRS クレデンシャル (openIRS, "IRS", myID, $Income) へのコミットメントである。
- ConfCredAML は (openAML, "AML", myID, "OK") への AML クレデンシャルである。
- $Income >= minIncomeAccredited。
- b = 1、すなわち「myID は税を全額支払った」。
- mySig は myID に対する freshCred 上の署名である。
- ProveNonRevoke()。

依拠当事者へ提示するクレデンシャルは freshCred と zkp である。

ProveNonRevoke に関する記法:

- revokeIRS: IRS からの失効ハンドラ。ConfCredIRS に属性として埋め込むことができ、失効処理に用いられる。
- witrhIRS: revokeIRS のアキュムレータ証人。
- revokeAML: AML からの失効ハンドラ。ConfCredAML に属性として埋め込むことができ、失効処理に用いられる。
- witrhAML: revokeAML のアキュムレータ証人。
- accIRS: IRS 用アキュムレータ。
- CommRevokeIRS: revokeIRS へのコミットメント。保持者は証明のリンク可能性を避けるため、失効ごとに新しいコミットメントを生成する。
- accAML: AML 用アキュムレータ。
- CommRevokeAML: revokeAML へのコミットメント。保持者は証明のリンク可能性を避けるため、失効ごとに新しいコミットメントを生成する。

ZkPoK{ (witness: rhIRS, openrhIRS, wrhIRS, rhAML, openrhAML, wrhAML; statements: CIRS, aIRS, CAML, aAML) : Predicate: ... }

述語:

- CIRS は (openrhIRS, rhIRS) への有効なコミットメントである。
- rhIRS は証人 wrhIRS の下でアキュムレータ aIRS の一部である。
- rhIRS は CertIRS の属性である。
- CAML は (openrhAML, rhAML) への有効なコミットメントである。
- rhAML は証人 wrhAML の下でアキュムレータ aAML の一部である。
- rhAML は CertAML の属性である。

myCred は myID、sigIRS、sigAML などと関連付けられない。部分的侵害にも耐える。たとえ IRS が myID と sigIRS を漏えいしても、それを用いて sigAML を明らかにしたり、myID と myCred を関連付けたりすることはできない。

4.6 資産移転

4.6.1 プライバシー保護型資産移転と残高更新

本節では、代替可能または非代替的デジタル資産を移転するため、ZK 証明 (ZKP) を用いて私的資産移転を促進する 2 つのユースケースを検討する。これらのユースケースは、利用者が取引の基礎詳細を明らかにせず、取引が有効であることを証明しなければならないプライバシー保護型暗号通貨に動機づけられている。2 つの異なるフレームワークを探索し、それぞれに必要な技術詳細と証明システムを概説する。

代替可能デジタル資産の追跡には、資産の所有権を個別に追跡する方法と、口座残高を追跡する方法という 2 つの主要パラダイムがある。Bitcoin システムは、UTXO モデルとして知られる資産追跡形式を導入した。そこでは未使用取引出力 (Unspent Transaction Outputs) が概ね一回限り使用される「コイン」に対応する。一方 Ethereum は残高モデルを用いる。各口座は関連する残高を持ち、資金移転は送信者残高を減少させ、受信者残高を対応して増加させることに対応する。

これら 2 つのモデルは利用者にとって異なるプライバシー含意を持ち、取引が有効であることを保証する異なる規則を持つ。したがって、プライバシー保護型取引を促進する ZK 証明システム構築の要件とアーキテクチャは、各モデルでやや異なる。以下では各モデルを別々に探索する。

最も単純な形では、資産追跡モデルは非代替的資産を追跡するために使用できる。このシナリオでは、取引は単なる資産所有権の移転であり、送信者がその資産の現在所有者であれば取引は有効である。残高モデル（代替可能資産）では、各口座が残高を持ち、取引は送信者の口座残高を減少させると同時に受信者の残高を増加させる。「残高」モデルでは、取引が有効である条件は、(1) 送信者残高の減少額が受信者残高の増加額と等しいこと、(2) 送信者残高が非負のままであること、(3) 送信者の鍵で取引が署名されていることである。

4.6.2 資産追跡モデルにおけるゼロ知識証明

本節では、資産追跡 (UTXO) モデルにおけるプライバシー保護型取引のための単純な ZK 証明システムを説明する。概説するアーキテクチャは本質的には ZCash システムの単純化である。主な単純化は、各資産（「コイン」）が分割不可能であると仮定することにある。言い換えると、各資産には所有者があるが、関連する価値はなく、取引は単に資産所有権の移転である。

動機: 資産の所有権を公に明らかにせず、資産が決して生成または破壊されないことを保証しながら、利害関係者が非代替的資産を移転できるようにする。

当事者: このシステムには 3 種類の当事者がいる。送信者、受信者、分散された検証者集合である。送信者は取引と妥当性証明を生成する。（分散された）検証者は検証者として振る舞い、取引の妥当性を確認する。受信者は直接的な役割を持たないが、送信者は取引に受信者の公開鍵を含めなければならない。

何が証明されるか: 高水準では、送信者は検証者に取引が有効であることを納得させるため、3 つの事柄を証明しなければならない。

- 移転される資産（または「note」）が送信者に所有されていること。各資産は一意な文字列で表される。
- 送信者が入力 note の秘密支出鍵を持っており、資産を送信する権限を持つこと。
- 入力資産の秘密支出鍵が、取引全体に対する署名と暗号学的にリンクしており、これらの秘密鍵を知らない当事者が取引を変更できないこと。

検証者に必要な情報:

- 検証者は、証明システムで用いられる CRS へアクセスできる必要がある。
- 検証者は取引履歴全体へアクセスする必要がある。これには後述するすべての UTXO、コミットメント、nullifier が含まれる。この履歴は分散台帳（Bitcoin ブロックチェーンなど）に保存できる。

可能な攻撃:

- **CRS 侵害:** 攻撃者が CRS 生成に用いられた秘密乱数を学ぶと、基礎システムにおいて証明を偽造できる。
- **台帳攻撃:** 取引検証には取引履歴全体を読む必要があるため、誤った取引履歴ビューを持つ検証者は、不正な取引を有効として受理するよう納得させられ得る。
- **再識別攻撃:** このシステムに ZKP を組み込む目的は、送信者と受信者の身元を明らかにせずに取引を促進することである。匿名性が不要であれば、Bitcoin のように ZKP を完全に避けられる。このシステムは各取引の送信者と受信者を隠すが、取引が発生した事実（および発生時刻）は公開記録されるため、個々の利用者の再識別に用いられる可能性がある。
- **IP レベル攻撃:** ネットワーク・トラフィックを監視することで、攻撃者は取引を特定の送信者または受信者にリンクしたり（各取引には送信者と受信者の通信が必要）、公開鍵（仮名）を現実世界のアイデンティティにリンクしたりできる。
- **中間者攻撃:** 攻撃者は送信者をだまして、資産を「不正な」公開鍵へ移転させる可能性がある。

セットアップ・シナリオ: このシステムは本質的に、分割不可能資産向けに修正された Zcash 証明システムの単純化版である。各資産は一意な `AssetID` で表され、簡単化のため、資産集合全体が配布済みであり、資産は決して生成または破壊されないと仮定する。

任意時点で、システムの公開状態は「asset notes」の集合からなる。これらの note は Merkle Tree の葉として保存され、各葉は一意な `assetID` で表される単一の分割不可能資産を表す。より詳しくは、「note」は `{Nullifier, publicKey, assetID}` へのコミットメントであり、`publicKey` が `assetID` を「所有」していることを示す。

主な取引型: 現在所有者 A から新所有者 B への資産送信。

セキュリティ目標:

- 現在所有者だけが資産を移転できる。
- 資産は決して生成または破壊されない。

プライバシー目標: 理想的には、システムは利用者の所有権と取引パターンに関するすべての情報を隠すべきである。下記で概説するシステムはそのような高水準のプライバシーを達成しないが、以下のプライバシー保護機能を達成する。

- 取引は公開可視であり、誰でも取引が発生したことを見られる。
- 取引はどの資産が移転されているかを明らかにしない。
- 取引は送信者または受信者の身元（公開鍵）を明らかにしない。
- 制限: 前所有者は資産が移転された時点を知ることができる。緩和策として、資産を受け取った後に自分自身へ送ることが考えられる。

移転の詳細: 各取引は、資産所有権を現在所有者から新所有者へ移すことを意図する。本節では、取引妥当性を保証するために用いられる証明を概説する。この説明全体で、青色はプロトコル／命題内でグローバルかつ公開可視のデータを表し、赤色は秘密データ、たとえば証明者が保持する秘密証人または現在所有者と新所有者の間で共有される情報を表す。

現在所有者 A は以下の情報を持つ。

- 公開鍵 `publicKey` と対応する秘密鍵 `secretKey`。
- 移転対象資産に対応する `assetID`。
- 資産に対応する MerkleTree 内の note。
- コミットメント `(Nullifier, assetID, publicKey)` を開く方法、および新所有者 B の `publicKeyOut` を知る。

現在所有者 A は以下を生成する。

- 新しい `NullifierOut`。
- 新しいコミットメント `commitment(NullifierOut, assetID, publicKey)`。

現在所有者 A は以下を送る。

- B へ私的に: `NullifierOut, publicKeyOut, assetID`。
- ブロックチェーンへ公開に: `Nullifier, comOut, ZKProof`（ZKProof の構造は以下に概説する）。

`Nullifier` が MerkleTree に存在せず、かつ ZKProof が検証されるなら、`comOut` が MerkleTree に追加される。

ゼロ知識証明の構造: 証明構造を記述するため、Camenisch-Stadler 記法の修正版を用いる。

公開状態: Notes の MerkleTree。`Note = Commitment to { Nullifier, publicKey, assetID }`。

```
ZKProof = ZkPoK_pp{ (witness: publicKey, publicKeyOut, merkleProof, NullifierOut, com, assetID, sig;
statement: MerkleTree, Nullifier, comOut) : predicate: ... }
```

述語:

- `com` は `merkleProof` を用いて MerkleTree に含まれる。
- `com` は `(Nullifier, publicKey, assetID)` へのコミットメントである。
- `comOut` は `(NullifierOut, publicKeyOut, assetID)` へのコミットメントである。
- `sig` は `publicKey` に対する `comOut` 上の署名である。

4.6.3 残高モデルにおけるゼロ知識証明

本節では、「残高モデル」において代替可能資産を私的に移転する単純なシステムを概説する。このシステムは本質的に zkLedger の単純化版である。システム状態は各利用者の（暗号化された）口座残高である。各口座残高は、加法準同型暗号システムを用いて、口座保持者の鍵の下で暗号化される。取引は送信者の口座残高を減少させ、受信者の口座を対応する量だけ増加させる。利用者数が固定され事前に知られている場合、取引はすべての口座残高を同時に更新することで、送信者と受信者に関するすべての情報を隠せる。これは高いプライバシーを提供し、zkLedger が採用するアプローチである。利用者集合が非常に大きい、動的に変化する、または送信者に知られていない場合、送信者は「匿名集合」を選ばなければならず、取引はその匿名集合のメンバーが関与したことを明らかにするが、取引額やどのメンバーが関与したかは明らかにしない。提示を簡単にするため、ここではシステム内の当事者集合が固定で事前に知られている zkLedger のようなモデルを仮定する。ただし、この仮定は関与するゼロ知識証明の詳細には影響しない。

動機: 各実体が私的な口座残高を維持し、取引が送信者残高を減少させ、受信者残高を対応する量だけ増加させる。各取引がすべての口座残高を更新すると仮定するため、取引の発信元、宛先、価値に関するすべての情報は完全に隠される。プロトコルによって明らかになる唯一の情報は、取引が発生したという事実である。

当事者:

- 代替可能資産を匿名で移転したい n 人の利害関係者集合。
- 取引を開始する利害関係者は「証明者」または「送信者」と呼ばれる。
- 受信者または複数受信者は、取引内で特別な役割を持たない。
- システムの公開状態を維持する検証者集合（ブロックチェーンや他の DLT を用いるなど）。

何が証明されるか: 送信者は、提案された取引が「有効」であり、システム状態を新しい取引を反映するよう更新すべきであることを、検証者に納得させなければならない。取引は n 個の暗号文 (`c_1, ..., c_n`) からなり、`c_i = Enc_pk(x_i)` である。取引が有効である条件は以下である。

- すべてのコミット値の和が 0 である、すなわち `x_1 + ... + x_n = 0`。
- 送信者がすべての負の `x_i` に対応する秘密鍵を所有している。
- 更新後、すべての口座残高が正のままである。

検証者に必要な情報:

- 検証者は証明システムで用いられる CRS へアクセスできる必要がある。
- 検証者はシステムの現在状態、すなわち n 個の暗号化口座残高の現在ベクトルへアクセスする必要がある。この状態は分散台帳に保存できる。

可能な攻撃:

- **CRS 侵害:** 攻撃者が CRS 生成に用いられた秘密乱数を学ぶと、基礎システムにおいて証明を偽造できる。
- **台帳攻撃:** 取引検証にはシステムの現在状態（暗号化口座残高）を知る必要があるため、現在状態の誤ったビューを持つ検証者は、不正な取引を有効として受理するよう納得させられ得る。
- **再識別攻撃:** このシステムに ZKP を組み込む目的は、送信者と受信者の身元を明らかにせずに取引を促進することである。匿名性が不要であれば、Bitcoin のように ZKP を完全に避けられる。このシステムは各取引の送信者と受信者を隠すが、取引が発生した事実（および発生時刻）は公開記録されるため、個々の利用者の再識別に用いられる可能性がある。
- **IP レベル攻撃:** ネットワーク・トラフィックを監視することで、攻撃者は取引を特定の送信者または受信者にリンクしたり（各取引には送信者と検証者の間の通信が必要）、公開鍵（仮名）を現実世界のアイデンティティにリンクしたりできる。
- **中間者攻撃:** 攻撃者は送信者をだまして、資産を「不正な」公開鍵へ移転させる可能性がある。利用者基盤が固定で、すべての公開鍵が事前に知られている状況では、この懸念はやや小さいかもしれない。

セットアップ・シナリオ: 利用者数 n は固定である。利用者 i は既知の公開鍵 `pk_i` を持つ。各利用者は、現在残高を自分の `pk` の下で加法準同型暗号化したものとして維持される口座残高を持つ。各取引は、各残高が取引によって増減されるべき量に対応する n 個の暗号化のリストである。貨幣が決して生成または破壊されないことを保証するため、暗号化された取引内の平文は和が 0 でなければならない。すべての口座残高は非負値で初期化されていると仮定する。

主な取引型: 利用者 i から利用者 j への資金移転。

セキュリティ目標:

- 口座残高は、その口座の所有者によってのみ減少させられる。
- 口座残高は常に非負のままである。
- システム内の貨幣総額は一定である。

プライバシー目標: 理想的には、システムは利用者の所有権と取引パターンに関するすべての情報を隠すべきである。下記で概説するシステムはそのような高水準のプライバシーを達成しないが、以下のプライバシー保護機能を達成する。

- 取引は公開可視であり、誰でも取引が発生したことを見られる。
- 取引はどの資産が移転されているかを明らかにしない。
- 取引は送信者または受信者の身元（公開鍵）を明らかにしない。
- 制限: 取引時刻は漏れる。

移転の詳細: 各取引は、システム内の現在の口座残高を更新することを意図する。以下では、取引の妥当性を保証するために用いられる証明を概説する。青の表現はプロトコル／命題内でグローバルかつ公開可視の情報を表し、赤の表現は秘密情報、たとえば証明者が保持する秘密証人を表す。

送信者 A は以下の情報を持つ。

- 公開鍵 `pk_1, ..., pk_n`。
- `publicKey_i` に対応する `secretKey_i` と、利用者 `j` へ移転する値 `x_j`。
- 送信者自身の現在口座残高 `y_i`。

送信者 A は以下を生成する。

- 暗号文ベクトル `C_1, ..., C_n`。ここで `C_t = Enc_{pk_t}(x_t)`。

送信者 A は以下を送る。

- 暗号文ベクトル `C_1, ..., C_n` と、後述する `ZKProof` をブロックチェーンへ送る。

ZK 回路:

公開状態: システムの現在状態、すなわち暗号化口座残高ベクトル `B_1, ..., B_n`。

```
ZKProof = ZkPoK_pp{ (witness: i, x_1, ..., x_n, sk; statement: C_1, ..., C_n) : predicate: ... }
```

述語:

- 各 `t=1,...,n` について、`C_t` は公開鍵 `pk_t` の下で `x_t` への暗号化である。
- `x_1 + ... + x_n = 0`。
- 各 `t=1,...,n` について、`x_t >= 0` または `sk` が `pk_t` に対応する。
- 各 `t=1,...,n` について、`x_t >= 0` または現在残高 `B_t` が `|x_t|` 以上の値を暗号化している。

4.7 規制遵守

4.7.1 概要

ゼロ知識プロトコルが有用である重要な応用パターンは、規制当局が、規制対象当事者が管理する何らかの項目を監視したり、それに関連するリスクを評価したりしたい設定である。その一例は、口座保持者が税を正しく支払っているか、銀行または他の金融実体が支払能力を持つか、あるいは安定しているか、である。

このような場合、規制当局は「結論」を知ることに関心がある。これは通常、より詳細な基礎データに対する何らかの集計尺度から導出されるが、必ずしもすべての詳細を知る必要はない。たとえば「銀行は過剰な融資を抱えているか」という質問への答えは最終的には 1 ビット（Yes/No）で答えられ、銀行が提供した各融資の詳細、受取人、その所得、その他関連データを明らかにせずに答えられる。

このようなシナリオの追加例には、以下を確認することが含まれる。

- ある会社または個人が税を適切に支払ったこと。
- ある融資が過度に危険ではないこと。
- ある記録保持者がデータを保持していること（そのデータを明らかにしたり送信したりせずに）。
- 飛行機が適切に整備され、飛行に適していること。

ゼロ知識証明の利用により、集計結果の正しさを示す証明を生成できる。考え方は、次のような命題を示すことである。

「記録へのコミットメント（場合によってはブロックチェーン上）が存在し、それが結果の正しさを示している。」

計算へ供給されるデータを信頼すること。隠されたデータ上の計算が価値を証明するためには、供給されるデータも根拠づけられていなければならない。そうでなければ、計算の正しさを証明しても意味がない。この点を具体化するため、何らかの隠されたデータから計算された信用スコアを考える。信用スコアは、何らかの財務記録から正しく計算され得る。しかし、これらの記録が証明の受信者に公開されない場合、受信者はそれらが捏造されていないことをどのように信頼できるだろうか。

証明に用いられるデータは一般に、証明者とは別で、証明者と結託する可能性が低い当事者によってコミットされるべきである。先ほどの例を続けると、個人は、以前の貸し手によって生成されたデータ・コミットメントに基づき、自分が高い信用スコアを持つことを証明できる。以前の貸し手をこのような方法で正確に報告するものとして信頼できるのかという疑問はあり得るが、これは伝統的な信用スコアリングでも暗黙に置かれている仮定である。

後にゼロ知識を用いて監査されるプロセスの運用・管理に関するコミットメントを蓄積する必要性は、コミットメントを不可逆的な形で置けるブロックチェーン・システムとよく合うことが多い。コミットメントは秘匿性を持つため、このような公開共有データはプライバシーを侵害しないが、データの真実性への信頼を固定するために使用できる。

4.7.2 詳細な例: 航空機の遵守証明

運航者が航空機を飛ばしており、その航空機の整備作業ログを保持している。これらの記録は、異なる会社が製造した可能性のある異なる部品に関するものである。整備および飛行記録は、世界各地の技術者によって証明される（彼らが運航者と結託しないと仮定する）。

規制当局は、航空機が特定の規則集合に従って飛行を許可されていることを知りたい（Volkswagen の排出ガス不正事件を想起されたい）。

問題: 現在、規制当局は記録を、ときどきだけ自分で確認する（または監査者に確認させる）。遵守が「リアルタイム」で強制されるシステムへ移行したいが、素朴に行うと航空機のリアルタイム運用が明らかになる。

なぜゼロ知識が必要か。事業上機微な航空機の基礎運用データを明らかにせず、規制が守られていることを証明したい。規制当局自身も、データを保持しないことを好む（記録喪失からの責任とリスク）。また可能な範囲で企業に自己規制させることを好む。

技術者と運航者が結託しないという点を超えて、脅威モデルは何か。部品製造者はどうか。規制当局はどうか。部品製造者同士に敵対関係はあるか。このスキームは、航空規制のように曖昧でない規制に対して機能する。一部の場合、規則は意図的に曖昧にされ、解釈の余地を残している。

4.7.3 プロトコル高水準

当事者:

- **運航者／規制対象当事者:** 規制に適合する必要がある運用を行う。たとえば航空機を運航する航空会社運航者。
- **リスク負担者／規制当局:** すべての規制対象当事者が規則に適合することを検証し、リスクが進化したとき規則を更新する。たとえば FAA は、すべての航空機が常時 airworthy であることを規制・強制する。航空機をリースする航空機所有者にとっては、運用と整備が資産価値を低下させないことを知りたい。同様に、航空機が融資の担保である場合には、融資を行った銀行も同じ関心を持つ。
- **発行者／データを証明する第三者:** 部品を検査した技術者、各地に航空機が到着したことを証明する航空管制官、センサー値に署名した搭載機器など。

運航者が証明すること:

- 最新の整備データが航空機の airworthiness を示すことを規制当局へ証明する。
- その航空機の整備状態が、銀行の提供する式に従ってある価値を持つことを銀行へ証明する。

プライバシー要件:

- 運航者: 運用の詳細および資産整備状態を競合他社へ明らかにしない。
- 航空機アイデンティティ: 規制当局と技術者を除くすべての当事者から匿名に保たれなければならない。
- 技術者のアイデンティティ: 規制当局から匿名に保たれなければならない。ただし必要であれば、運航者は規制当局が報告を検証するためにコミットメントを開くよう求められる。

証明述語: 「運航者は航空機の所有者であり、規制規則への遵守を証明する署名付きデータを知っている。すなわち、すべての構成要素が飛行に安全である。」

- 飛行機は部品 `x_1, ..., x_n` から構成され、それぞれについて以下が成り立つ。
  - 部品を確認して「問題ない」と署名した技術者による正当な証明がある。
  - 技術者による最新の証明が新しい。すなわち検査のタイムスタンプが日付 `D` より前である。

公開／秘密データ:

- 秘密: (i) 運航者のアイデンティティ、(ii) 航空機記録、(iii) 技術者の検査報告、(iv) 報告に署名した技術者のアイデンティティ。
- 公開: 航空機記録へのコミットメント。

公開台帳にコミットされるのは、飛行マイルを含む航空機の記録、および整備士による修理／検査を証明する記録である。デコミットメントは運航者に伝えられる。これらの記録は飛行機の識別子を参照する。

飛行機が飛ぶたびに、古い飛行機記録は無効化され、追加マイルを含む新しい記録がコミットされる必要がある。「airworthiness」の証明が必要なとき、運航者は各部品について、走行距離が交換を必要とする距離を下回ること、または技術者がその部品を交換したこと（技術者によってコミットされた記録を指す）を証明する。

ガジェット・レベルでは以下である。

- 証明者は航空機記録のデコミットメントの知識を証明する（decommitment）。
- その記録はブロックチェーン上の記録集合に属する（集合メンバーシップ）。
- 部品に関する記録のデコミットメントの知識も証明し（decommitment）、それらも台帳上のコミットメント集合に属する（集合メンバーシップ）。
- 航空機記録は失効していない、すなわち最新のものである（公開された nullifier 集合に対する集合非メンバーシップ）。
- 部品内に記された飛行機 ID は飛行機記録内の飛行機 ID と同じである（等価性）。
- 飛行機の走行距離は、各部品を交換する必要がある走行距離より低い（範囲証明）。または、部品が技術者によって交換されたことを示す記録が存在する（集合メンバーシップ）。技術者の署名を検証する（ZK の外でリング署名を用いてもよい）。

4.8 結論

- 資産移転と規制遵守は、アイデンティティ・フレームワーク内で利用でき、それらの追加によりフレームワークを補完できる。
- データ・コミットメントへの参照を保存するためにブロックチェーンなどの外部オラクルを用いることができる。

謝辞

本コミュニティ参照文書の開発は、多数の個人の支援を受けている。

Version 0。第 1 回 ZKProof ワークショップ（Boston, May 2018）の「proceedings」が本書の初期基盤となった。貢献は 3 つのトラックに整理された。

- **Implementation track**。 Chairs: Sean Bowe, Kobi Gurkan, Eran Tromer. Participants: Benedikt Bünz, Konstantinos Chalkias, Daniel Genkin, Jack Grigg, Daira Hopwood, Jason Law, Andrew Poelstra, abhi shelat, Muthu Venkitasubramaniam, Madars Virza, Riad S. Wahby, Pieter Wuille.
- **Applications Track**。 Chairs: Daniel Benarroch, Ran Canetti, Andrew Miller. Participants: Shashank Agrawal, Tony Arcieri, Vipin Bharathan, Josh Cincinnati, Joshua Daniel, Anuj Das Gupta, Angelo De Caro, Michael Dixon, Maria Dubovitskaya, Nathan George, Brett Hemenway Falk, Hugo Krawczyk, Jason Law, Anna Lysyanskaya, Zaki Manian, Eduardo Morais, Neha Narula, Gavin Pacini, Jonathan Rouach, Kartheek Solipuram, Mayank Varia, Douglas Wikstrom, Aviv Zohar.
- **Security track**。Chairs: Jens Groth, Yael Kalai, Muthu Venkitasubramaniam. Participants: Nir Bitansky, Ran Canetti, Henry Corrigan-Gibbs, Shafi Goldwasser, Charanjit Jutla, Yuval Ishai, Rafail Ostrovsky, Omer Paneth, Tal Rabin, Maryana Raykova, Ron Rothblum, Alessandra Scafuro, Eran Tromer, Douglas Wikström.

Version 0.1。第 2 回 ZKProof ワークショップに先立ち、ZKProof 組織チームは、開発中の文書について NIST からフィードバックを求めた。NIST PEC チーム（Luís Brandão, René Peralta, Angela Robinson）はその後、「Community Reference Document」のさらなる開発のため、28 件のコメント／提案を含む“NIST comments on the initial ZKProof documentation”[BPR19] を作成した。Luís Brandão は proceedings を LaTeX 版へ移植し、インラインコメントを付けた。これが version 0.1 と名付けられた。

Version 0.2。 Version 0.1 から version 0.2 への貢献は、第 2 回 ZKProof Workshop（Berkeley, April 2019）で開始された編集プロセスに従った。いくつかの提案された貢献は、Workshop Notes [ZKP19] に記録されているように、書記、モデレータ、参加者の協力によって可能になったワークショップ内の breakout discussions から生じた。実際の内容貢献はその後、Yu Hang, Eduardo Morais, Justin Thaler, Ivan Visconti, Riad Wahby, Yupeng Zhang などの貢献者、および NIST PEC チーム（Luís Brandão, René Peralta, Angela Robinson）と編集者チーム（Daniel Benarroch, Luís Brandão, Eran Tromer）によって開発された。変更、貢献、貢献者の詳細説明は、コミュニティ参照文書の“diff”version に現れる。

Version 0.3。主な更新はパラダイム章の改訂であった。古い節“Taxonomy of Constructions”は、新しい“background”節に置き換えられ、その後“IT Proof Systems”と“Cryptographic Compilers”の 2 節が追加された。この変更は、Yuval Ishai による“Zero-Knowledge Proofs from Information-Theoretic Proof Systems”(parts 1 and 2) という 2 つのブログ投稿 [Ish20] に着想を得て、主としてそれらから適応された。Linear IOP の下の IP based 小節の内容は Justin Thaler によって提供された。改訂章は ZKProof Editors team（Daniel Benarroch, Luís Brandão, Mary Maller, Eran Tromer）によって構成・編集された。公開レビュー段階中など、いくつかの編集上の修正が外部から提案された。これは“Call for contributions to the ZKProof Community Reference —Review cycle 2020: from version 0.2 to 0.3.”[ZKP-CC20] に記載されている。Jens Groth からの提案は、1.2 節の表 1.1 を改善した。さまざまな読者からの他のコメントにも感謝する。受領した提案の一部の統合は将来のバージョンに残されている。

その他。ZKProof イニシアチブにこれまで協力したすべての人へ一般的な謝意を表する。これには、ワークショップの講演者、参加者、主催者、スポンサー、ならびに ZKProof steering committee と program committee のメンバー、オンライン ZKProof forum の参加者が含まれる。ZKProof に関する詳細情報は zkproof.org ウェブサイトで入手できる。

参考文献（原文書誌情報）

以下の参考文献一覧は、固有の書誌情報として原文表記を保持している。引用キー、論文題名、著者名、URL、DOI は、原典照合性を損なわないよう原則として翻訳していない。

[AHIKV17] B. Applebaum, N. Haramaty-Krasne, Y. Ishai, E. Kushilevitz, and V. Vaikuntanathan. “Low-Complexity Cryptographic Hash Functions.”In: 8th Innovations in Theoretical Computer Science Conference (ITCS 2017). Ed. by C. H. Papadimitriou. Vol. 67. Leibniz International Proceedings in Informatics (LIPIcs). Pub. by Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017, 7:1–7:31. doi: 10.4230/LIPIcs.ITCS.2017.7. Also at ia.cr/2017/036 (Cited on p. 35).

[AHIV17] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup.”In: Proc. 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS ’17. Pub. by ACM, 2017, pp. 2087–2104. doi: 10.1145/3133956. 3134104 (Cited on p. 41).

[AW09] S. Aaronson and A. Wigderson. “Algebrization: A New Barrier in Complexity Theory.”In: ACM Trans. Comput. Theory 1.1 (February 2009). doi: 10 . 1145 / 1490270 . 1490272. Also at ECCC TR08/005 (Cited on p. 34).

[BBBF18] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch. “Verifiable Delay Functions.”In: Ad- vances in Cryptology —CRYPTO 2018. Ed. by H. Shacham and A. Boldyreva. Pub. by Springer International Publishing, 2018, pp. 757–788. Also at ia.cr/2018/601 (Cited on p. 60).

[BBCGI19] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai. “Zero-Knowledge Proofs on Secret-Shared Data via Fully Linear PCPs.”In: Advances in Cryptology —CRYPTO 2019. Ed. by A. Boldyreva and D. Micciancio. Pub. by Springer International Publishing, 2019, pp. 67–97. doi: 10.1007/978-3-030-26954-8_3. Also at ia.cr/2019/188 (Cited on pp. 32, 34, 40).

[BBHR18] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. “Fast Reed-Solomon Interactive Oracle Proofs of Proximity.”In: 45th International Colloquium on Automata, Languages, and Pro- gramming (ICALP 2018). Ed. by I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella. Vol. 107. Leibniz International Proceedings in Informatics (LIPIcs). Pub. by Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018, 14:1–14:17. doi: 10.4230/ LIPIcs.ICALP.2018.14. Also at ECCC TR17/134 (Cited on p. 40).

[BBHR18] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR Cryptology ePrint Archive, ia.cr/ 2018/046. 2018 (Cited on p. 40).

[BCCGLRT17] N. Bitansky, R. Canetti, A. Chiesa, S. Goldwasser, H. Lin, A. Rubinstein, and E. Tromer. “The Hunting of the SNARK.”In: Journal of Cryptology 30 (2017), pp. 989–1066. doi: 10 . 1007 / s00145 - 016 - 9241 - 9. Also at ia . cr / 2014 / 580. Merging of (i) “From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again”, ITCS ’12, pp. 326-349, ACM, 2012, DOI:10.1145/2090236.2090263, ia.cr/2011/443, and (ii) “Delegation of Computation without Rejection Problem from Designated Verifier CS-Proofs”, 2011, ia.cr/2011/456 (Cited on p. 21).

[BCCT13] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. “Recursive Composition and Bootstrapping for SNARKS and Proof-carrying Data.”In: 45th ACM Symposium on the Theory of Computing. STOC ’13. Pub. by ACM, 2013, pp. 111–120. doi: 10.1145/2488608.2488623. Also at ia.cr/ 2012/095 (Cited on pp. 42, 65).

[BCDEK+14] P. Bichsel, J. Camenisch, M. Dubovitskaya, R. R. Enderlein, S. Krenn, I. Krontiris, A. Lehmann, G. Neven, J. D. Nielsen, C. Paquin, F.-S. Preiss, K. Rannenberg, A. Sabouri, and M. Stausholm. D2.2 —Architecture for Attribute-based Credential Technologies —Final Version. Ed. by A. Sabour. August 2014. https://abc4trust.eu/download/Deliverable_D2.2.pdf (Cited on p. 67).

[BCDLRSY17] F. Baldimtsi, J. Camenisch, M. Dubovitskaya, A. Lysyanskaya, L. Reyzin, K. Samelin, and S. Yakoubov. “Accumulators with Applications to Anonymity-Preserving Revocation.”In: 2017 IEEE European Symposium on Security and Privacy (EuroS&P). April 2017, pp. 301–315. doi: 10.1109/ EuroSP.2017.13. Also at ia.cr/2017/043 (Cited on pp. 67, 75).

[BCGGHJ17] J. Bootle, A. Cerulli, E. Ghadafi, J. Groth, M. Hajiabadi, and S. K. Jakobsen. “Linear-Time Zero-Knowledge Proofs for Arithmetic Circuit Satisfiability.”In: Advances in Cryp- tology —

ASIACRYPT 2017. Ed. by T. Takagi and T. Peyrin. Pub. by Springer International Publishing, 2017, pp. 336–365. doi: 10.1007/978-3-319-70700-6_12. Also at ia.cr/2017/872 (Cited on pp. 32, 35, 41).

[BCGGMTV14] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. "Zerocash: Decentralized Anonymous Payments from Bitcoin."In: Proc. 2014 IEEE Symposium on Security and Privacy. SP '14. May 2014, pp. 459–474. doi: 10.1109/SP.2014.36. Also at ia.cr/2014/349 (Cited on pp. 56, 67).

[BCGGRS19] E. Ben-Sasson, A. Chiesa, L. Goldberg, T. Gur, M. Riabzev, and N. Spooner. "Linear-Size Constant-Query IOPs for Delegating Computation."In: Theory of Cryptography. Ed. by D. Hofheinz and A. Rosen. TCC 2019. Pub. by Springer International Publishing, 2019, pp. 494–521. doi: 10.1007/978-3-030-36033-7_19. Also at ia.cr/2019/1230 (Cited on p. 40).

[BCGT13] E. Ben-Sasson, A. Chiesa, D. Genkin, and E. Tromer. "On the Concrete Efficiency of Probabilistically-checkable Proofs."In: Proc. 45th Annual ACM Symposium on Theory of Computing. STOC '13. Pub. by ACM, 2013, pp. 585–594. doi: 10 . 1145 / 2488608 . 2488681. Also at ECCC TR12/045 (Cited on p. 43).

[BCGTV13] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza. "SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge."In: Advances in Cryptology —CRYPTO 2013. Ed. by R. Canetti and J. A. Garay. Pub. by Springer Berlin Heidelberg, 2013, pp. 90–108. doi: 10.1007/978-3-642-40084-1_6. Also at ia.cr/2013/507 (Cited on pp. 28, 50, 51).

[BCIKS20] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf. "Proximity Gaps for Reed–Solomon Codes."In: 2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS). 2020, pp. 900–909. doi: 10.1109/ FOCS46700.2020.00088. Also at ia.cr/ 2020/654 (Cited on p. 40).

[BCIOP13] N. Bitansky, A. Chiesa, Y. Ishai, R. Ostrovsky, and O. Paneth. "Succinct Non- interactive Arguments via Linear Interactive Proofs."In: Theory of Cryptography. Ed. by A. Sahai. Pub. by Springer Berlin Heidelberg, 2013, pp. 315–333. doi: 10.1007/978-3-642-36594-2_18. Also at ia.cr/ 2012718 (Cited on pp. 35, 38, 39).

[BCMS20] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. "Recursive Proof Composition from Accumulation Schemes."In: TCC 2020: Theory of Cryptography. Ed. by R. Pass and K. Pietrzak. Vol. 12551. LNCS. Pub. by Springer, 2020, pp. 1–18. doi: 10.1007/978-3-030-64378-2_1 (Cited on p. 42).

[BCRSVW19] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward. "Aurora: Transparent Succinct Arguments for R1CS."In: Advances in Cryptology —EUROCRYPT 2019. Ed. by Y. Ishai and V. Rijmen. Pub. by Springer International Publishing, 2019, pp. 103–128. doi: 10.1007/978-3-030-17653-2_4. Also at ia.cr/2018/828 (Cited on pp. 33, 40).

[BCS16] E. Ben-Sasson, A. Chiesa, and N. Spooner. "Interactive Oracle Proofs."In: Theory of Cryptography. Ed. by M. Hirt and A. Smith. Pub. by Springer Berlin Heidelberg, 2016, pp. 31–60. doi: 10.1007/978-3-662-53644-5_2. Also at ia.cr/2016/116 (Cited on pp. 32, 42, 43).

[BCTV14] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves."In: Advances in Cryptology —CRYPTO 2014. Ed. by J. A. Garay and R. Gennaro. Pub. by Springer Berlin Heidelberg, 2014, pp. 276–294. doi: 10.1007/978-3-662-44381-1_16. Extended version at Journal Algorithmica (79), pp. 1102–1160, Dec-2017, doi:10.1007/s00453- 016-0221-0. Also at ia.cr/2014/595 (Cited on pp. 42, 63, 65).

[BCTV14] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. "Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture."In: USENIX Security 2014. 2014, pp. 781–796. Also at ia.cr/2013/879 (Cited on p. 28).

[BDFG20] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. Halo Infinite: Recursive zk-SNARKs from any Additive Polynomial Commitment Scheme. IACR Cryptology ePrint Archive, ia.cr/2020/ 1536. 2020 (Cited on p. 42).

[BFLS91] L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy. "Checking Computations in Polylogarithmic Time."In: Proc. 23rd Annual ACM Symposium on Theory of Computing. STOC '91. Pub. by Association for Computing Machinery, 1991, pp. 21–32. doi: 10 . 1145 / 103418 . 103428 (Cited on p. 34).

[BFS20] B. Bünz, B. Fisch, and A. Szepieniec. "Transparent SNARKs from DARK Compilers."In: Advances in Cryptology —EUROCRYPT 2020. Ed. by A. Canteaut and Y. Ishai. Pub. by Springer

International Publishing, 2020, pp. 677–706. doi: 10.1007/978-3-030-45721-1_24. Also at ia.cr/2019/1229 (Cited on pp. 33, 41).

[BGGHKMR90] M. Ben-Or, O. Goldreich, S. Goldwasser, J. Håstad, J. Kilian, S. Micali, and P. Rogaway. "Everything Provable is Provable in Zero-Knowledge."In: Advances in Cryptology — CRYPTO'88. Ed. by S. Goldwasser. Vol. 403. LNCS. Pub. by Springer New York, 1990, pp. 37–56. doi: 10.1007/0-387-34799-2_4 (Cited on p. 39).

[BGH19] S. Bowe, J. Grigg, and D. Hopwood. Recursive Proof Composition without a Trusted Setup. IACR Cryptology ePrint Archive, ia.cr/2019/1021. 2019 (Cited on p. 42).

[BGHSV04] E. Ben-Sasson, O. Goldreich, P. Harsha, M. Sudan, and S. Vadhan. "Robust PCPs of Proximity, Shorter PCPs and Applications to Coding."In: Proc. 36th Annual ACM Symposium on Theory of Computing. STOC '04. Pub. by Association for Computing Machinery, 2004, pp. 1–10. doi: 10.1145/1007352.1007361. Also at ECCC TR04/021 (Cited on p. 34).

[BGKS20] E. Ben-Sasson, L. Goldberg, S. Kopparty, and S. Saraf. "DEEP-FRI: Sampling Outside the Box Improves Soundness."In: 11th Innovations in Theoretical Computer Science Conference (ITCS 2020). Ed. by T. Vidick. Vol. 151. Leibniz International Proceedings in Informatics (LIPIcs). Pub. by Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2020, 5:1–5:32. doi: 10.4230/LIPIcs.ITCS.2020.5. Also at ia.cr/2019/336 (Cited on p. 40).

[BGM17] S. Bowe, A. Gabizon, and I. Miers. Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model. IACR Cryptology ePrint Archive, ia.cr/2017/1050. 2017. See also https://github.com/ebfull/powersoftau (Cited on p. 60).

[BGW88] M. Ben-Or, S. Goldwasser, and A. Wigderson. "Completeness Theorems for Non-Cryptographic Fault-Tolerant Distributed Computation."In: Proc. 20th Annual ACM Symposium on Theory of Computing. STOC '88. Pub. by Association for Computing Machinery, 1988, pp. 1–10. doi: 10.1145/62212.62213 (Cited on p. 31).

[BKS18] E. Ben-Sasson, S. Kopparty, and S. Saraf. "Worst-Case to Average Case Reductions for the Distance to a Code."In: Proc. 33rd Computational Complexity Conference. CCC '18. Pub. by Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018. Also at ECCC TR18/090 (Cited on p. 40).

[Blu87] M. Blum. "How to prove a theorem so no one else can claim it."In: Proc. International Congress of Mathematicians. 1987, pp. 1444–1451 (Cited on p. 37).

[BMRS20] J. Bonneau, I. Meckler, V. Rao, and E. Shapiro. Coda: Decentralized Cryptocurrency at Scale. IACR Cryptology ePrint Archive, ia.cr/2020/352. 2020 (Cited on p. 42).

[BPR19] L. Brandão, R. Peralta, and A. Robinson. NIST comments on the initial ZKProof documentation. https://csrc.nist.gov/Projects/pec/zkproof. Accessed in 2021. April 2019 (Cited on p. 89).

[BTVW14] A. J. Blumberg, J. Thaler, V. Vu, and M. Walfish. Verifiable computation using mul- tiple provers. IACR Cryptology ePrint Archive, ia.cr/2014/846. 2014 (Cited on p. 33).

[CC06] H. Chen and R. Cramer. "Algebraic Geometric Secret Sharing Schemes and Secure Multi-Party Computations over Small Fields."In: Advances in Cryptology —CRYPTO 2006. Ed. by C. Dwork. Pub. by Springer Berlin Heidelberg, 2006, pp. 521–536. doi: 10.1007/11818175_31 (Cited on p. 31).

[CCD88] D. Chaum, C. Crépeau, and I. Damgard. "Multiparty Unconditionally Secure Protocols." In: Proc. 20th Annual ACM Symposium on Theory of Computing. STOC '88. Pub. by Association for Computing Machinery, 1988, pp. 11–19. doi: 10.1145/62212.62214 (Cited on p. 31).

[CCHLRRW19] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, R. D. Rothblum, and D. Wichs. "Fiat-Shamir: From Practice to Theory."In: Proc. 51st Annual ACM SIGACT Symposium on Theory of Computing. STOC 2019. Pub. by ACM, 2019, pp. 1082–1090. doi: 10.1145/3313276.3316380 (Cited on p. 43).

[CD01] R. Cramer and I. Damgård. "Secure Distributed Linear Algebra in a Constant Number of Rounds."In: Advances in Cryptology —CRYPTO 2001. Ed. by J. Kilian. Pub. by Springer Berlin Heidelberg, 2001, pp. 119–136. doi: 10.1007/3-540-44647-8_7 (Cited on p. 31).

[CD98] R. Cramer and I. Damgård. “Zero-knowledge proofs for finite field arithmetic, or: Can zero-knowledge be for free?”In: Advances in Cryptology —CRYPTO ’98. Ed. by H. Krawczyk. Pub. by Springer Berlin Heidelberg, 1998, pp. 424–441. doi: 10.1007/BFb0055745 (Cited on p. 27).

[CDD17] J. Camenisch, M. Drijvers, and M. Dubovitskaya. “Practical UC-Secure Delegatable Credentials with Attributes and Their Application to Blockchain.”In: Proc. 2017 ACM SIGSAC

Conference on Computer and Communications Security. CCS ’17. Pub. by ACM, 2017, pp. 683–699. doi: 10.1145/3133956.3134025 (Cited on p. 67).

[CDIKLOV19] M. Chase, Y. Dodis, Y. Ishai, D. Kraschewski, T. Liu, R. Ostrovsky, and V. Vaikuntanathan. “Reusable Non-Interactive Secure Computation.”In: Advances in Cryptology —CRYPTO 2019. Ed. by A. Boldyreva and D. Micciancio. Pub. by Springer International Publishing, 2019, pp. 462–488. doi: 10.1007/978-3-030-26954-8_15. Also at ia.cr/2018/940 (Cited on p. 27).

[CDIKMRR13] G. Cohen, I. B. Damgård, Y. Ishai, J. Kölker, P. B. Miltersen, R. Raz, and R. D. Rothblum. “Efficient Multiparty Protocols via Log-Depth Threshold Formulae.”In: Advances in Cryptology —CRYPTO 2013. Ed. by R. Canetti and J. A. Garay. Pub. by Springer Berlin Heidelberg, 2013, pp. 185–202. doi: 10.1007/978-3-642-40084-1_11. Also at ia.cr/2013/480 (Cited on p. 31).

[CFIK03] R. Cramer, S. Fehr, Y. Ishai, and E. Kushilevitz. “Efficient Multi-party Computation over Rings.”In: Advances in Cryptology —EUROCRYPT 2003. Ed. by E. Biham. Pub. by Springer Berlin Heidelberg, 2003, pp. 596–613. doi: 10.1007/3- 540- 39200- 9_37. Also at ia.cr/2003/030 (Cited on p. 31).

[CGH04] R. Canetti, O. Goldreich, and S. Halevi. “The Random Oracle Methodology, Revisited.”In: J. ACM 51.4 (July 2004), pp. 557–594. doi: 10.1145/1008731.1008734. Also at ia.cr/1998/011 (Cited on p. 37).

[CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS.”In: Advances in Cryptology —EUROCRYPT 2020. Ed. by A. Canteaut and Y. Ishai. Pub. by Springer International Publishing, 2020, pp. 738–768. doi: 10.1007/978-3-030-45721-1_26. Also at ia.cr/2019/1047 (Cited on pp. 33, 41).

[CKS10] J. Camenisch, M. Kohlweiss, and C. Soriente. “Solving Revocation with Efficient Update of Anonymous Credentials.”In: Security and Cryptography for Networks. Ed. by J. A. Garay and R. De Prisco. Pub. by Springer Berlin Heidelberg, 2010, pp. 454–471. doi: 10.1007/978- 3- 642- 15317-4_28 (Cited on p. 67).

[CL18] R. Canetti and A. Lichtenberg. “Certifying Trapdoor Permutations, Revisited.”In: Theory of Cryptography. Ed. by A. Beimel and S. Dziembowski. TCC 2018. Pub. by Springer International Publishing, 2018, pp. 476–506. doi: 10.1007/978-3-030-03807-6_18. Also at ia.cr/2017/631 (Cited on p. 37).

[CMS19] A. Chiesa, P. Manohar, and N. Spooner. “Succinct Arguments in the Quantum Random Oracle Model.”In: Theory of Cryptography. Ed. by D. Hofheinz and A. Rosen. TCC 2019. Pub. by Springer International Publishing, 2019, pp. 1–29. doi: 10.1007/978-3-030-36033-7_1. Also at ia.cr/2019/834 (Cited on p. 39).

[CMT12] G. Cormode, M. Mitzenmacher, and J. Thaler. “Practical Verified Computation with Streaming Interactive Proofs.”In: Proc. 3rd Innovations in Theoretical Computer Science Conference. ITCS ’12. Pub. by Association for Computing Machinery, 2012, pp. 90–112. doi: 10.1145/2090236.2090245. Also at arXiv:cs/1105.2003 (Cited on p. 33).

[COS20] A. Chiesa, D. Ojha, and N. Spooner. “Fractal: Post-quantum and Transparent Recursive Proofs from Holography.”In: Advances in Cryptology —EUROCRYPT 2020. Ed. by A. Canteaut

and Y. Ishai. Pub. by Springer International Publishing, 2020, pp. 769–793. doi: 10.1007/978-3- 030-45721-1_27. Also at ia.cr/2019/1076 (Cited on pp. 33, 40, 42).

[CT10] A. Chiesa and E. Tromer. “Proof-Carrying Data and Hearsay Arguments from Signature Cards.”In: 1st Conference on Innovations in Computer Science —ICS 2010. Vol. 10. 2010, pp. 310–331. Also at https://projects.csail.mit.edu/pcd/ (Cited on pp. 42, 63, 65).

[CTV15] A. Chiesa, E. Tromer, and M. Virza. “Cluster Computing in Zero Knowledge.”In: Advances in Cryptology —EUROCRYPT 2015. Ed. by E. Oswald and M. Fischlin. Pub. by Springer

Berlin Heidelberg, 2015, pp. 371–403. doi: 10.1007/978-3-662-46803-6_13. Also at ia.cr/2015/377 (Cited on p. 42).

[Dam10] I. Damgård. On Σ-protocols. CPT 2010, v.2. https://www.cs.au.dk/~ivan/Sigma.pdf. 2010 (Cited on p. 37).

[DFKP16] A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, and B. Parno. "Cinderella: Turning Shabby X.509 Certificates into Elegant Anonymous Credentials with the Magic of Verifiable Computation."In: 2016 IEEE Symposium on Security and Privacy (SP). May 2016, pp. 235–254. doi: 10.1109/SP.2016.22 (Cited on p. 56).

[ECFR] ECFR. Code of Federal Regulations. eCFR –Title 17, Chapter II, § 230.500. https://www.ecfr.gov/current/title-17/section-230.501. Accessed in 2018 (Cited on p. 77).

[EFKP20] N. Ephraim, C. Freitag, I. Komargodski, and R. Pass. "SPARKs: Succinct Parallelizable Arguments of Knowledge."In: Advances in Cryptology —EUROCRYPT 2020. Ed. by A. Canteaut and Y. Ishai. Pub. by Springer International Publishing, 2020, pp. 707–737. doi: 10.1007/978-3- 030-45721-1_25. Also at ia.cr/2020/994 (Cited on p. 35).

[FLS99] U. Feige, D. Lapidot, and A. Shamir. "Multiple NonInteractive Zero Knowledge Proofs Under General Assumptions."In: SIAM Journal on Computing 29.1 (1999), pp. 1–28. doi: 10.1137/S0097539792230010 (Cited on p. 37).

[FS11] L. Fortnow and R. Santhanam. "Infeasibility of instance compression and succinct PCPs for NP."In: Journal of Computer and System Sciences 77.1 (2011). Celebrating Karp's Kyoto Prize, pp. 91–106. doi: https://doi.org/10.1016/j.jcss.2010.06.007. Also at ECCC TR07/096 (Cited on p. 31).

[FS87] A. Fiat and A. Shamir. "How To Prove Yourself: Practical Solutions to Identification and Signature Problems."In: Advances in Cryptology —CRYPTO'86. Ed. by A. M. Odlyzko. Vol. 263. LNCS. Pub. by Springer Berlin Heidelberg, 1987, pp. 186–194. doi: 10.1007/3- 540- 47721- 7_12 (Cited on p. 37).

[Gen09] C. Gentry. "A Fully Homomorphic Encryption Scheme."AAI3382729. PhD thesis. 2009. doi: 10.5555/1834954 (Cited on p. 38).

[GGPR13] R. Gennaro, C. Gentry, B. Parno, and M. Raykova. "Quadratic Span Programs and Succinct NIZKs without PCPs."In: Advances in Cryptology —EUROCRYPT 2013. Ed. by T. Johansson and P. Q. Nguyen. Pub. by Springer Berlin Heidelberg, 2013, pp. 626–645. doi: 10.1007/ 978-3-642-38348-9_37. Also at ia.cr/2012/215 (Cited on pp. 28, 38, 39, 43, 50).

[GH97] O. Goldreich and J. Håstad. On the Complexity of Interactive Proof with Bounded Communication. Tech. rep. April 1997. Also at ECCC TR96/018 (Cited on p. 37).

[GK96] O. Goldreich and A. Kahan. "How to Construct Constant-Round Zero-Knowledge Proof Systems for NP."In: J. Cryptol. 9.3 (June 1996), pp. 167–189. doi: 10.1007/BF00208001 (Cited on p. 36).

[GKR08] S. Goldwasser, Y. Kalai, and G. Rothblum. "Delegating Computation: Interactive Proofs for Muggles."In: vol. 62. May 2008, pp. 113–122. doi: 10.1145/1374376.1374396. Also at Journal of the ACM, Vol. 62, Issue 4, No. 27, 2015, DOI:10.1145/2699436. Also at ECCC TR07/108. (Cited on pp. 32–34).

[GMR89] S. Goldwasser, S. Micali, and C. Rackoff. "The Knowledge Complexity of Interactive Proof Systems."In: SIAM Journal on Computing 18.1 (1989), pp. 186–208. doi: 10.1137/0218012. Also later at: SIAM Journal on Computing, Vol. 18, Issue 1, ACM, 1989, doi:110.1137/0218012 (Cited on p. 1).

[GMW91] O. Goldreich, S. Micali, and A. Wigderson. "Proofs That Yield Nothing but Their Validity or All Languages in NP Have Zero-knowledge Proof Systems."In: J. ACM 38.3 (July 1991), pp. 690–728. doi: 10.1145/116825.116852 (Cited on pp. 7, 24).

[Gol01] O. Goldreich. The Foundations of Cryptography —Volume 1, Basic Techniques. https://www.wisdom.weizmann.ac.il/~oded/foc-vol1.html. June 2001 (Cited on p. 36).

[Gol13] O. Goldreich. "A Short Tutorial of Zero-Knowledge."In: Secure Multi-Party Computation. Ed. by M. M. Prabhakaran and A. Sahai. Vol. 10. Cryptology and Information Security Series. 2013, pp. 28–60. doi: 10.3233/978-1-61499-169-4-28 (Cited on p. 44).

[Gol18] O. Goldreich. “On Doubly-Efficient Interactive Proof Systems.”In: 13.3 (2018), pp. 158–246. doi: 10.1561/0400000084 (Cited on p. 33).

[GOS06] J. Groth, R. Ostrovsky, and A. Sahai. “Perfect Non-interactive Zero Knowledge for NP.”In: Advances in Cryptology —EUROCRYPT 2006. Ed. by S. Vaudenay. LNC. Pub. by Springer Berlin Heidelberg, 2006, pp. 339–358. doi: 10.1007/11761679_21. Also at ia.cr/2005/290 (Cited on p. 44).

[Gro10] J. Groth. “Short Pairing-Based Non-interactive Zero-Knowledge Arguments.”In: Ad- vances in Cryptology —ASIACRYPT 2010. Ed. by M. Abe. Pub. by Springer Berlin Heidelberg, 2010, pp. 321–340. doi: 10.1007/978-3-642-17373-8_19 (Cited on pp. 38, 39).

[Gro16] J. Groth. “On the Size of Pairing-Based Non-interactive Arguments.”In: Advances in Cryptology —EUROCRYPT 2016. Ed. by M. Fischlin and J.-S. Coron. Pub. by Springer Berlin Heidelberg, 2016, pp. 305–326. doi: 10.1007/978-3-662-49896-5_11. Also at ia.cr/2016/260 (Cited on pp. 39, 43, 59, 60).

[GVW02] O. Goldreich, S. Vadhan, and A. Wigderson. “On Interactive Proofs with a Laconic Prover.” In: Comput. Complex. 11.1/2 (June 2002), pp. 1–53. doi: 10.1007/s00037- 002- 0169- 0 (Cited on p. 37).

[HGBNL21] D. Hopwood, J. Grigg, S. Bowe, K. Nuttycombe, and Y. T. Lai. Zcash Improvement Proposal 224: Orchard Shielded Protocol. https://zips.z.cash/zip-0224. 2021 (Cited on p. 42).

[Hop18] D. Hopwood. ZCon0 Circuit Optimisation handout. https://github.com/ZcashFounda tion/ zcon0-workshop-notes. 2018 (Cited on p. 49).

[IKO07] Y. Ishai, E. Kushilevitz, and R. Ostrovsky. “Efficient Arguments without Short PCPs.”In: Twenty-Second Annual IEEE Conference on Computational Complexity (CCC’07). 2007, pp. 278–291. doi: 10.1109/CCC.2007.10 (Cited on p. 26).

[IKOS09] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. “Zero-Knowledge Proofs from Secure Multiparty Computation.”In: SIAM Journal on Computing 39.3 (2009), pp. 1121–1152. doi: 10.1137/080725398. See also “ZK from SMPC”at Proc. STOC ’07, DOI:10.1145/1250790.1250794 (Cited on pp. 30, 31).

[IMSX15] Y. Ishai, M. Mahmood, A. Sahai, and D. Xiao. On Zero-Knowledge PCPs: Limitations, Simplifications, and Applications. 2015 (Cited on p. 37).

[Ish20] Y. Ishai. Zero-Knowledge Proofs from Information-Theoretic Proof Systems. Parts I and II. In: The Art of Zero Knowledge (ZKProof Blog) —https : / / zkproof . org / 2020 / 08 / 12 / information-theoretic-proof-systems. 2020 (Cited on p. 89).

[JSI96] M. Jakobsson, K. Sako, and R. Impagliazzo. “Designated Verifier Proofs and Their Applications.”In: Advances in Cryptology —EUROCRYPT ’96. Ed. by U. Maurer. Pub. by Springer Berlin Heidelberg, 1996, pp. 143–154. doi: 10.1007/3-540-68339-9_13 (Cited on p. 66).

[Kil92] J. Kilian. “A Note on Efficient Zero-Knowledge Proofs and Arguments (Extended Ab- stract).” In: Proc. 24th Annual ACM Symposium on Theory of Computing. STOC ’92. Pub. by Association for Computing Machinery, 1992, pp. 723–732. doi: 10.1145/129712.129782 (Cited on pp. 25, 37).

[Kil95] J. Kilian. “Improved Efficient Arguments.”In: Advances in Cryptology —CRYPTO’95. Ed. by D. Coppersmith. Vol. 1070. LNCS. Pub. by Springer Berlin Heidelberg, 1995, pp. 311–324. doi: 10.1007/3-540-44750-4_25 (Cited on p. 43).

[KKW18] J. Katz, V. Kolesnikov, and X. Wang. “Improved Non-Interactive Zero Knowledge with Applications to Post-Quantum Signatures.”In: Proc. 2018 ACM SIGSAC Conference on Computer and Communications Security. CCS ’18. Pub. by Association for Computing Machinery, 2018, pp. 525–537. doi: 10.1145/3243734.3243805. Also at ia.cr/2018/475 (Cited on p. 31).

[Kla03] H. Klauck. “Rectangle size bounds and threshold covers in communication complexity.”In: Proc. 18th IEEE Annual Conference on Computational Complexity. 2003, pp. 118–134. doi: 10.1109/ CCC.2003.1214415. Also at arXiv:cs/0208006 (Cited on p. 34).

[KMSWP16] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. “Hawk: The Blockchain Model of Cryptography and Privacy-Preserving Smart Contracts.”In: 2016 IEEE Symposium on Security and Privacy (SP). May 2016, pp. 839–858. doi: 10.1109/SP.2016.55 (Cited on p. 67).

[KPV19] A. Kattis, K. Panarin, and A. Vlasov. RedShift: Transparent SNARKs from List Poly- nomial Commitment IOPs. IACR Cryptology ePrint Archive, ia.cr/2019/1400. 2019 (Cited on p. 41).

[KR08] Y. T. Kalai and R. Raz. “Interactive PCP.”In: Proc. 35th International Colloquium on Automata, Languages and Programming, Part II. Ed. by L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfsdóttir, and I. Walukiewicz. ICALP ’08. Pub. by Springer-Verlag, 2008, pp. 536–547. doi: 10.1007/978-3-540-70583-3_44. Also at ECCC TR07/031 (Cited on p. 31).

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polyno- mials and Their Applications.”In: Advances in Cryptology —ASIACRYPT 2010. Ed. by M. Abe. Pub. by Springer Berlin Heidelberg, 2010, pp. 177–194. doi: 10.1007/978-3-642-17373-8_11 (Cited on p. 41).

[Lee20] J. Lee. Dory: Efficient, Transparent arguments for Generalised Inner Products and Polynomial Commitments. IACR Cryptology ePrint Archive, ia.cr/2020/1274. 2020 (Cited on p. 41).

[LFKN92] C. Lund, L. Fortnow, H. Karloff, and N. Nisan. “Algebraic Methods for Interactive Proof Systems.”In: J. ACM 39.4 (October 1992), pp. 859–868. doi: 10.1145/146585.146605 (Cited on pp. 32, 33).

[libsnark] libsnark. libsnark: a C++ library for zkSNARK proofs. https : / / github . com / scipr - lab/ libsnark. Accessed in 2022 (Cited on p. 28).

[Lin20] Y. Lindell. “Secure Multiparty Computation.”In: Commun. ACM 64.1 (December 2020), pp. 86–96. doi: 10.1145/3387108 (Cited on p. 30).

[Lip12] H. Lipmaa. “Progression-Free Sets and Sublinear Pairing-Based Non-Interactive Zero-Knowledge Arguments.”In: Theory of Cryptography. Ed. by R. Cramer. Pub. by Springer Berlin Heidelberg, 2012, pp. 169–189. doi: 10.1007/978-3-642-28914-9_10. Also at ia.cr/2011/009 (Cited on p. 38).

[LSTW21] J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby. “Linear-time zero-knowledge SNARKs for R1CS.”In: Cryptology ePrint Archive, Report ia.cr/2021/030 (2021) (Cited on p. 41).

[Mau06] U. Maurer. “Secure multi-party computation made simple.”In: Discrete Applied Mathematics 154.2 (2006). Coding and Cryptography, pp. 370–381. doi: https:// doi.org/ 10.1016/ j.dam. 2005.03.020 (Cited on p. 31).

[Mic00] S. Micali. “Computationally Sound Proofs.”In: SIAM J. Comput. 30.4 (October 2000), pp. 1253–1298. doi: 10.1137/S0097539795284959 (Cited on p. 25).

[Mik19] Mikelodder7/Ursa. Z-mix. 2019. https://github.com/mikelodder7/ursa/tree/master/libzmix (Cited on p. 67).

[Mos10] D. Moshkovitz. An Alternative Proof of The Schwartz-Zippel Lemma. TR10-096. 2010 (Cited on p. 27).

[Nao03] M. Naor. “On cryptographic assumptions and challenges.”In: Advances in Cryptology — CRYPTO 2003. Ed. by D. Boneh. Vol. 2729. Lecture Notes in Computer Science. Pub. by Springer, Berlin, Heidelberg, 2003. doi: 10.1007/978-3-540-45146-4_6 (Cited on p. 38).

[NVV18] N. Narula, W. Vasquez, and M. Virza. “zkLedger: Privacy-Preserving Auditing for Distributed Ledgers.”In: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). Pub. by USENIX Association, 2018, pp. 65–80. Also at ia.cr/2018/241 (Cited on p. 67).

[PHGR13] B. Parno, J. Howell, C. Gentry, and M. Raykova. “Pinocchio: Nearly Practical Veri- fiable Computation.”In: IEEE Symposium on Security and Privacy. May 2013, pp. 238–252. doi: 10.1109/ SP.2013.47. Also at ia.cr/2013/279 (Cited on pp. 28, 50, 51, 59).

[RR20] N. Ron-Zewi and R. D. Rothblum. “Local Proofs Approaching the Witness Length [Ex- tended Abstract].”In: 2020 IEEE 61st Annual Symposium on Foundations of Computer Science

(FOCS). 2020, pp. 846–857. doi: 10.1109/FOCS46700.2020.00083. Also at ia.cr/2019/1062 (Cited on p. 40).

[RRR16] O. Reingold, G. N. Rothblum, and R. D. Rothblum. “Constant-round Interactive Proofs for Delegating Computation.”In: Proc. 48th Annual ACM Symposium on Theory of Computing. STOC ’ 16. Pub. by ACM, 2016, pp. 49–62. doi: 10.1145/2897518.2897652. Also at SIAM Journal on Computing (50-3, 2021), doi:10.1137/16M1096773. Also at ECCC TR16/061 (Cited on pp. 32–34, 42).

[RTV04] O. Reingold, L. Trevisan, and S. Vadhan. “Notions of Reducibility between Crypto- graphic Primitives.”In: Theory of Cryptography. Ed. by M. Naor. Pub. by Springer Berlin Heidel- berg, 2004, pp. 1–20. doi: 10.1007/978-3-540-24638-1_1 (Cited on p. 37).

[Sch90] C. P. Schnorr. “Efficient Identification and Signatures for Smart Cards.”In: Advances in Cryptology —EUROCRYPT ’89. Ed. by J.-J. Quisquater and J. Vandewalle. Vol. 434. LNCS. Pub. by Springer Berlin Heidelberg, 1990, pp. 688–689. doi: 10.1007/3-540-46885-4_68 (Cited on p. 6).

[Set20] S. Setty. “Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup.”In: Advances in Cryptology —CRYPTO 2020. Ed. by D. Micciancio and T. Ristenpart. Pub. by Springer International Publishing, 2020, pp. 704–737. doi: 10.1007/978-3-030-56877-1_25. Also at ia.cr/ 2019/550 (Cited on pp. 33, 41).

[SL20] S. T. V. Setty and J. Lee. Quarks: Quadruple-efficient transparent zkSNARKs. IACR Cryptology ePrint Archive, ia.cr/2020/1275. 2020 (Cited on p. 41).

[Sov18] F. Sovrin. SovrinTM : A Protocol and Token for Self-Sovereign Identity and Decentralized Trust. January 2018. https: // sovrin .org / wp- content/ uploads/ 2018 /03 / Sovrin- Protocol- and-Token-White-Paper.pdf (Cited on p. 67).

[Spi96] D. A. Spielman. “Linear-time encodable and decodable error-correcting codes.”In: IEEE Transactions on Information Theory 42.6 (1996), pp. 1723–1731. doi: 10.1109/18.556668 (Cited on p. 35).

[Tha13] J. Thaler. “Time-Optimal Interactive Proofs for Circuit Evaluation.”In: Advances in Cryptology —CRYPTO 2013. Ed. by R. Canetti and J. A. Garay. Pub. by Springer Berlin Hei- delberg, 2013, pp. 71–89. doi: 10.1007/978- 3- 642- 40084- 1_5. Also at ia.cr/2013/351 (Cited on p. 33).

[Tha20] J. Thale. Blog series: In The Art of Zero Knowledge. Ed. by ZKProof. https://zkproof. org/ 2020/03/16/sum-checkprotocol. March 2020 (Cited on p. 32).

[Val08] P. Valiant. “Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/ Space Efficiency.”In: 5th Theory of Cryptography Conference. Ed. by R. Canetti. TCC ’08. Pub. by Springer Berlin Heidelberg, 2008. doi: 10.1007/978-3-540-78524-8_1 (Cited on p. 42).

[VSBW13] V. Vu, S. Setty, A. J. Blumberg, and M. Walfish. “A Hybrid Architecture for Interac- tive Verifiable Computation.”In: 2013 IEEE Symposium on Security and Privacy. 2013, pp. 223–237. doi: 10.1109/SP.2013.48 (Cited on p. 33).

[WJBSTWW17] R. S. Wahby, Y. Ji, A. J. Blumberg, A. Shelat, J. Thaler, M. Walfish, and T. Wies. “Full Accounting for Verifiable Outsourcing.”In: Proc. 2017 ACM SIGSAC Conference on

Computer and Communications Security. CCS ’17. Pub. by Association for Computing Machinery, 2017, pp. 2071–2086. doi: 10.1145/3133956.3133984. Also at ia.cr/2017/242 (Cited on p. 33).

[WTTW18] R. S. Wahby, I. Tzialla, a. shelat abhi, J. Thaler, and M. Walfish. “Doubly-efficient zkSNARKs without trusted setup.”In: 2018 IEEE Symposium on Security and Privacy (SP). IEEE. 2018, pp. 926–943. doi: 10.1109/SP.2018.00060. Also at ia.cr/2017/1132 (Cited on pp. 33, 41, 43).

[WZCPS18] H. Wu, W. Zheng, A. Chiesa, R. A. Popa, and I. Stoica. “DIZK: A Distributed Zero Knowledge Proof System.”In: 27th USENIX Security Symposium (USENIX Security 18). Pub. by USENIX Association, August 2018, pp. 675–692. Also at ia.cr/2018/691 (Cited on p. 34).

[XZZPS19] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song. “Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation.”In: Advances in Cryptology —CRYPTO 2019 Proceedings, Part III. Ed. by A. Boldyreva and D. Micciancio. Pub. by Springer International Publishing, 2019, pp. 733–764. doi: 10.1007/978-3-030-26954-8_24. Also at ia.cr/2019/317 (Cited on pp. 33, 41, 43).

[zca18] zcash-hackworks/babyzoe. Baby ZoE —first step towards Zerocash over Ethereum. 2018. https://github.com/zcash-hackworks/babyzoe (Cited on p. 67).

[ZGKPP17] Y. Zhang, D. Genkin, J. Katz, D. Papadopoulos, and C. Papamanthou. “vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases.”In: 2017 IEEE Symposium on Security and Privacy (SP). May 2017, pp. 863–880. doi: 10.1109/SP.2017.43. Also at ia.cr/ 2017/1145 (Cited on pp. 33, 41).

[ZKP-CC20] ZKProof. Call for contributions to the ZKProof Community Reference —Review cy- cle 2020: from version 0.2 to 0.3. Ed. by Z. Editors. https://github.com/zkpstandard/zkreference/ blob/ master/Call-2020-for-contribs-to-ZCRef.pdf. August 2020 (Cited on p. 90).

[ZKP-FF] ZKProof. ZKProof file formats. https://github.com/zkpstandard/file_formats. Ac- cessed in 2022 (Cited on p. 51).

[ZKP.Science] ZKP.Science. Zero-Knowledge Proofs —What are they, how do they work, and are they fast yet? https://zkp.science. Sourced from https://github.com/ZKProofs/ZKProofs.github.io. Accessed in 2022 (Cited on pp. 48, 49).

[ZKP19] ZKProof. Notes of the 2nd ZKProof Workshop. Ed. by D. Benarroch, L. T. A. N. Brandão, and E. Tromer. Pub. by zkproof.org, December 2019. (Workshop held in Berkeley, USA, in April 2019) (Cited on p. 89).

[ZLWZSXZ21] J. Zhang, T. Liu, W. Wang, Y. Zhang, D. Song, X. Xie, and Y. Zhang. "Doubly Efficient Interactive Proofs for General Arithmetic Circuits with Linear Prover Time."In: Proc. 2021 ACM SIGSAC Conference on Computer and Communications Security. CCS '21. Pub. by Association for Computing Machinery, 2021, pp. 159–177. doi: 10.1145/3460120.3484767. Also at ia.cr/2020/1247 (Cited on p. 33).

[ZXZS20] J. Zhang, T. Xie, Y. Zhang, and D. Song. "Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof."In: 2020 IEEE Symposium on Security and Privacy (SP). 2020, pp. 859–876. doi: 10.1109/SP40000.2020.00052. Also at ia.cr/2019/1482 (Cited on pp. 33, 41).

Page intentionally blank

付録 A. 略語および用語集

A.1 略語

- **3SAT:** 3-satisfiability（3 充足可能性）
- **AND:** AND gate（ブールゲート）
- **API:** application program interface（アプリケーション・プログラム・インターフェース）
- **CRH:** collision-resistant hash function（衝突困難ハッシュ関数）
- **CRS:** common-reference string（共通参照文字列）
- **DAG:** directed acyclic graph（有向非巡回グラフ）
- **DSL:** domain specific languages（ドメイン固有言語）
- **FFT:** fast-Fourier transform（高速フーリエ変換）
- **ILC:** ideal linear commitment
- **IOP:** interactive oracle proofs
- **LIP:** linear interactive proofs
- **MA:** Merlin-Arthur
- **NIZK:** non-interactive zero-knowledge（非対話型ゼロ知識）
- **NP:** non-deterministic polynomial（非決定性多項式時間）
- **PCD:** proof-carrying data
- **PCP:** probabilistic checkable proof
- **PKI:** public-key infrastructure（公開鍵基盤）
- **QAP:** quadratic arithmetic program
- **R1CS:** rank-1 constraint system
- **RAM:** random access memory
- **RSA:** Rivest-Shamir-Adleman
- **SHA:** secure hash algorithm
- **SMPC:** secure multiparty computation
- **SNARG:** succinct non-interactive argument
- **SNARK:** SNARG of knowledge
- **SRS:** structured reference string（構造化参照文字列）
- **UC:** universal composability / universally composable（ユニバーサル合成可能性／合成可能）
- **URS:** uniform random string（一様ランダム文字列）
- **XOR:** exclusive OR（ブールゲート）
- **ZK:** zero knowledge（ゼロ知識）
- **ZKP:** zero-knowledge proof（ゼロ知識証明）

A.2 用語集

- **NIZK:** Non-Interactive Zero-Knowledge（非対話型ゼロ知識）。証明者が検証者へ単一メッセージを送り、検証者がその後、受理または拒否を決定する証明システム。通常は共通参照文字列モデルで設定されるが、指定検証者 NIZK 証明も可能である。
- **SNARK:** Succinct Non-interactive Argument of Knowledge（簡潔な非対話型知識アーギュメント）。証明サイズが小さく、検証が高速である特殊な種類の非対話型証明システム。
- **インスタンス:** 証明者と検証者の双方に知られる公開入力。記法: `x`。一部の科学論文では「インスタンス」と「命題」を同義に用いるが、本書では両者を区別する。
- **証人:** 証明者への秘密入力。他者が証人について何かを知っている場合もあれば、知らない場合もある。記法: `w`。
- **アプリケーション入力:** 外部データソースから来る、応用への入力として解釈される証人の一部。完全な証人とインスタンスは、アプリケーション入力から証明者が計算できる。
- **関係:** インスタンスと証人の関係の仕様。関係は、許容されるペア `(instance, witness)` の集合として見ることができる。記法: `R`。
- **言語:** `R` において証人を持つインスタンスの集合。記法: `L`。
- **命題:** インスタンスと関係によって定義される。インスタンスが関係内に証人を持つことを主張し、これは真または偽である。記法: `x ∈ L`。
- **制約系:** 関係を指定するための言語。
- **証明システム:** ゼロ知識証明システムとは、命題が真であることを証明者が検証者に納得させるため、証明者と検証者がどのように相互作用できるかの仕様である。証明システムは完全性、健全性、ゼロ知識性を満たさなければならない。
  - **完全性:** 命題が真であり、証明者と検証者がプロトコルに従うなら、検証者は受理する。
  - **健全性:** 命題が偽であり、検証者がプロトコルに従うなら、検証者は納得しない。

  - **ゼロ知識性:** 命題が真であり、証明者がプロトコルに従うなら、検証者は証明者との相互作用から、命題が真であるという事実以外の機密情報を学ばない。
- **バックエンド:** ZK 証明システムの低水準暗号プロトコルの実装。
- **フロントエンド:** ZK 命題を便利な言語で表現し、それを低水準表現へコンパイルして適切な ZK バックエンドを呼び出すことで、ゼロ知識で証明する手段。
- **インスタンス還元:** フロントエンドによって、高水準命題のインスタンスを、バックエンドが消費するのに適した低水準命題のインスタンスへ変換すること。
- **証人還元:** フロントエンドによって、高水準命題の証人を、バックエンドが消費するのに適した低水準命題の証人へ変換すること。
- **R1CS (Rank 1 Constraint Systems):** 関係を双線形制約系、すなわち rank 1 quadratic constraint system として指定するための NP 完全言語。QAP のより直観的な再定式化である。
- **QAP (Quadratic Arithmetic Program):** 多項式の二次系を通じて関係を指定するための NP 完全言語。同値な定式化については R1CS を参照。

参照文字列:

- **CRS (Common Reference String):** NIZK の Generator アルゴリズムによって出力され、証明者と検証者の双方が利用できる文字列。証明パラメータと検証パラメータからなる。URS または SRS であり得る。
- **URS (Uniform Random String):** 何らかの空間から一様にサンプリングすることで作成される共通参照文字列。とくに作成過程で秘密を含まない。先行文献では Common Random String とも呼ばれるが、Common Reference String との頭字語衝突を避けるため本書ではこの語を避ける。
- **SRS (Structured Reference String):** 複雑な分布からサンプリングして作成される共通参照文字列。多くの場合、内部乱数を持つサンプリング・アルゴリズムを伴い、その内部乱数は明らかにされてはならない。なぜなら、それは偽の命題に対して説得力のある証明を作成できるトラップドアを生むからである。SRS は非ユニバーサル（特定関係に依存）でもユニバーサル（関係から独立、すなわち NP 全体の証明に使用可能）でもあり得る。
- **PP (Prover Parameters) または Proving Key:** 共通参照文字列のうち、証明者が使用する部分。
- **VP (Verifier Parameters) または Verification Key:** 共通参照文字列のうち、検証者が使用する部分。

付録 B. バージョン履歴

ZKProof Community Reference (ZkpComRef) の開発は、主要バージョンの系列に従う。

- **Version 0 [2018-08-01]: ベースライン文書**。第 1 回 ZKProof Workshop（May 2018）の proceedings は、2018-08-01 までに確定した貢献とともに、ZKProof.org で利用可能であり、ZKProof Charter と合わせて ZKProof Community Reference の出発点を構成する。3 つの Workshop トラック、すなわち security、applications、implementation は、それぞれ“ZKProof Standards Track Proceedings”と題する proceedings 文書につながった。ZKProof 憲章もベースライン文書の一部である。
- **Version 0.1 [2019-04-11]: LaTeX/PDF コンパイル**。 Version 0 を構成する複数の proceedings は LaTeX コードへ移植され、第 2 回 ZKProof workshop での提示と議論のため、“ZKProof Community Reference”(version 0.1) と題する単一 PDF 文書へコンパイルされた。そのバージョンには、索引化の容易さと一貫したスタイルのための編集上の調整が含まれていた。
- **Version 0.2 [2019-12-31]: 統合草稿**。草稿コミュニティ参照文書を統合する過程は、第 2 回 ZKProof workshop（April 2019）で開始された。そこでは編集プロセスが導入され、「NIST comments on the initial ZKProof documentation」を含む焦点化された話題についていくつかの breakout sessions が開催された。議論から、新バージョンの文書に開発・組込みすべき話題の提案が得られた。その後、いくつかの具体的な「proposed contributions」項目が GitHub issue として定義された。続いて提出されたさまざまな貢献は、次のような内容改善をもたらした。知識の ZKP とメンバーシップの ZKP の区別、ベンチマークのためのセキュリティ・パラメータ推奨、ZKP システムに関連する用語（命題、CRS、R1CS など）の明確化、対話性と非対話性、移転可能性と否認可能性の議論、ユースケースと応用の範囲明確化、「gadgets」表の更新、新しい参考文献の追加。新バージョンにはまた、統合文書へ向けた多数の編集上の改善が含まれる。具体的には、いくつかの新しい節（要旨、このコミュニティ参照文書について、変更履歴、謝辞、知的財産、エグゼクティブ・サマリー）を持つ大幅に再構成された frontmatter、および構成パラダイムに関する新章（未完成）の追加を含む再編成された構造である。変更は文書の“diff”version で追跡されている。
- **Version 0.3 [2022-07-17]: パラダイム追加**。主な更新はパラダイム章におけるものであり、古い“Taxonomy of Constructions”節を 3 つの新節、すなわち“Background”(§2.1)、“Information-Theoretic (IT) Proof Systems”(§2.2)、“Cryptographic Compilers (CC)”(§2.3) で置き換えた。また、“Specialized ZK Proofs”(§2.4) と“Proof Composition”(§2.5) の新しいプレースホルダー節（貢献募集）も含まれた。その他の小さな変更には、1.2 節の表 1.1 の改善、およびエグゼクティブ・サマリーの新しい段落が含まれる。

ZkpComRef の開発履歴を扱う追加文書は、ZKProof GitHub repository (https://github.com/zkpstandard/zkreference)、ZKProof webpage (https://docs.zkproof.org/reference)、および forum (https://community.zkproof.org) で見つけることができる。